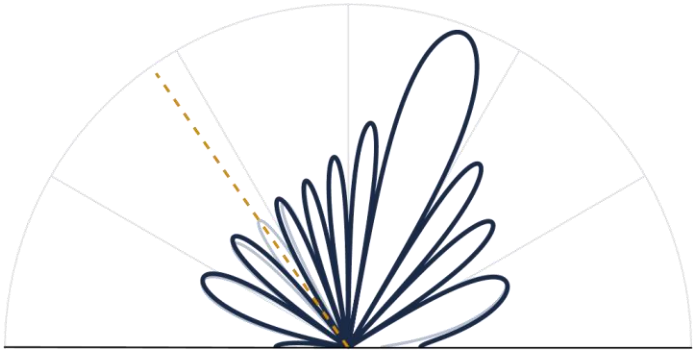
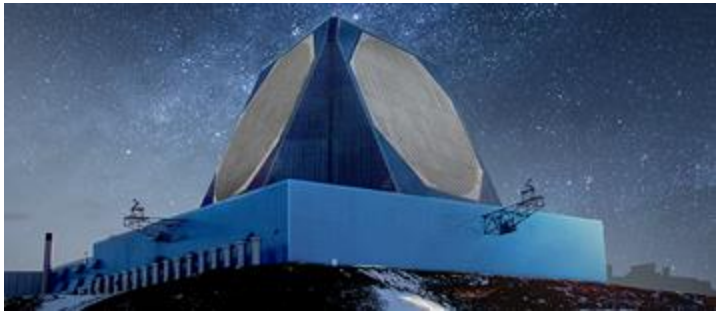
IEEE PAST 2026 · TUTORIAL

# Hands-On Adaptive Beamforming



From delay-and-sum to LMS, RLS, MVDR and LCMV — simulated in MATLAB, then run with 16-channel ADI data



**Mark Yeary** · University of Oklahoma
**Tarun Cousik** · Virginia Tech
**Jon Kraft** · Analog Devices
*with special guest appearance by **Jah'Shawn Ross** (Analog Devices)!*

ANALOG DEVICES
AHEAD OF WHAT'S POSSIBLE™

VIRGINIA TECH

The UNIVERSITY of OKLAHOMA

# Why is Adaptive Beamforming Important?

### Radar

The target echo travels out and back ($\propto 1/R^4$); a interferers only travels one way ($\propto 1/R^2$). The interferer can be tens of dB above the echo.

### Cellular

A user at the cell edge competes with nearby user equipment (low Tx power) and interferers that arrive tens of dB stronger (the near–far problem).

### Nobody tells you where

Interferers don't announce their direction, and they move. A null placed by hand is in the wrong place by the next look.

**Age Old Problem: Weak signals, Strong interferers, Unknown directions for interferers or Signals of Interest (SOIs)**

**How can we apply spatial-filters to reduce interferers, while maximizing our antenna gain in the direction we want?**

# What are We Covering Today?

| Fixed weights | | Adaptive: iterative (gradient-based) | | Adaptive: statistical (optimum) | |
|---|---|---|---|---|---|
| **Conventional** | **Tapering** | **LMS** | **RLS** | **MVDR / LCMV** | **SMI** |
| + Point at the signal of interest (SOI) and sum; SNR grows with the number of elements | + Shape the element amplitudes: sidelobes drop from −13 to −30 dB | + Learns the weights from data; nulls unknown interferers, given a reference signal | + Converges much faster, even with strong interferers | + The best achievable signal-to-interference ratio; LCMV adds extra beams and nulls | + Estimates that knowledge from a finite block of data: MVDR/LCMV in practice |
| – Fixed sidelobes let a strong interferer through | – Wider beam, less gain; sidelobes still jammable; nulls need the interferer's direction | – Slow to converge; choosing a step size; needs the reference | – More computation per sample; choosing a forgetting factor; numerical care | – Assumes perfect knowledge of the interference statistics; needs the SOI direction | – Needs ~2× as many samples as elements; a matrix inverse; an SOI in the training data cancels itself |

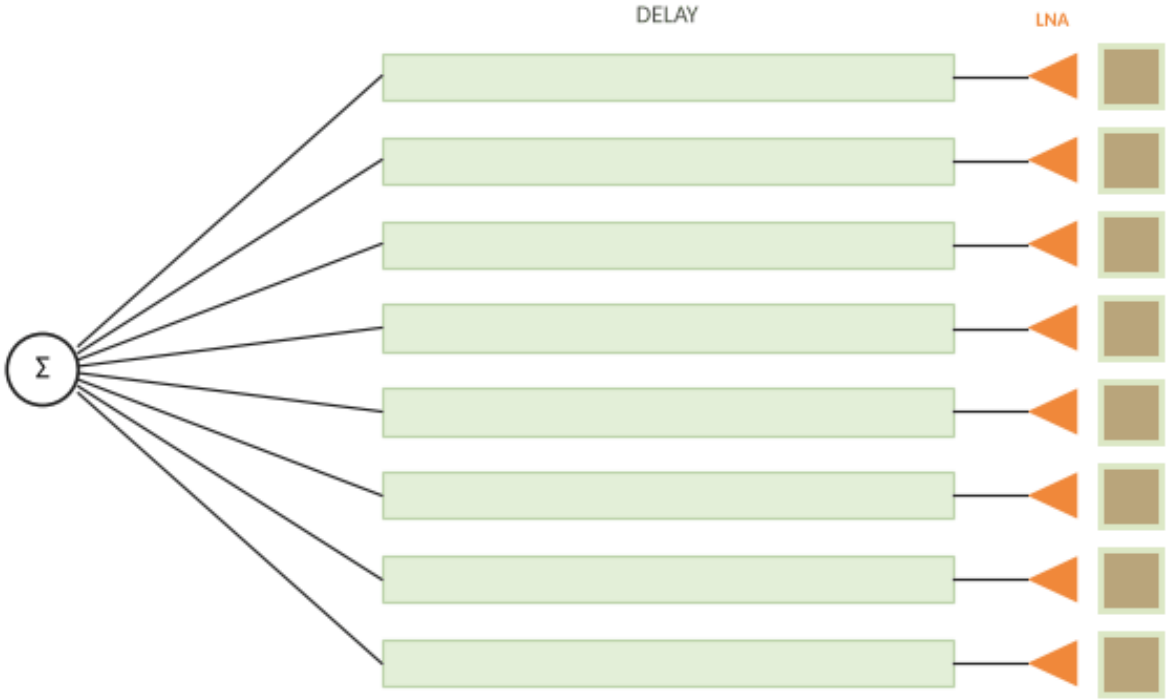
Problem Statement → Fundamentals → **Labs** → Deep Dive → Pros and Cons

# Module 1: Conventional Beamforming

Steering vectors, array patterns in 1-D and 2-D, and tapering

# Basics of phased arrays: delay and sum

**1 · A plane wave reaches each element at a different time**




RF source

# Understanding Steering Angle

There are 3 ways to describe this delay:

**1. An incremental distance to travel:**

$$L = d \sin \theta$$

**2. A time delay between elements:**

$$\Delta t = L / c = d \sin \theta / c$$

$$\rightarrow \quad \theta = \sin^{-1}(\Delta t\, c / d)$$

**3. A phase shift between elements:**

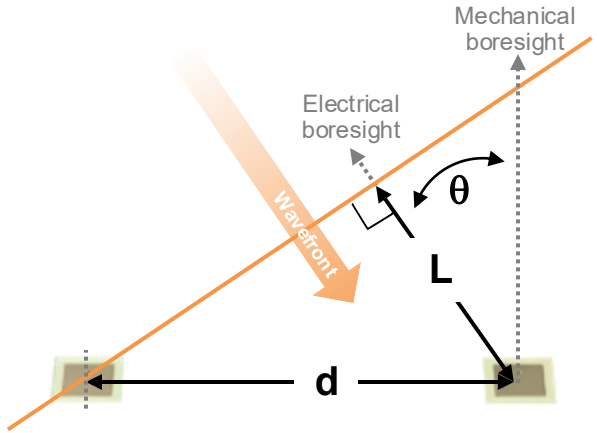
$$\Delta\psi = 2\pi L / \lambda = 2\pi f L / c$$

$$\rightarrow \Delta\psi = 2 \pi f d \sin \theta / c$$

$$\rightarrow \theta = \sin^{-1}(\Delta\psi\, c / (2 \pi f d))$$

Rewrite $\Delta\psi$ relative to wavelength

$$\Delta\psi = \frac{2\pi d sin\theta}{\lambda}$$




$\theta$ - Beam electrical angle
L - Incremental propagation distance between elements
d - Distance between elements
$\Delta t$ - Incremental time delay between elements
$\Delta\psi$ - Incremental phase shift between elements

# Steering Vector

Steering Angle is θ (30° for our example)

Steering Vector ($\boldsymbol{s}; \boldsymbol{s} \in \boldsymbol{C}^{N}; N = 8$) is the array vector that corresponds to that steering angle

- For a linear array of size $N = 8$, is $\boldsymbol{\Psi}$ a column vector (8 row, 1 column),
- Where each element, $s_n$, is: $s_n = a_n \cdot e^{-j\,\psi_n}$

$$\mathbf{s} = \begin{bmatrix} a_1 \cdot e^{-j\psi_1} \\ a_2 \cdot e^{-j\psi_2} \\ a_3 \cdot e^{-j\psi_3} \\ a_4 \cdot e^{-j\psi_4} \\ a_5 \cdot e^{-j\psi_5} \\ a_6 \cdot e^{-j\psi_6} \\ a_7 \cdot e^{-j\psi_7} \\ a_8 \cdot e^{-j\psi_8} \end{bmatrix} \rightarrow \boldsymbol{\Psi} \begin{bmatrix} 0° \\ 86° \\ 172° \\ 258° \\ 344° \\ 70° \\ 156° \\ 242° \end{bmatrix} \rightarrow \mathbf{s}\text{ (polar form)} \begin{bmatrix} 1 \cdot e^{-j\,0} \\ 1 \cdot e^{-j\,1.5} \\ 1 \cdot e^{-j\,3} \\ 1 \cdot e^{-j\,4.5} \\ 1 \cdot e^{-j\,6} \\ 1 \cdot e^{-j\,1.2} \\ 1 \cdot e^{-j\,2.7} \\ 1 \cdot e^{-j\,4.2} \end{bmatrix} \quad \text{(for } a_{1\text{-}8} = 1\text{)}$$

**or**

$$\mathbf{s}\text{ (rect form)} \begin{bmatrix} (1 + 0j) \\ (0.07 + 1j) \\ (-1 + 0.14j) \\ (-0.2 - 0.98j) \\ (0.96 - 0.28j) \\ (0.34 + 0.94j) \\ (-0.9 + 0.4j) \\ (-0.47 - 0.88j) \end{bmatrix} \quad \text{(for } a_{1\text{-}8} = 1\text{)}$$

*where* $\psi_n = (n-1)\,2\,\pi\,f\,d\,\sin\theta\,/\,c$

**Remember:**

Vector Notations:
- c : scalar
- $\mathbf{c}^{N}$: Nx1 column vector

Rectangular to Polar:
- a = sqrt($x^2 + y^2$)
- φ = arctan(y/x)

Polar to Rectangular:
- x = a cos(Δψ)
- y = a sin(Δψ)

# Beamweights (w)

- **w** is the vector of beamweights (gain and delay)
  - Applied to the array elements to steer the array
  - For N=8, linear array, $\boldsymbol{w}$; $\mathbf{w} \in \boldsymbol{C}^{N}; N = 8$ is a row vector (1 row, 8 column)

- In **conventional** beamforming, the **beamweights are equal** to the steering vector:

→ **w = s**

**Remember:**

Vector Notations:
- c : scalar
- $\mathbf{c}^{N}$: Nx1 column vector
- $\mathbf{c}^{H}$: Hermitian (complex conjugate transpose) of **c**
- $c_n^*$: complex conjugate of scaler $c_n$

- We sometimes alter those beamweights, but more on that later!

# Applying Beamformer Weights

- The received data, from each element, is $\mathbf{x}_n$
- We then apply the weights (**w**) to the received samples (**x**)

$$y = w_1^* x_1 + w_2^* x_2 + \ldots + w_N^* x_N$$

$$y = \begin{bmatrix} w_1^* & w_2^* & \ldots & w_N^* \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_N \end{bmatrix}$$

$$y = \mathbf{w}^H \mathbf{x}$$

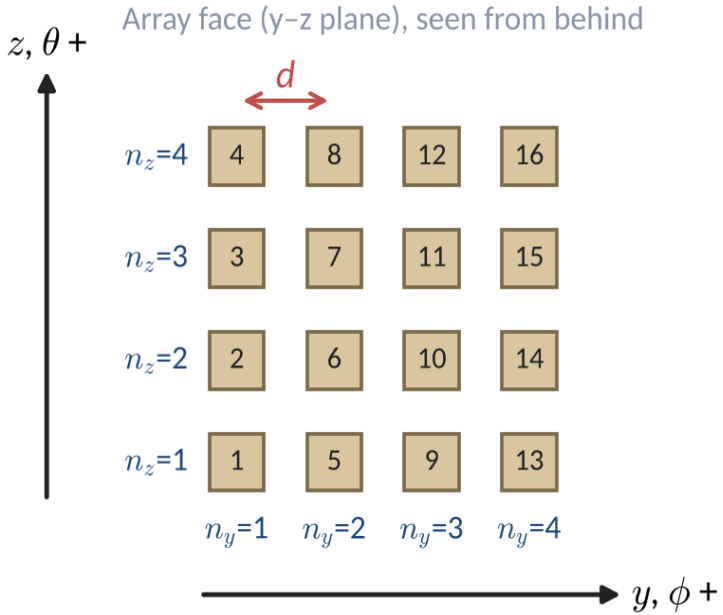
**Remember:**

Vector Notations:
- c : scalar
- **c** : vector
- $\mathbf{c}^N$: Nx1 column vector
- $\mathbf{c}^H$: Hermitian (complex conjugate transpose) of **c**
- $c_n^*$: complex conjugate of scaler $c_n$

**How are these weights applied?**

→ Let's talk about Analog vs Digital Beamformers

# From a line to a plane: the 2-D steering vector




**Direction cosines (u, v) replace the 1-D sin θ**

$$u = \cos\theta \sin\phi, \qquad v = \sin\theta$$

**One phase step per axis**

$$s_n(u,v) = e^{-j2\pi\frac{d}{\lambda}[(n_y-1)\,u+(n_z-1)\,v]}$$

element $n = (n_y - 1)N_z + n_z$, with $n_y = 1, \ldots, N_y$ and $n_z = 1, \ldots, N_z$

**Stack it: two line-array vectors**

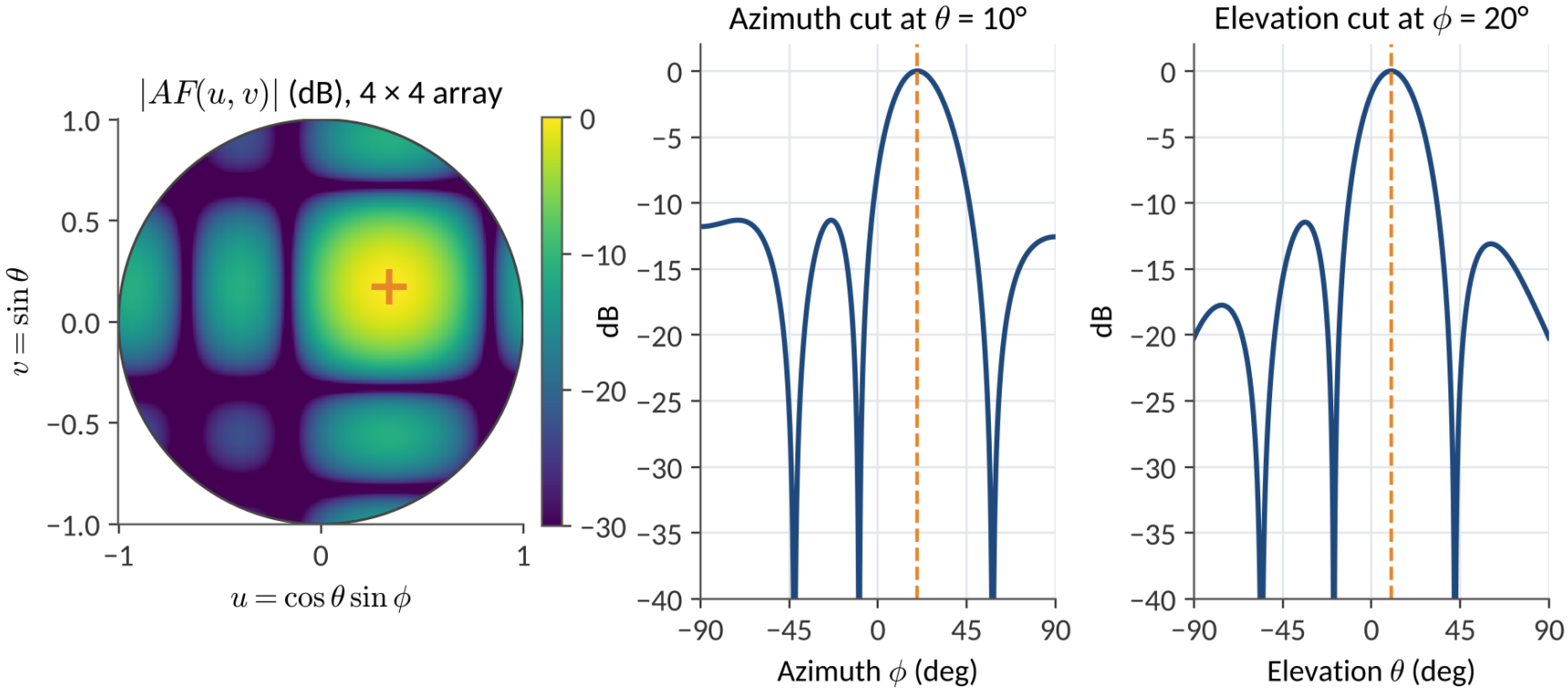
$$\mathbf{s}(u,v) = \mathbf{s}_y(u) \otimes \mathbf{s}_z(v), \qquad AF(u,v) = \mathbf{w}^H \mathbf{s}(u,v)$$

$\mathbf{s}_y(u) \in \mathbb{C}^{N_y}$, $\mathbf{s}_z(v) \in \mathbb{C}^{N_z}$: line-array steering vectors along each axis

Steer with $\mathbf{w} = \mathbf{s}(u_0, v_0)/N$, $N = N_y N_z$, exactly as in 1-D.

In 2-D, ϕ takes the role of the 1-D θ; θ is now elevation.

# The 4 × 4 array's pattern




Steered to ϕ = 20°, θ = 10°. Four elements per axis (N_y = N_z = 4): a ≈ 26° main lobe and −11 dB first sidelobes. This is the array in the labs.

# Quick Recap - What we assume (for now)

- **Far field**
  the wavefront is planar across the array
- **Narrowband**
  a time delay across the array is just a phase shift
- **Uniform spacing d = λ/2**
  no grating lobes
- **Identical, calibrated elements**
  only the phase differs from element to element
- **Known look direction $\theta_0$**
  we know where the signal of interest (SOI) is
- **Independent noise**
  uncorrelated from element to element

**Notation used throughout**

$N$ elements, index $n = 1, \ldots, N$

$\mathbf{x}(k) \in \mathbb{C}^N$: snapshot at time $k$

$\mathbf{s}(\theta)$: steering vector

$\mathbf{w}$: weights; $y(k) = \mathbf{w}^H \mathbf{x}(k)$

$AF(\theta) = \mathbf{w}^H \mathbf{s}(\theta)$: array pattern

$\theta$: angle from broadside; $d/\lambda$: spacing in wavelengths

2-D: $N = N_y N_z$; $\phi$ azimuth, $\theta$ elevation

$(\cdot)^H$ conjugate transpose, $(\cdot)^*$ conjugate

Bold = vector or matrix

*Later modules will add interferers that we know nothing about.*

# Understanding Array Patterns

**Element Factor(EF)**

Describes the radiation intensity around a single element of the array

**Array Factor(AF)**

Composite response of all array elements after weighting (assumes isotropic elements) for a given pointing direction

**Total Radiation Pattern(RP)**

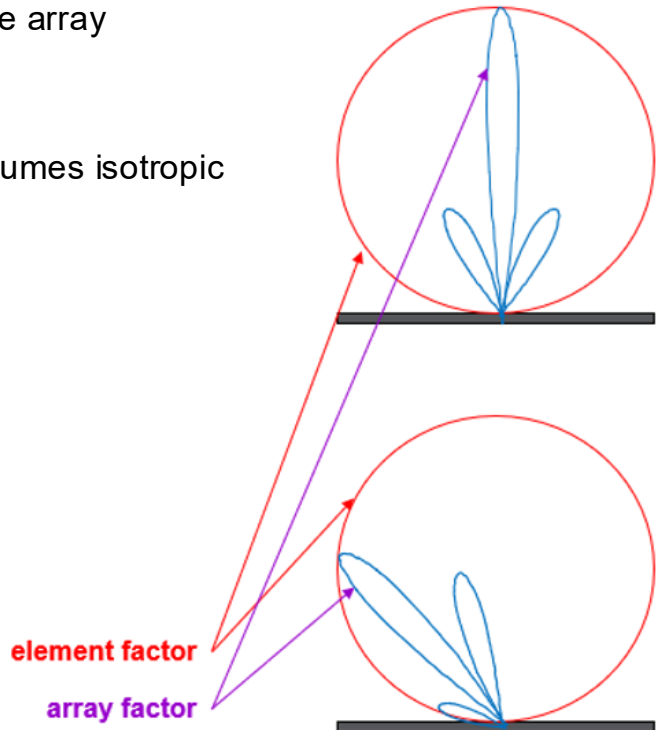
Describes spatial variation of a E-field component

(kypically, at the far field)as a function of angle

$\mathrm{RP} = \boldsymbol{EF} \odot \boldsymbol{AF}$ (element-wise multiplication of **EF** & **AF**)

or $\boldsymbol{EF} + \boldsymbol{AF}\ (dB)$

In this tutorial, we focus on Array Factor(**AF**)



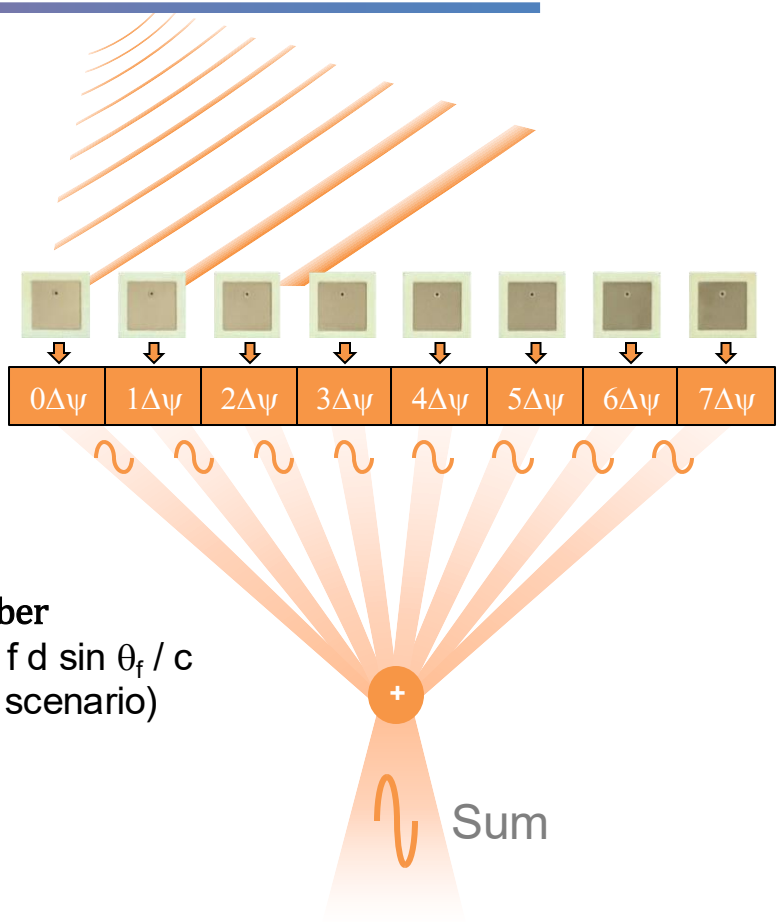
# Array Factor

- Recall that each element receives a signal that is delayed relative to the element next to it; assume this is from ($\theta_f$)
- And the array factor is the weighted sum of all those signals:

$$AF(\theta_f) = w_1 e^{j0\cdot\Delta\psi_f} + w_2 e^{j1\cdot\Delta\psi_f} + \cdots + w_8 e^{j7\cdot\Delta\psi_f}$$

- Expanding and Simplifying the above gives:

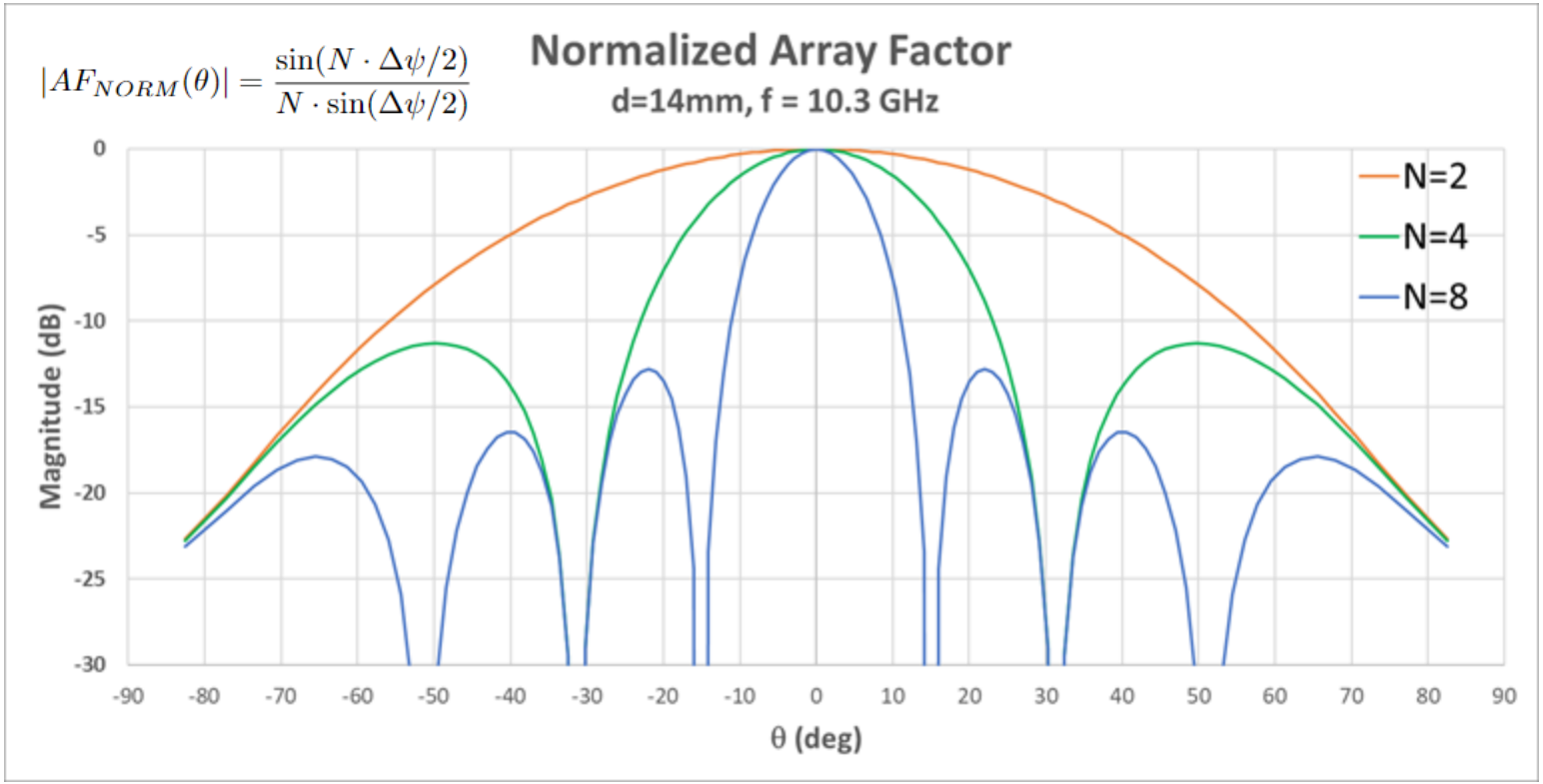
$$|AF(\theta_f)_{norm}| = \frac{\sin(N\cdot\Delta\psi_f/2)}{N\cdot\sin(\Delta\psi_f/2)}$$



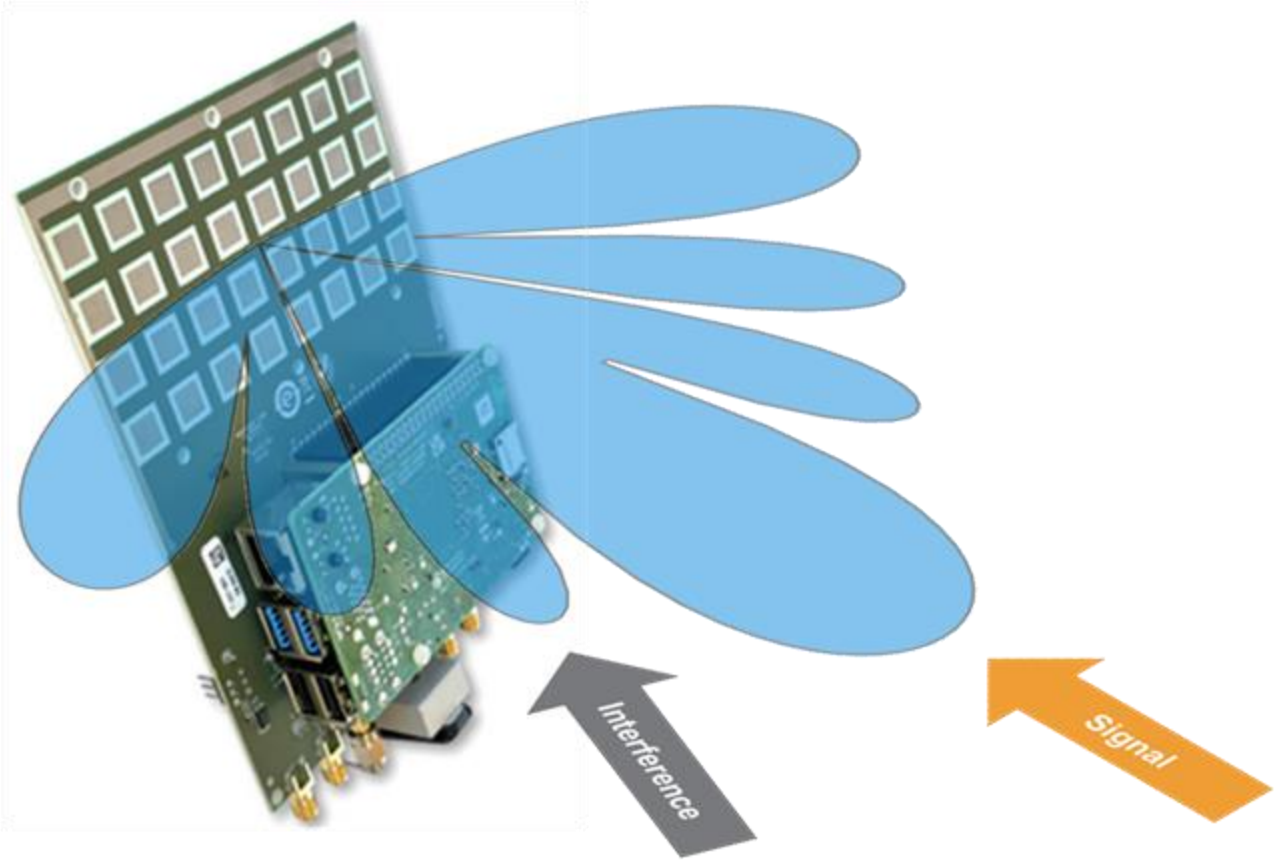

**Remember**
$\psi_f = 2\,\pi\,f\,d\,\sin\theta_f / c$
(for our scenario)

Remember
**w** is set to a specific direction

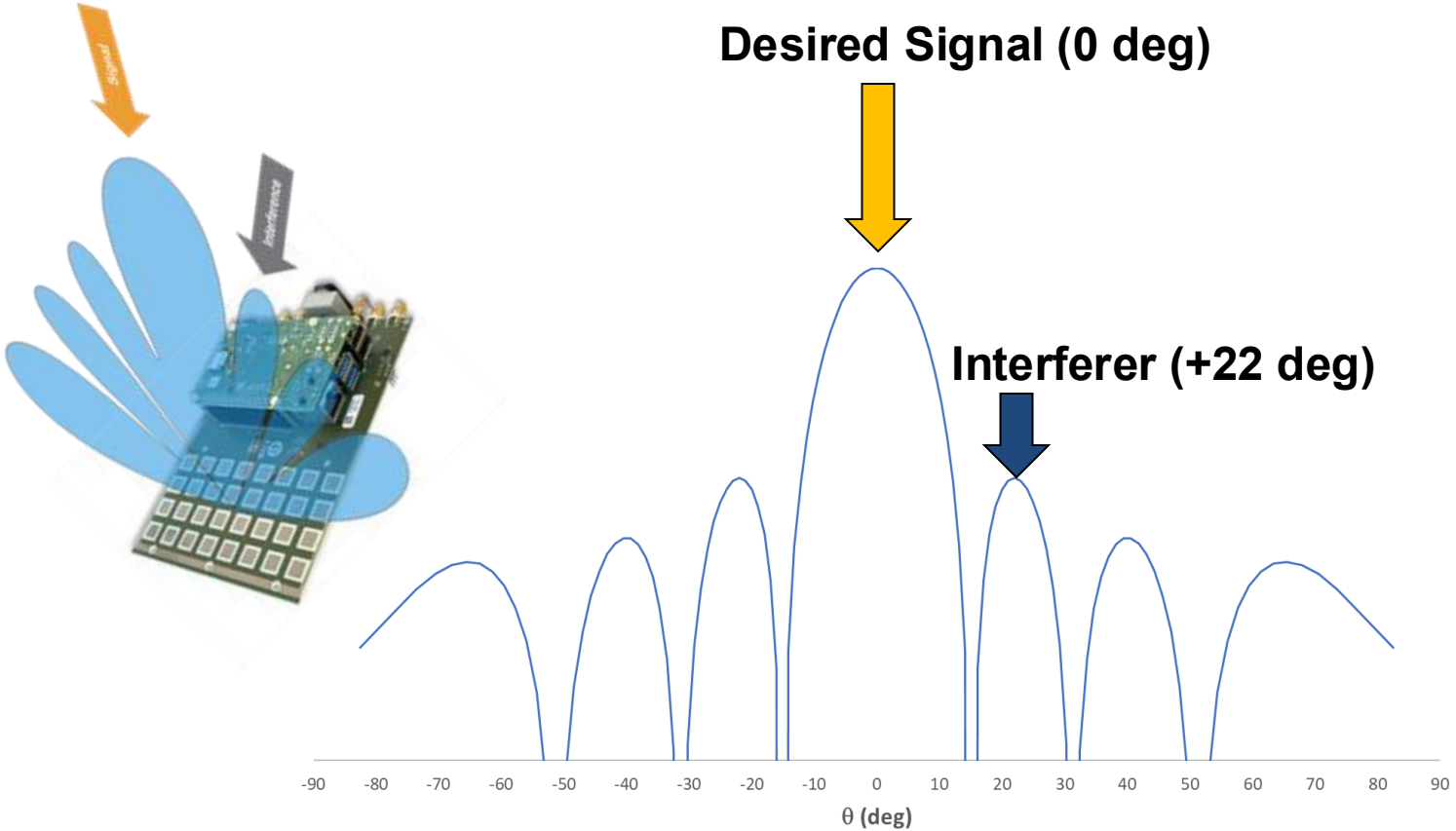
# Visualizing Array Factor: A 1D Linear Pattern/Slice



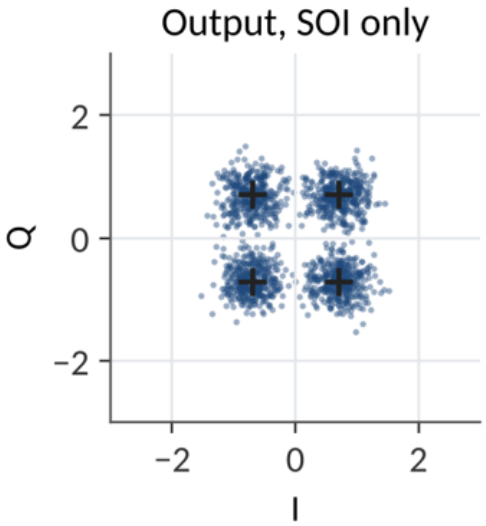
# The Problem With Sidelobes

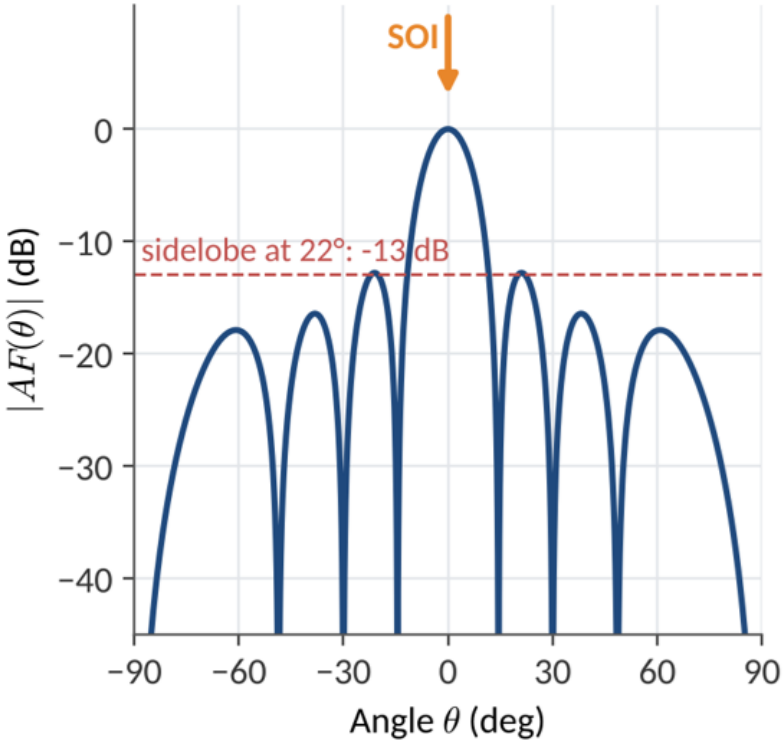


# **Example**: Desired signal is at 0 deg, Interferer at +22 deg

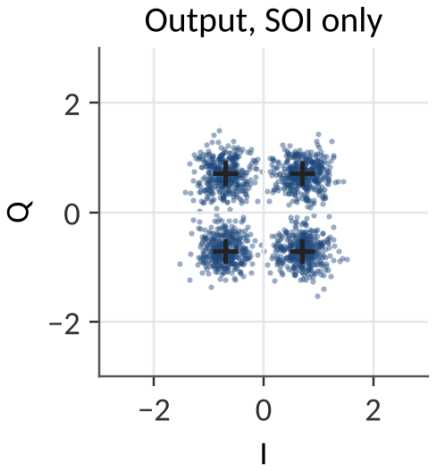


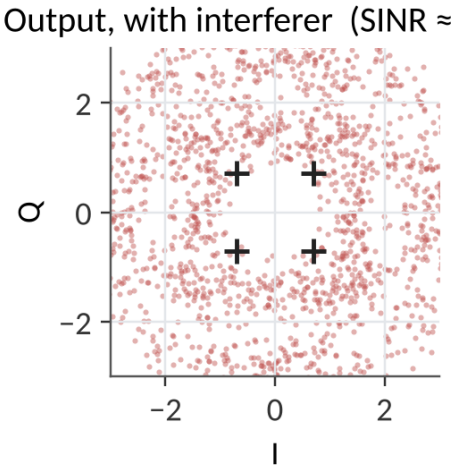
# Good QPSK Constellation (no Interferrer)

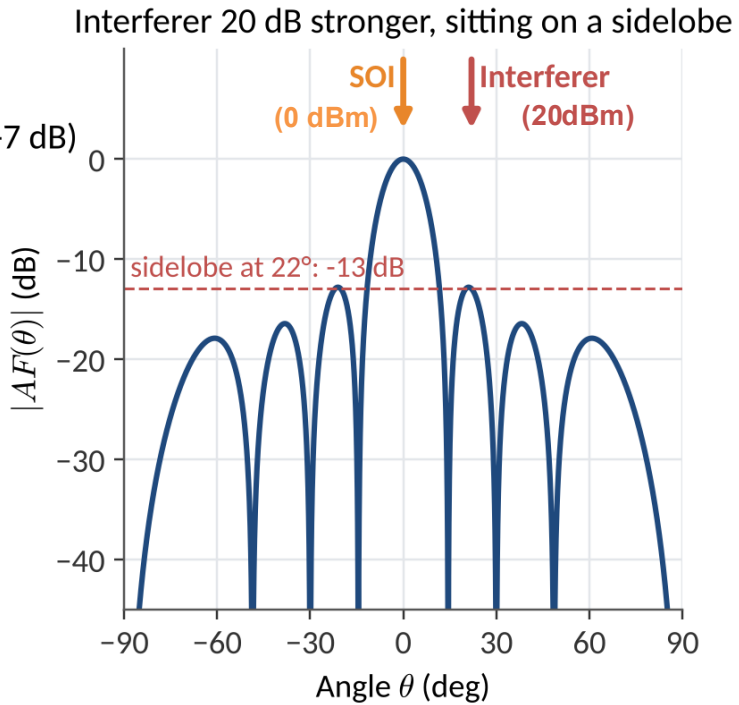


# What a sidelobe interferer does to the signal




Interferer 20 dB stronger at +22°: the -13 dB sidelobe leaves it 7 dB above the SOI, so the signal-to-interference-plus-noise ratio (SINR) ≈ -7 dB.

# **First** Method to Deal with Sidelobes: Antenna Tapering

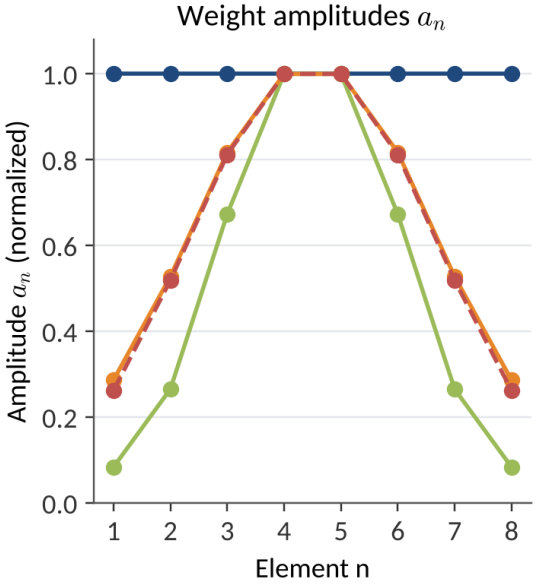
# Tapering: same phases, shaped amplitudes

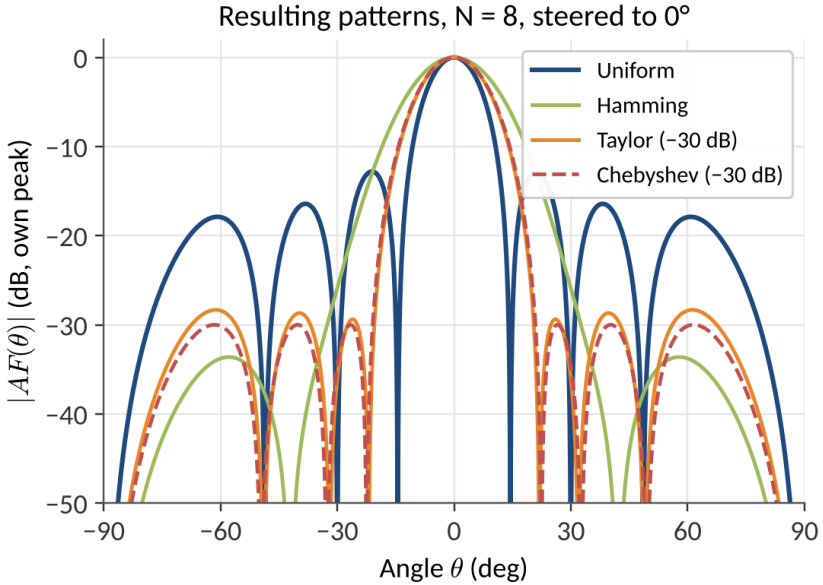
$$w_n = \frac{a_n}{\sum_m a_m} s_n(\theta_0)$$

Uniform ($a_n = 1$) is exactly the conventional beamformer we just built.

Other windows keep the phases (the beam still points at $\theta_0$) and only shape $|w_n|$: wider main lobe, lower sidelobes.

Taylor and Chebyshev are designed for a target sidelobe level (here −30 dB).



# Connecting Back to Beamweights

Steering Angle is θ (30° for our example)

Steering Vector ($\boldsymbol{s}$; $\boldsymbol{s} \in \boldsymbol{C}^{N}; N = 8$) is the array vector that corresponds to that steering angle

- For a linear array of size $N = 8$, is $\boldsymbol{\Psi}$ a column vector (8 row, 1 column),
- Where each element, $w_n$, is: $w_n = a_n \cdot e^{-j\,\Psi_n}$

$$\mathbf{w} = \begin{pmatrix} a_1 \cdot e^{-j\psi_1} \\ a_2 \cdot e^{-j\psi_2} \\ a_3 \cdot e^{-j\psi_3} \\ a_4 \cdot e^{-j\psi_4} \\ a_5 \cdot e^{-j\psi_5} \\ a_6 \cdot e^{-j\psi_6} \\ a_7 \cdot e^{-j\psi_7} \\ a_8 \cdot e^{-j\psi_8} \end{pmatrix} \rightarrow$$

**w** (polar form)

$$\begin{pmatrix} \boldsymbol{a_1} \cdot e^{-j\,0} \\ \boldsymbol{a_2} \cdot e^{-j\,1.5} \\ \boldsymbol{a_3} \cdot e^{-j\,3} \\ \boldsymbol{a_4} \cdot e^{-j\,4.5} \\ \boldsymbol{a_5} \cdot e^{-j\,6} \\ \boldsymbol{a_6} \cdot e^{-j\,1.2} \\ \boldsymbol{a_7} \cdot e^{-j\,2.7} \\ \boldsymbol{a_8} \cdot e^{-j\,4.2} \end{pmatrix}$$

**or**

**s** (rect form)

$$\begin{pmatrix} (1+0j) \\ (0.07+1j) \\ (-1+0.14j) \\ (-0.2-0.98j) \\ (0.96-0.28j) \\ (0.34+0.94j) \\ (-0.9+0.4j) \\ (-0.47-0.88j) \end{pmatrix}$$

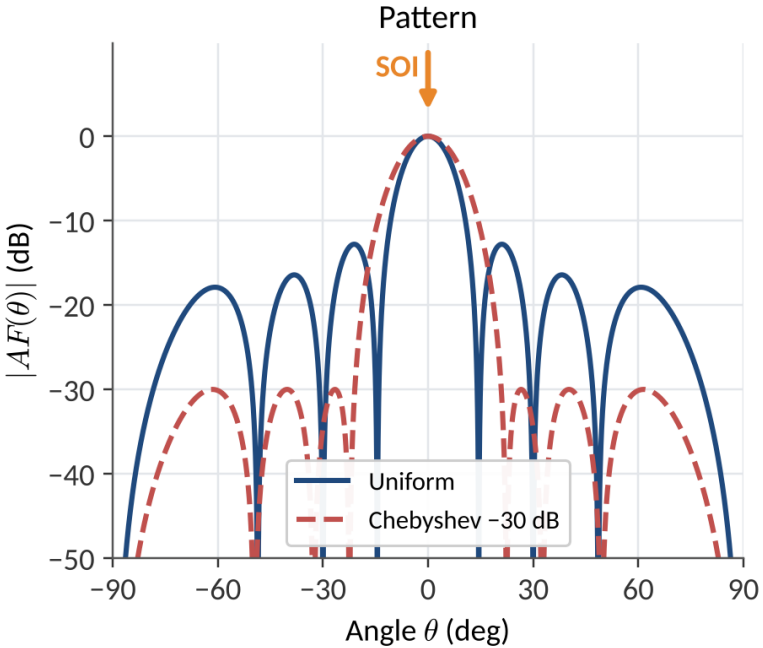
**Remember:**

Vector Notations:
- c : scalar
- $\mathbf{c}^{N}$: Nx1 column vector

Rectangular to Polar:
- $a = \sqrt{x^2 + y^2}$
- $\varphi = \arctan(y/x)$

Polar to Rectangular:
- $x = a\cos(\Delta\psi)$
- $y = a\sin(\Delta\psi)$

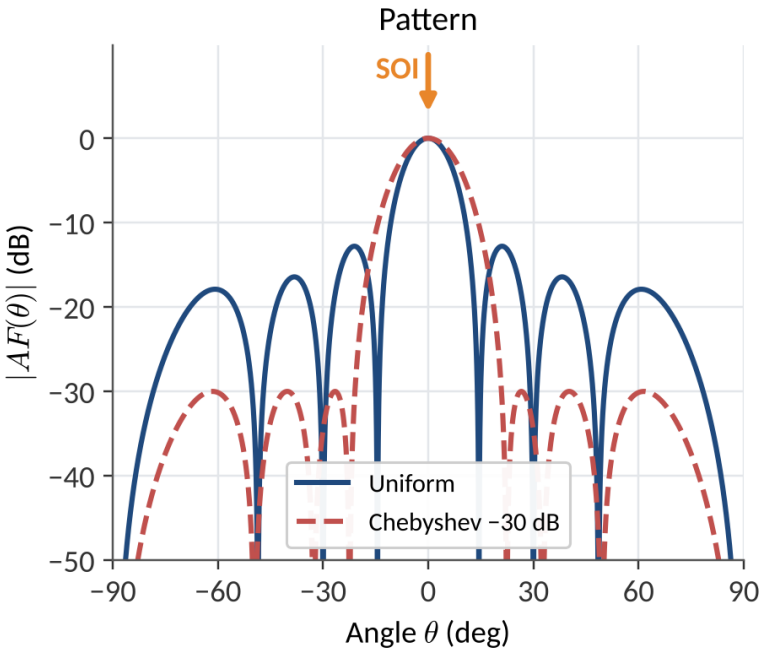
# Tapering in action

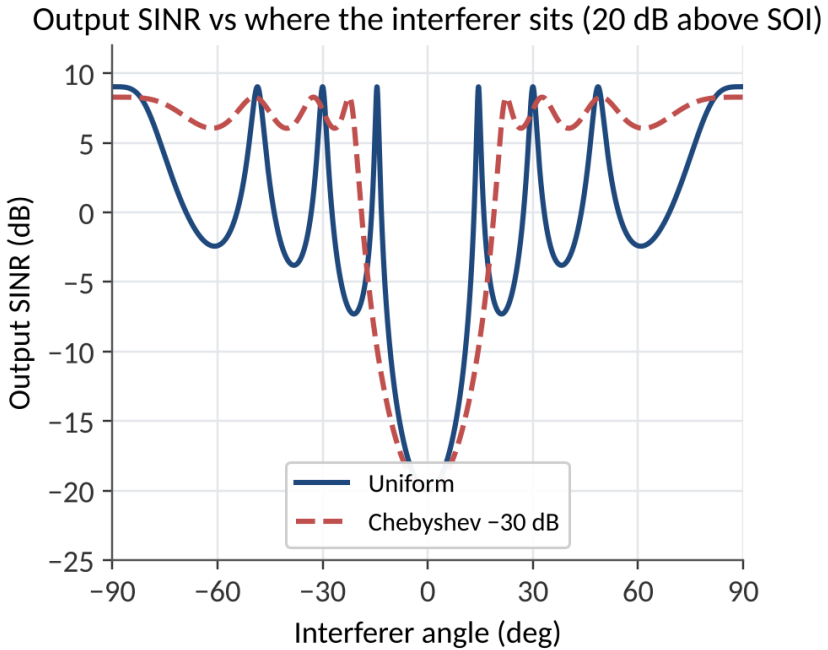



**Uniform → Chebyshev −30 dB**

| Beamwidth | Peak sidelobe | SNR gain |
|---|---|---|
| **12.8° → 16.4°** | **-13 → -30 dB** | **9.0 → 8.3 dB** |

# Tapering in action




**Uniform → Chebyshev −30 dB**

| Beamwidth | Peak sidelobe | SNR gain | Worst SINR beyond ±25° |
|---|---|---|---|
| **12.8° → 16.4°** | **-13 → -30 dB** | **9.0 → 8.3 dB** | **-5 → +6 dB** |

# Quad Apollo – “Triton” – ADXBAND16EBZ

System Development Platform

## Description/Key Features

### 4x AD9084 Apollo MxFE

- 16T / 16R Direct Sample L/S/C/X-Band
- Primary Use Case: 12.8/25.6 Gsps, 800 MSPS IQ

### On Board Front End Network

- Tx: Balun, Tunable BPF w/ Bypass, Gain Stage
- Rx: DSA, Gain Stage, Tunable BPF w/ Bypass, Balun
- SMPM Inputs/Outputs

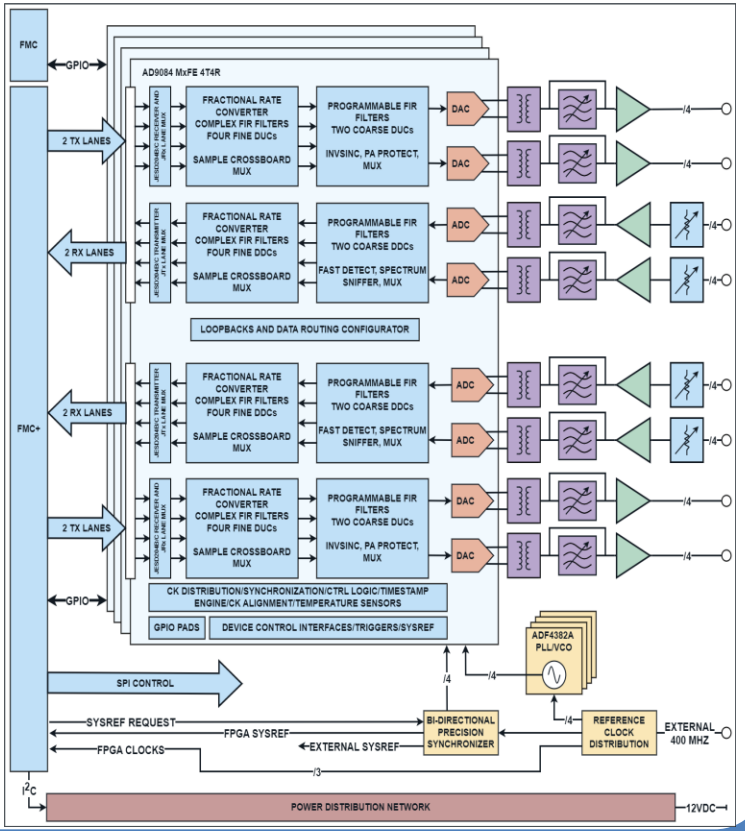
### FPGA

- 3rd Party Carrier: AMD VCU118 Eval

## Triton Board



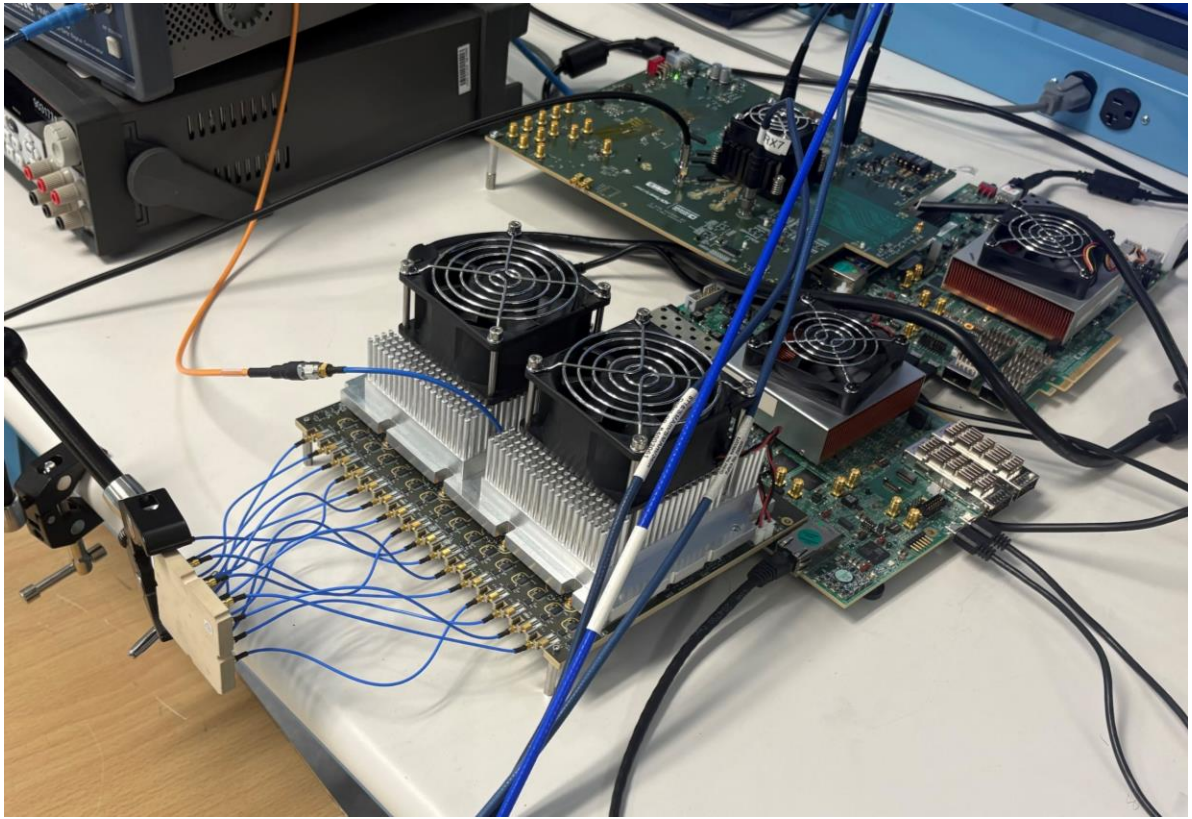
## Block Diagram




©2026 Analog Devices, Inc. All Rights Reserved.

# Basic Lab Setup:

- Triton is connected to a 4x4 Receive Array
- 1-4 transmitters from AD9084 Eval board are placed in the fair field at various azimuth and elevations
- Tx antennas were ~0.84 meters away
- Ultra-wideband WA5VJB antennas were used for each Tx source



# Control Triton from MATLAB:

Setup Details Here:
https://analogdevicesinc.github.io/system-level/solutions/platforms/triton/triton-bring-up/matlab-bring-up/

Key Steps:

1. Create Object

```
8
9   %% Open a connection and initialise Triton Board
10  x = Triton('ip:10.132.12.210'); % create Triton Object
11  x.initialize; % Initialize Triton
12
```

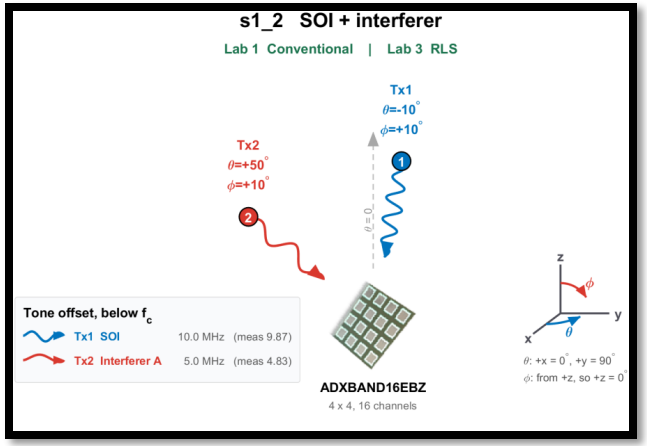
2. Create Tx Vector:

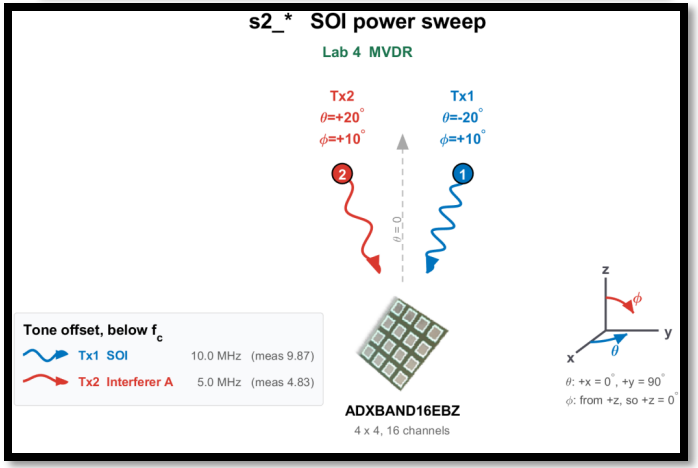
```
16  %% Create CW Waveform for DAC's
17  wave = x.createWaveform('cw',-1,x.basebandFreq); %create DMA waveform
18
```

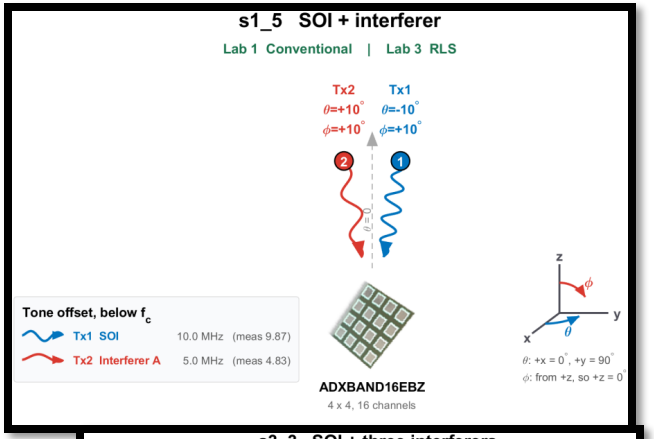
3. Send Tx, or Record Rx:

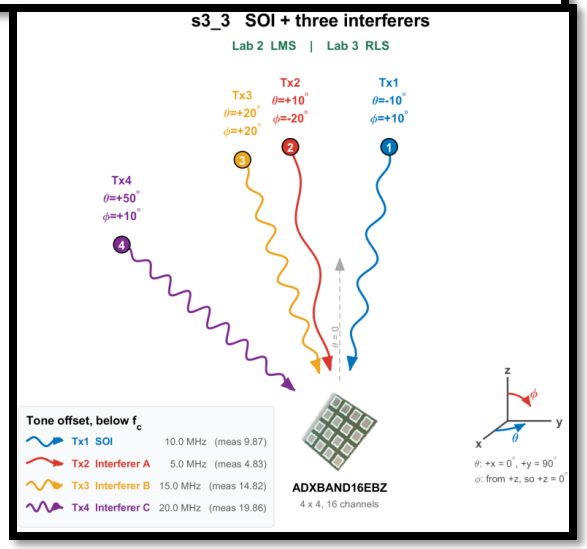
```
27  %% Apply the created vector to all DAC's
28  x.txWaveform(wave); % all DACs
29
30  %% Capture ADC data across all 16 channels and move to
31  allDAC = x.rx(); %ADC data capture
32
```

# Data Sets


s1_2 SOI + interferer
Lab 1 Conventional | Lab 3 RLS
Tx1
θ=-10°
ϕ=+10°
Tx2
θ=+50°
ϕ=+10°
θ = 0
Tone offset, below f_c
Tx1 SOI 10.0 MHz (meas 9.87)
Tx2 Interferer A 5.0 MHz (meas 4.83)
ADXBAND16EBZ
4 x 4, 16 channels
θ: +x = 0°, +y = 90°
ϕ: from +z, so +z = 0°


s2_* SOI power sweep
Lab 4 MVDR
Tx2
θ=+20°
ϕ=+10°
Tx1
θ=-20°
ϕ=+10°
θ = 0
Tone offset, below f_c
Tx1 SOI 10.0 MHz (meas 9.87)
Tx2 Interferer A 5.0 MHz (meas 4.83)
ADXBAND16EBZ
4 x 4, 16 channels
θ: +x = 0°, +y = 90°
ϕ: from +z, so +z = 0°

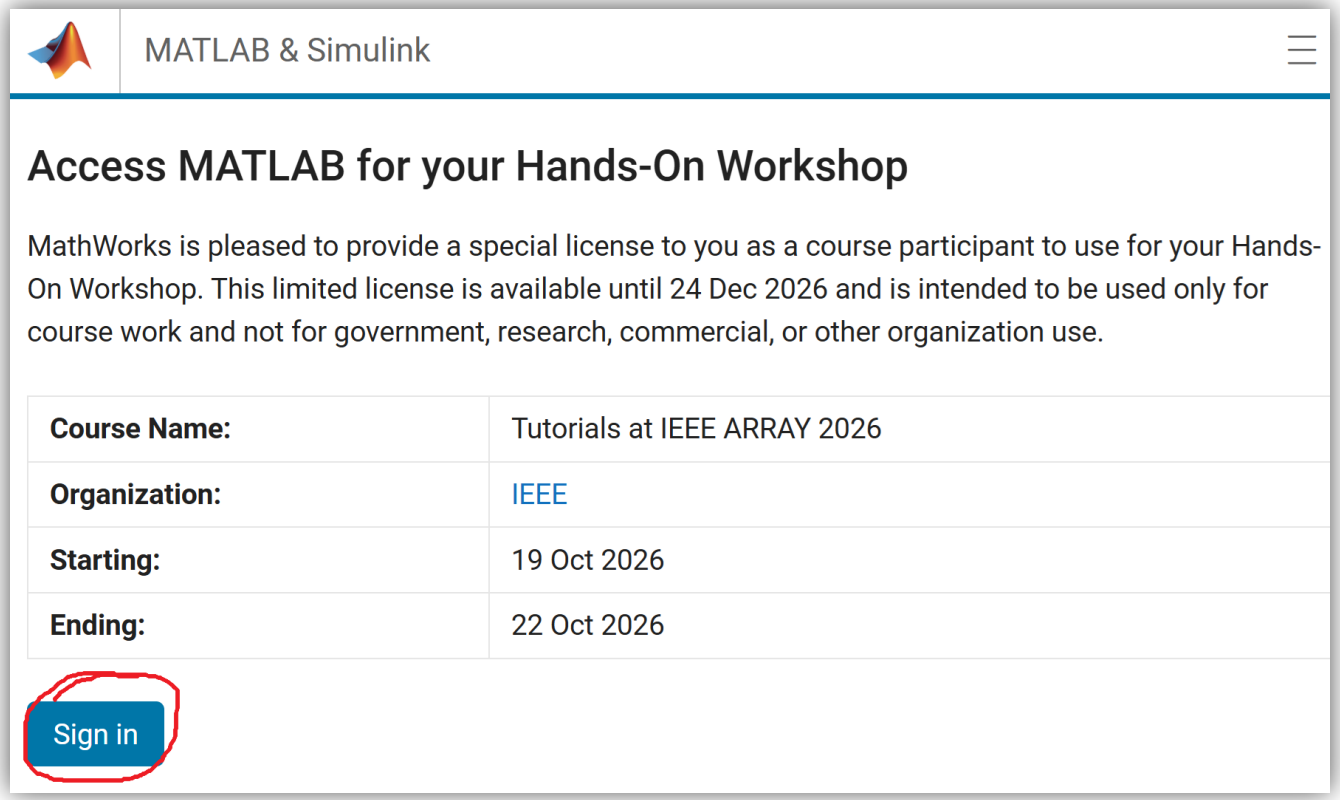

s1_5 SOI + interferer
Lab 1 Conventional | Lab 3 RLS
Tx2
θ=+10°
ϕ=+10°
Tx1
θ=-10°
ϕ=+10°
θ = 0
Tone offset, below f_c
Tx1 SOI 10.0 MHz (meas 9.87)
Tx2 Interferer A 5.0 MHz (meas 4.83)
ADXBAND16EBZ
4 x 4, 16 channels
θ: +x = 0°, +y = 90°
ϕ: from +z, so +z = 0°


s3_3 SOI + three interferers
Lab 2 LMS | Lab 3 RLS
Tx3
θ=+20°
ϕ=+20°
Tx2
θ=+10°
ϕ=-20°
Tx1
θ=-10°
ϕ=+10°
Tx4
θ=+50°
ϕ=+10°
θ = 0
Tone offset, below f_c
Tx1 SOI 10.0 MHz (meas 9.87)
Tx2 Interferer A 5.0 MHz (meas 4.83)
Tx3 Interferer B 15.0 MHz (meas 14.82)
Tx4 Interferer C 20.0 MHz (meas 19.86)
ADXBAND16EBZ
4 x 4, 16 channels
θ: +x = 0°, +y = 90°
ϕ: from +z, so +z = 0°

# MATLAB Online Setup

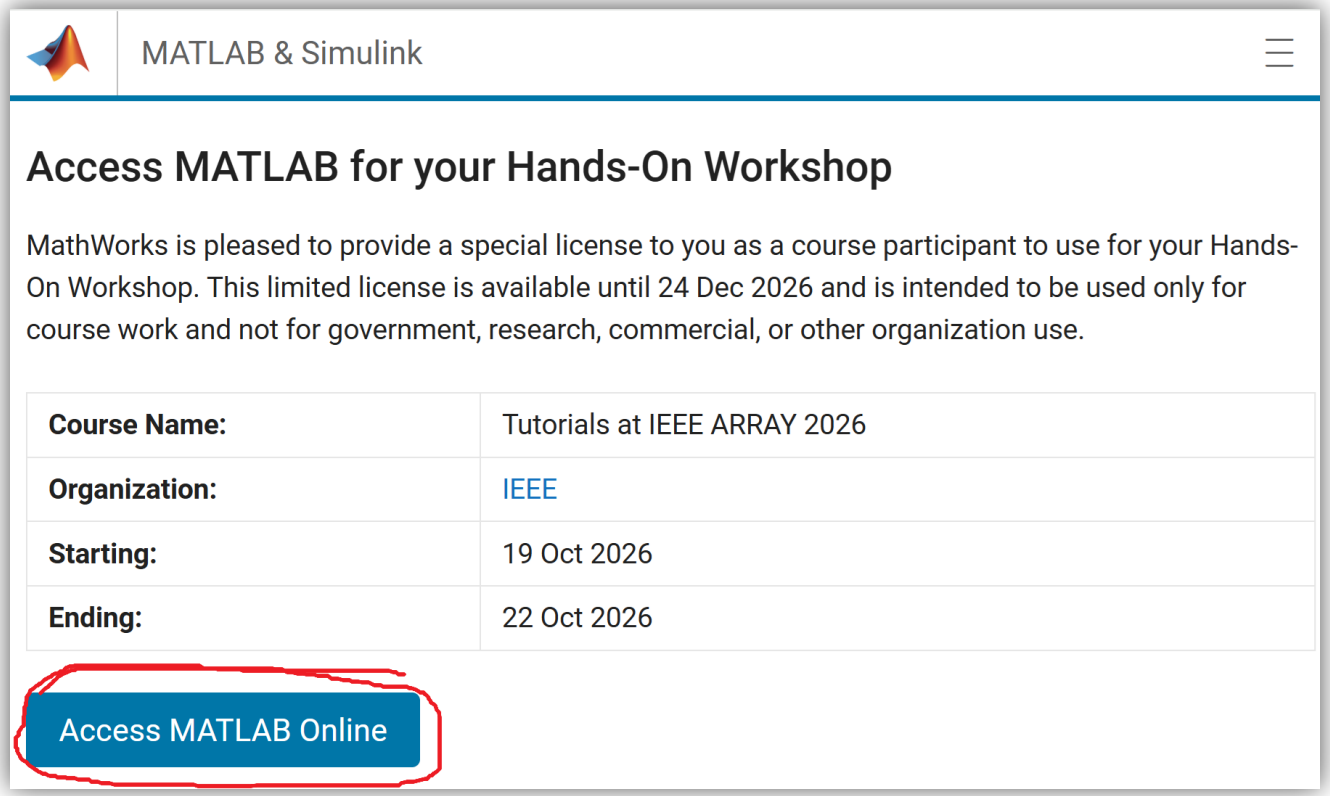
# Access free MATLAB Online Trial

- https://www.mathworks.com/licensecenter/classroom/5584450/



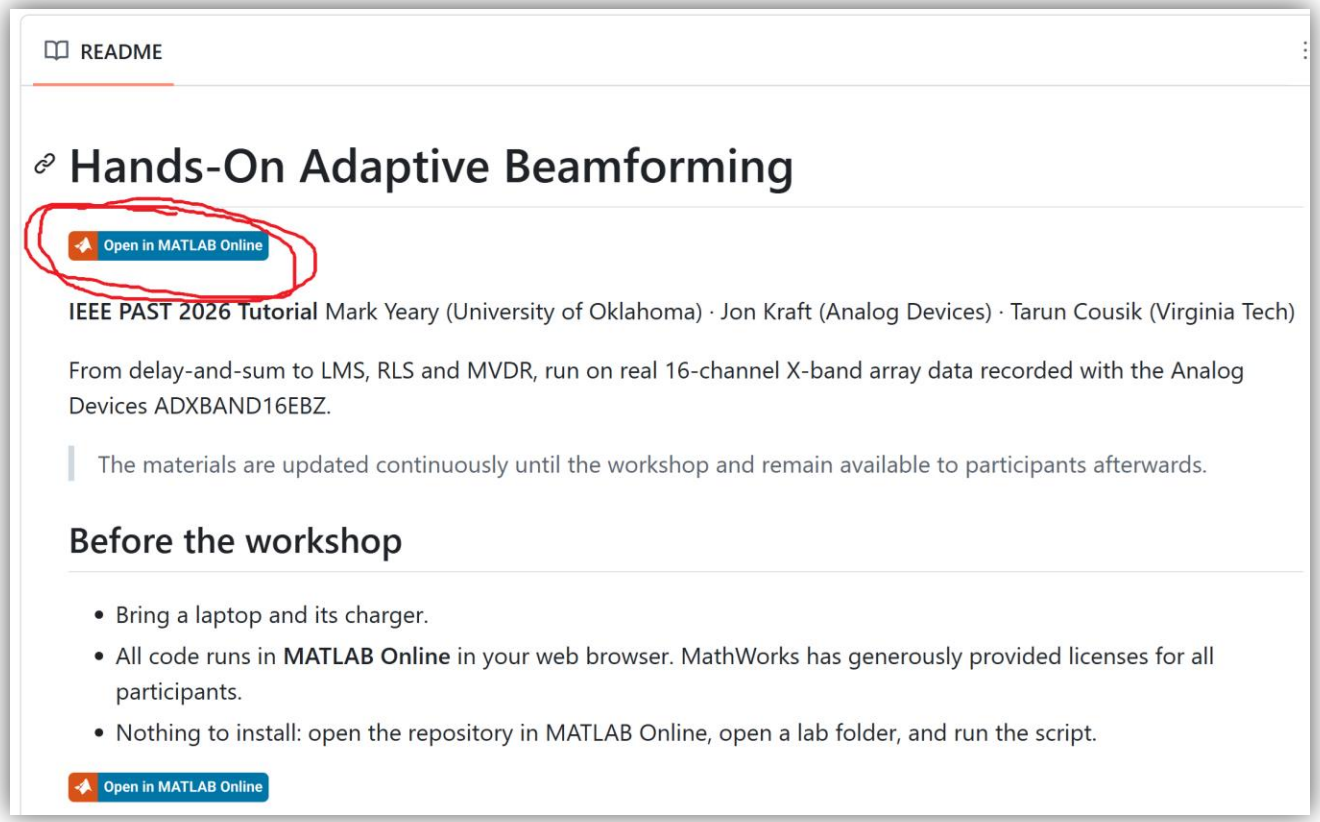
# Access free MATLAB Online Trial

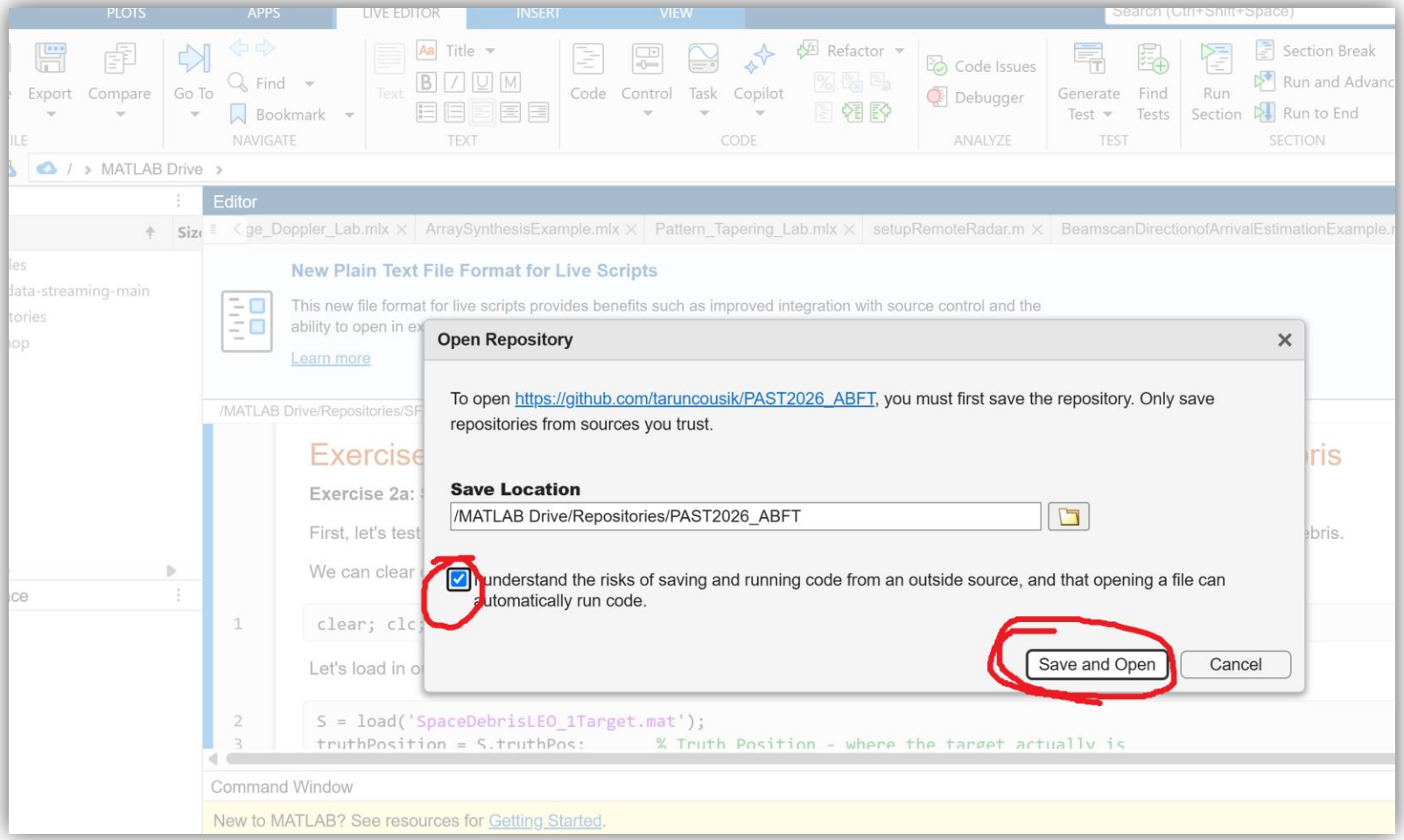
- https://www.mathworks.com/licensecenter/classroom/5584450/



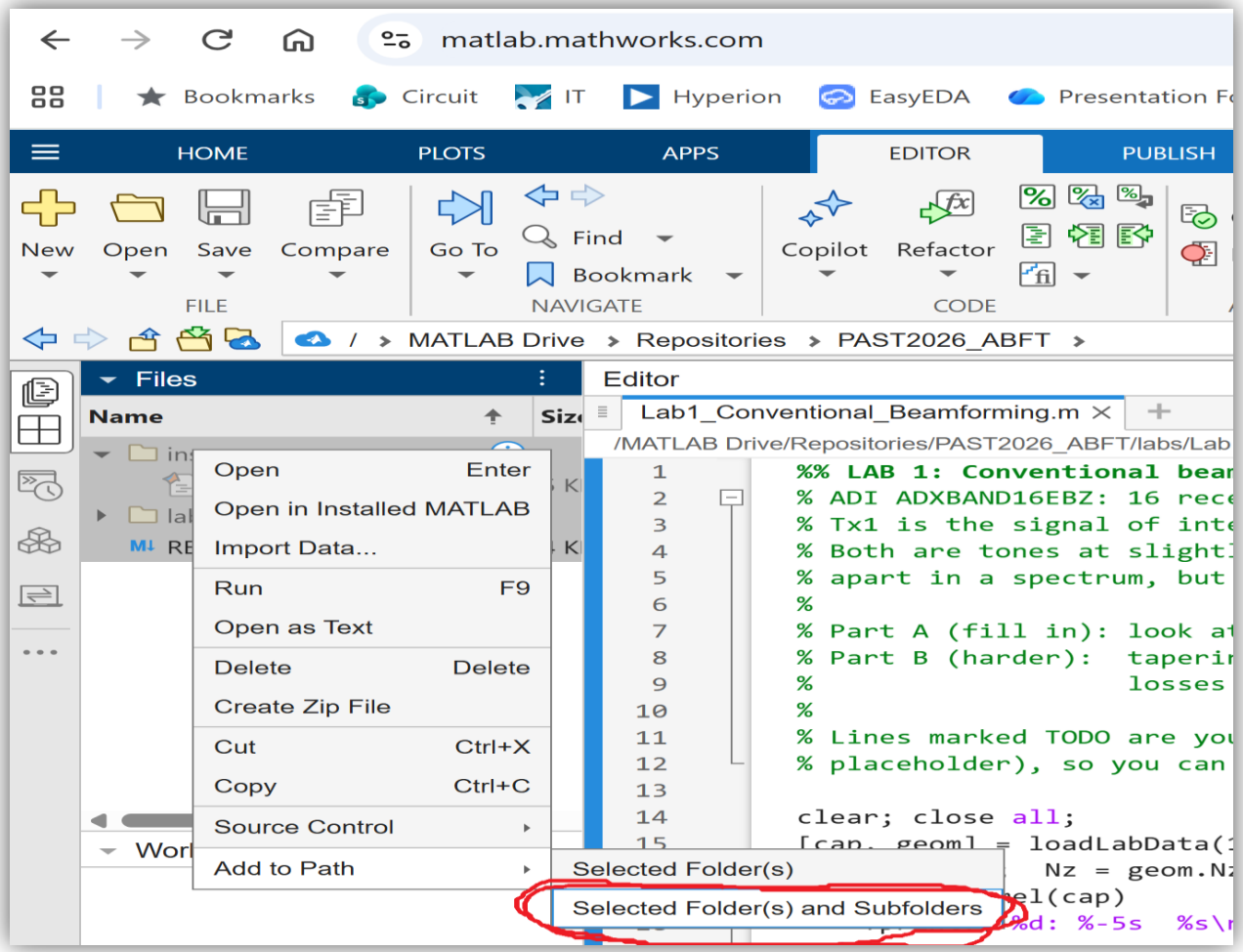
# Go to GITHUB link and click “Open in MATLAB Online”

- https://github.com/taruncousik/PAST2026_ABFT

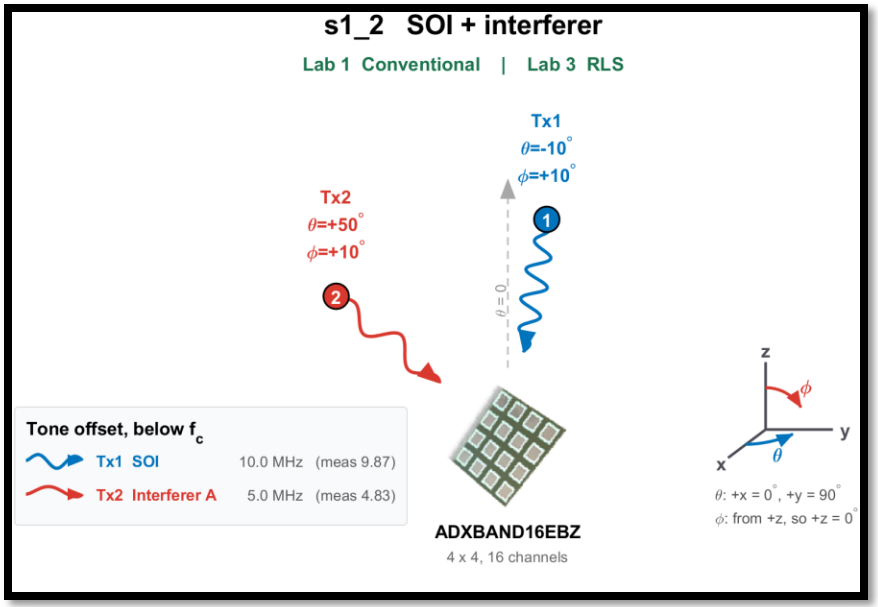


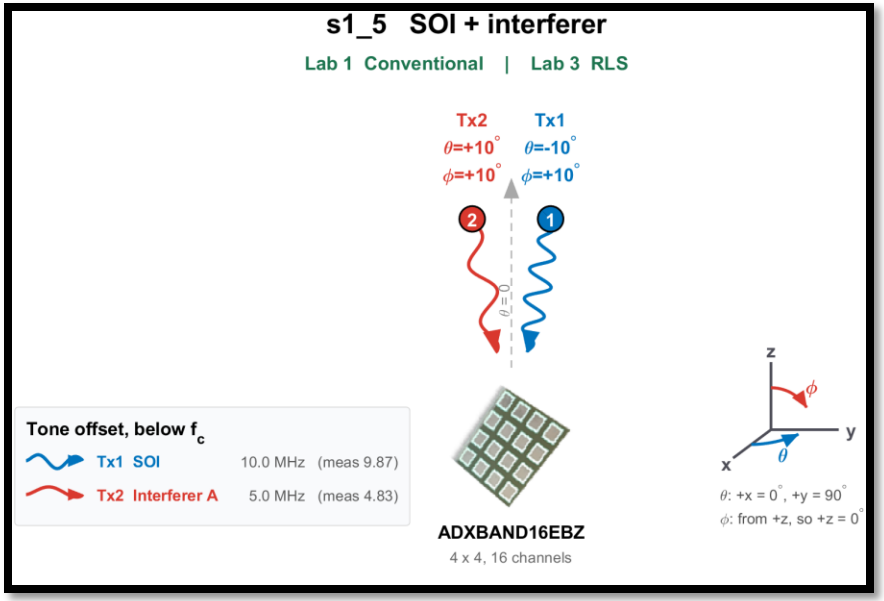

PLOTS
APPS
LIVE EDITOR
INSERT
VIEW
Export
Compare
Go To
Find
Bookmark
NAVIGATE
Title
Text
TEXT
Code
Control
Task
Copilot
Refactor
CODE
Code Issues
Debugger
ANALYZE
Generate Test
Find Tests
TEST
Run Section
Section Break
Run and Advanc
Run to End
SECTION
MATLAB Drive
Editor
ge_Doppler_Lab.mlx
ArraySynthesisExample.mlx
Pattern_Tapering_Lab.mlx
setupRemoteRadar.m
BeamscanDirectionofArrivalEstimationExample.
New Plain Text File Format for Live Scripts
This new file format for live scripts provides benefits such as improved integration with source control and the ability to open in ex
Learn more
Open Repository
To open https://github.com/taruncousik/PAST2026_ABFT, you must first save the repository. Only save repositories from sources you trust.
Save Location
/MATLAB Drive/Repositories/PAST2026_ABFT
I understand the risks of saving and running code from an outside source, and that opening a file can automatically run code.
Save and Open
Cancel
Exercise
Exercise 2a:
First, let's test
We can clear
clear; clc;
Let's load in o
S = load('SpaceDebrisLEO_1Target.mat');
truthPosition = S.truthPos; % Truth Position - where the target actually is
Command Window
New to MATLAB? See resources for Getting Started.

- If the folders are grayed out, then
  - Select (highlight) all the folders
  - Right click and select "**Add to Path**"
  - Further select "**Selected Folder(s) and Subfolders**"



# Lab 1 : Location of Signals

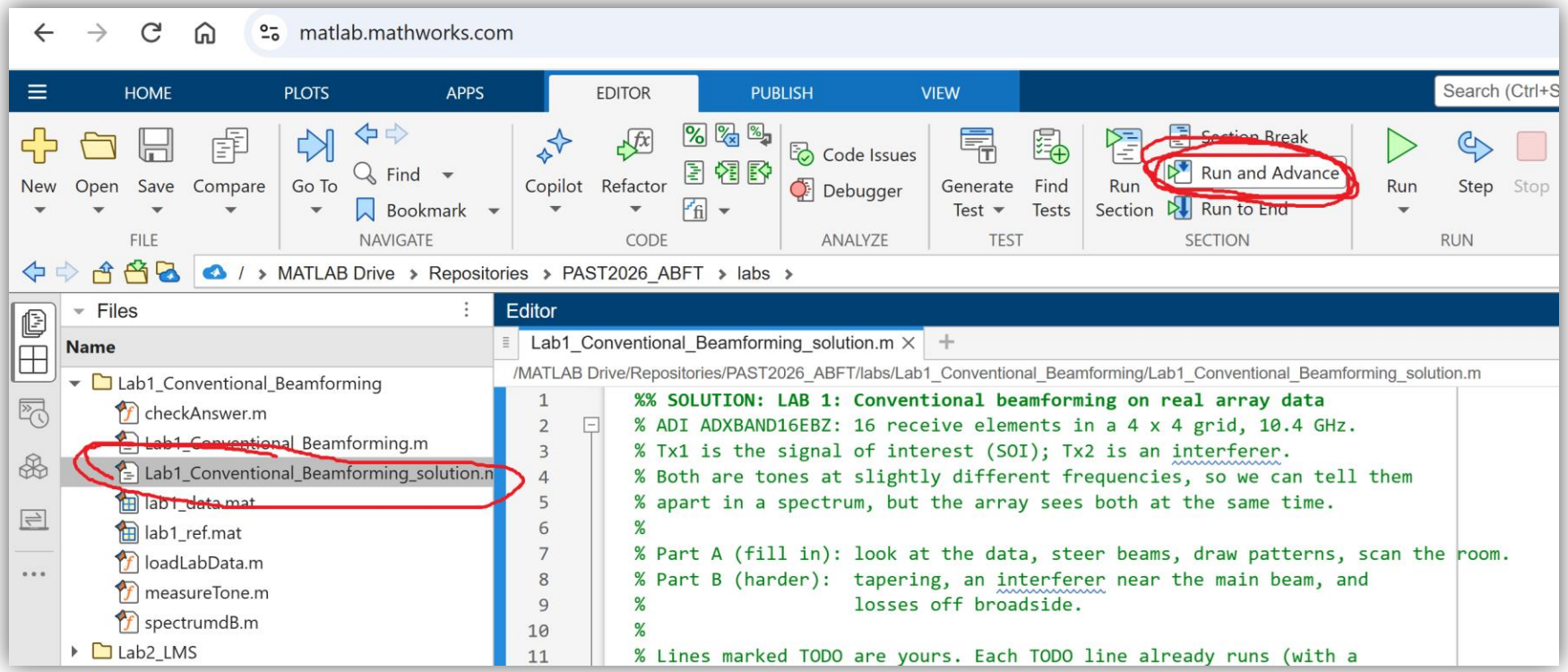


# Lab 1 (Move to MATLAB Online)

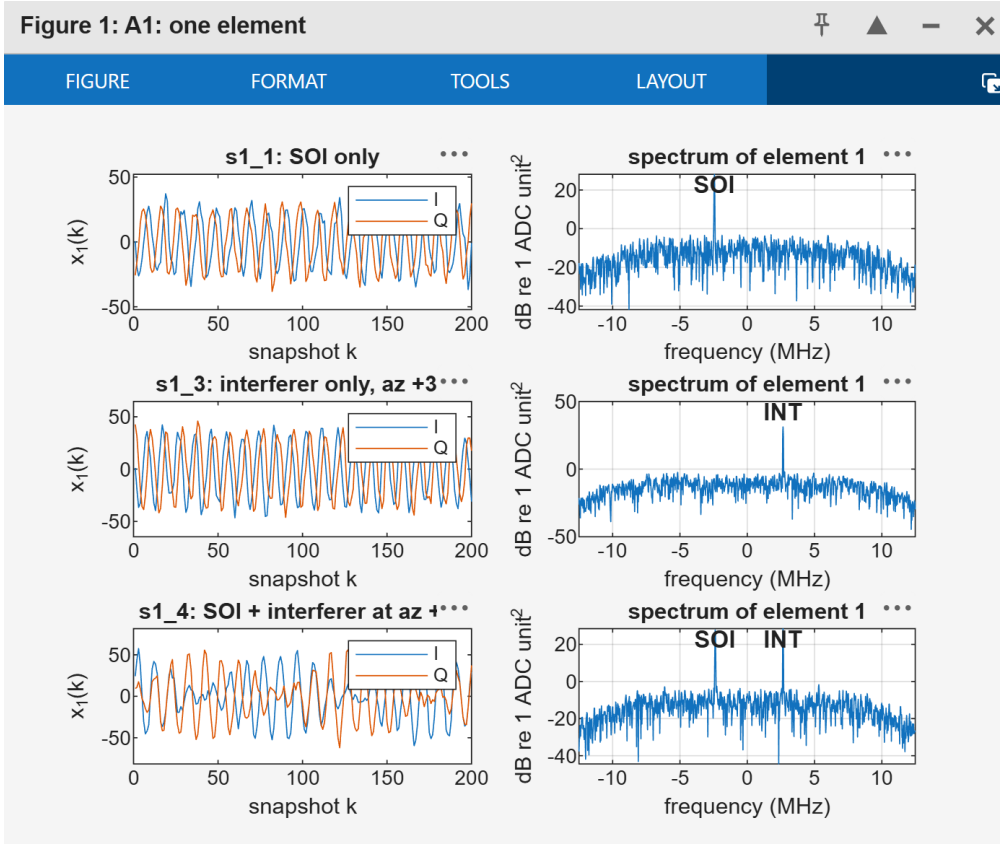
# Go to GITHUB link and click “Open in MATLAB Online”

- Open “Lab1_Conventional_Beamforming_**solutions**.m”, click “**Run and Advance**”



# Go to GITHUB link and click “Open in MATLAB Online”

- Open “Lab1_Conventional Beamforming.m” and click “Run and Advance”



# After Lab 1: what works, and what doesn't

## Good

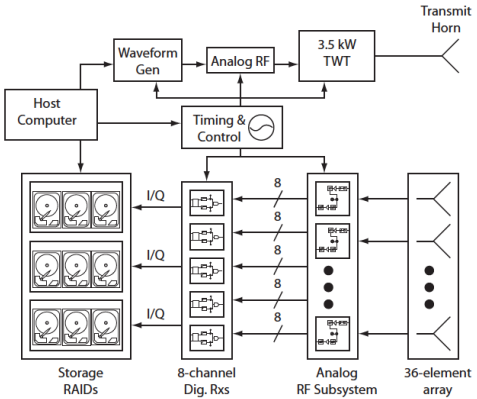
- Steering works: point $\mathbf{s}(\theta_0)$ and the SOI adds coherently (gain ≈ N)
- The pattern is fully predictable: $AF(\theta) = \mathbf{w}^H \mathbf{s}(\theta)$
- Same recipe in 1-D and 2-D
- Tapering buys lower sidelobes, and the lab results should match the simulation
- Cheap: the weights need no data at all

## Bad

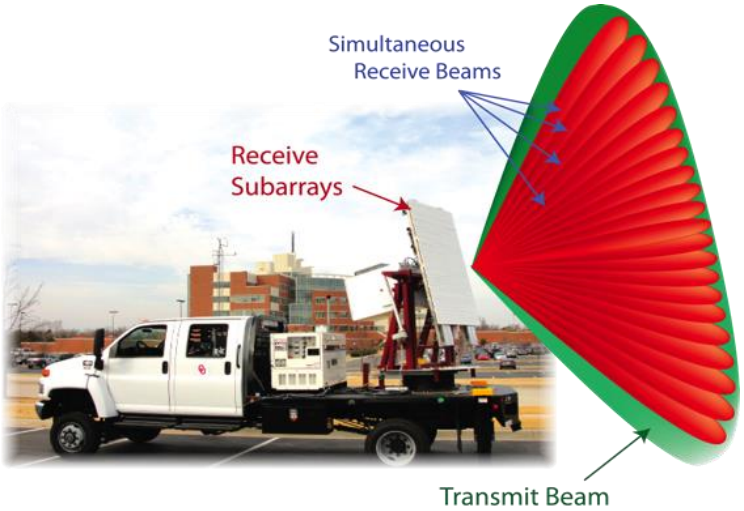
- The weights ignore what the array actually receives
- A strong interferer sits far above any sidelobe level (−13 dB uniform, ≈ −30 dB tapered)
- Tapering costs beamwidth and gain
- Placing a null needs the interferer's direction, and interferers move
- Net: against a strong interferer, performance is nothing to write home about

# Atmospheric Imaging Radar (AIR)

- Digital beamforming on receive: simultaneous measurements across the field of view, with as many beams as needed ("ubiquitous radar")
- High temporal resolution: RHI updates in under 1 second
- Clutter rejection with adaptive array processing (null steering, non-stationary clutter)




Each beam is a weighted sum; the Fourier weights are steering vectors:




$$y(k) = \mathbf{w}^H \mathbf{x}(k)$$

$$\mathbf{w} = \mathbf{s}(\theta) = \left[1,\ e^{-j\psi}, \ldots, e^{-j(N-1)\psi}\right]^T$$

Cite: [23]

# AIR: Fourier and windowed beams

## Fourier beamforming

$N$ elements, $L$ look angles $\theta_1, \ldots, \theta_L$; beam $l$:

$$y_l(k) = \mathbf{w}_l^H \mathbf{x}(k)$$

$$\mathbf{W} = [\mathbf{w}_1 \cdots \mathbf{w}_L] = 1/\sqrt{N}\,[\mathbf{s}(\theta_1) \cdots \mathbf{s}(\theta_L)]$$

- Simple to implement
- Efficient: FFT-based implementations
- But the fixed, high sidelobes suffer in clutter [7]

## Windowed beamforming

Same beams, with an amplitude taper across the array (Module 1):

$$\mathbf{W}_{\text{win}} = \text{diag}(\mathbf{a})\,\mathbf{W}, \qquad \mathbf{a} = [a_1, \ldots, a_N]^T$$

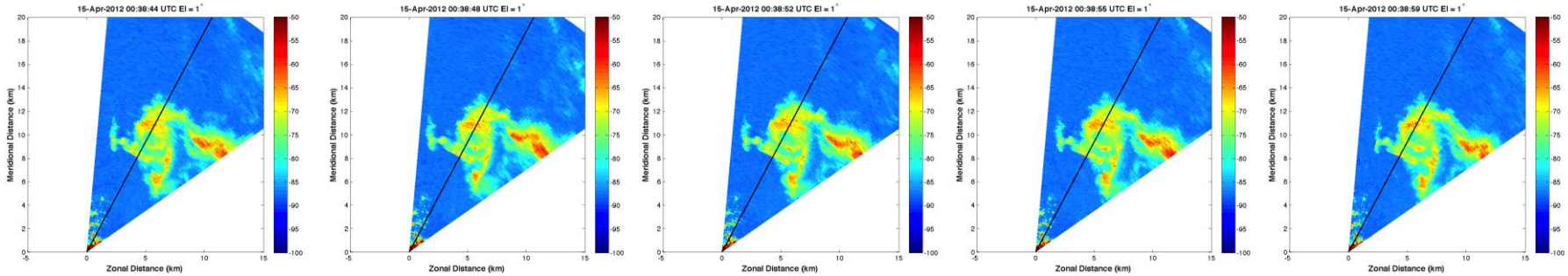
- Lower sidelobes, slightly wider beams
- Still fixed weights: nothing adapts to the clutter or interference

Cite: [23]; additional readings [7, 26, 41, 42, 48, 54]

# AIR: a tornado in Fourier-beamformed images

- Observed a tornado near Carmen, OK on April 14, 2012
- Mechanical azimuth scan, updating every ~3.5 s; simultaneous beams formed in elevation
- PPI images from Fourier beamforming (next five frames)



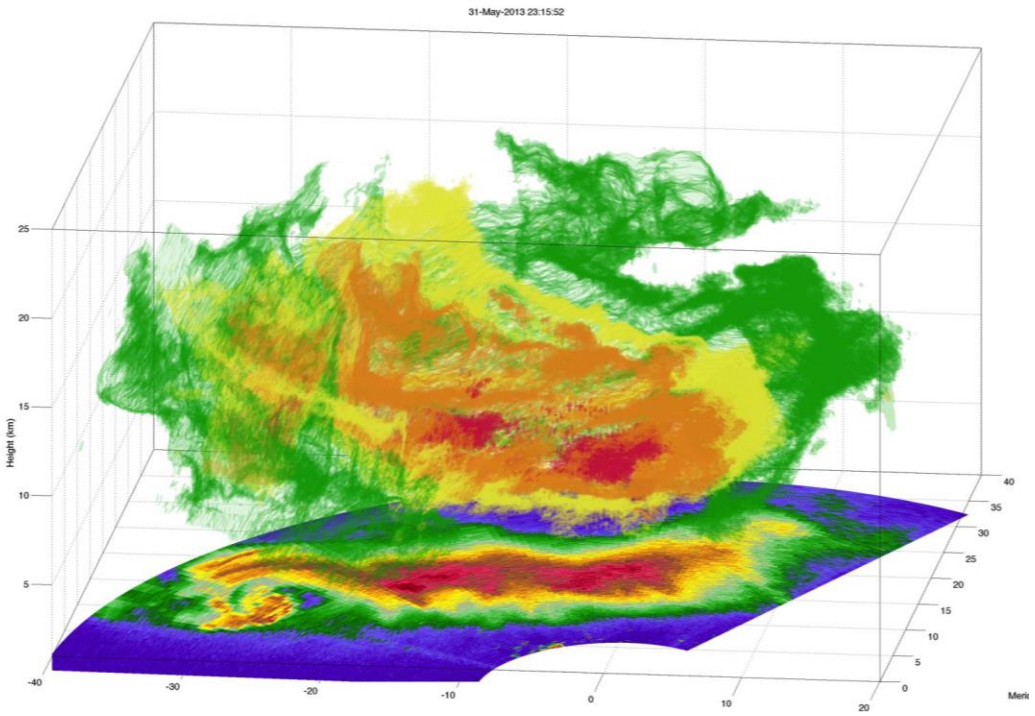



Cite: [23]

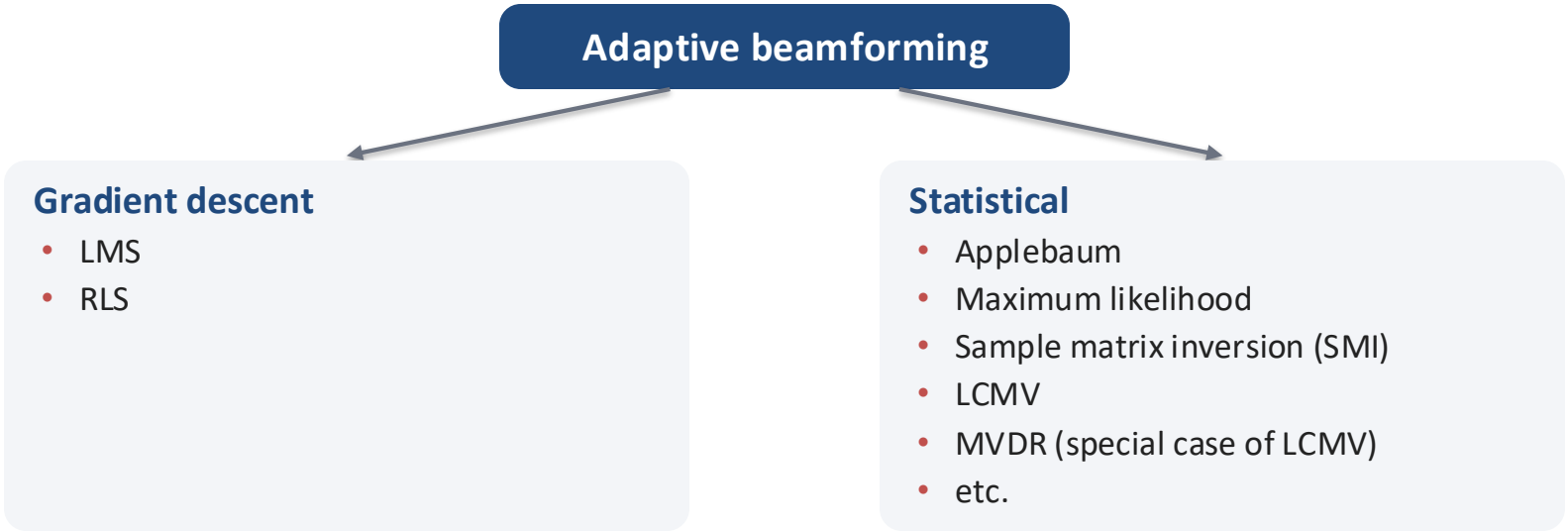
# AIR: A tornado in Fourier-beamformed images



**Key Point**: DBF on receive allows simultaneous measurements within the radar's field of view.



# Adaptive beamforming, A Tale of Two families

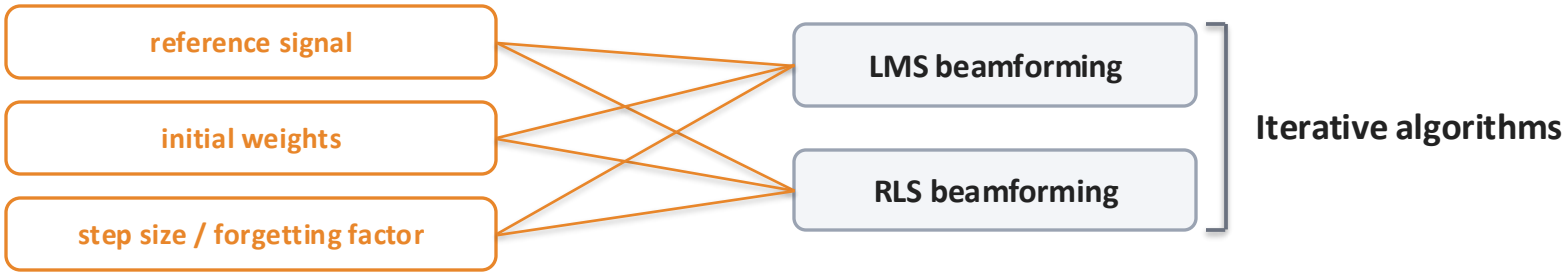



**Van Veen & Buckley [54]: beamformers are either data independent or statistically optimum.**

- Data independent: weights do not depend on the array data; chosen for a specified response in every scenario
- Statistically optimum: weights chosen from the statistics of the array data to "optimize" the response

Cite: [54]

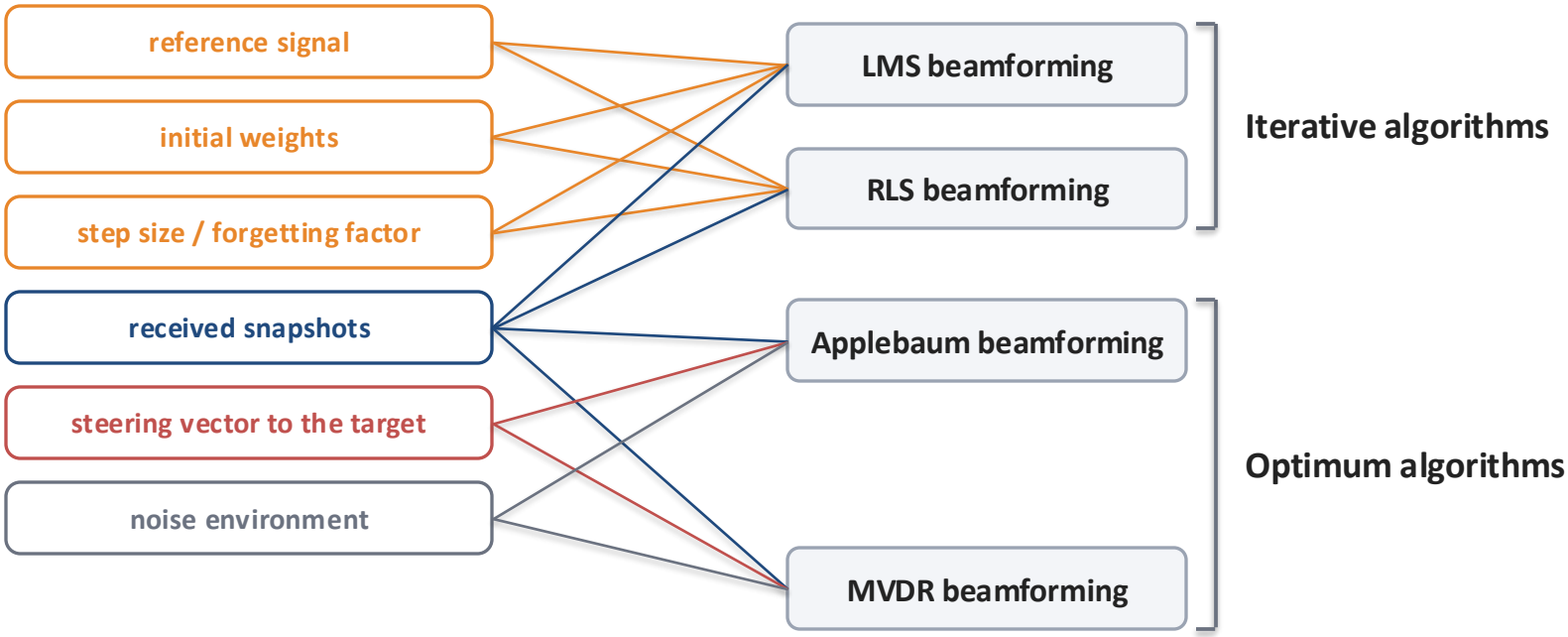
# What each algorithm needs




Key point: many algorithms exist. Iterative ones are computationally simple (no matrix inverse); optimum ones perform better (more beams, more accurate beamforming, deeper nulls).

Additional readings: [2–6, 8, 9, 13–15, 17–19, 24, 25, 27, 29–32, 36–40, 43–47, 49, 52–59]

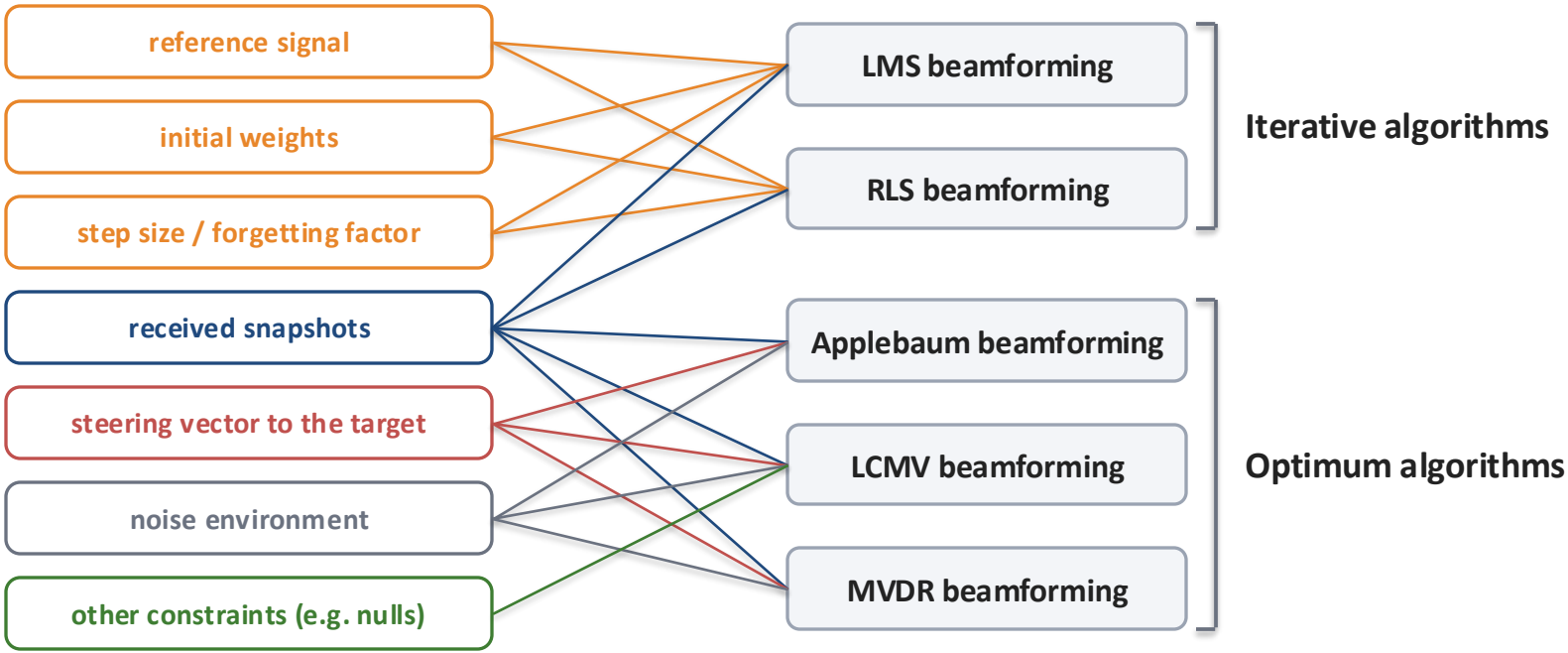
# What each algorithm needs




Key point: many algorithms exist. Iterative ones are computationally simple (no matrix inverse); optimum ones perform better (more beams, more accurate beamforming, deeper nulls).

Additional readings: [2–6, 8, 9, 13–15, 17–19, 24, 25, 27, 29–32, 36–40, 43–47, 49, 52–59]

# What each algorithm needs



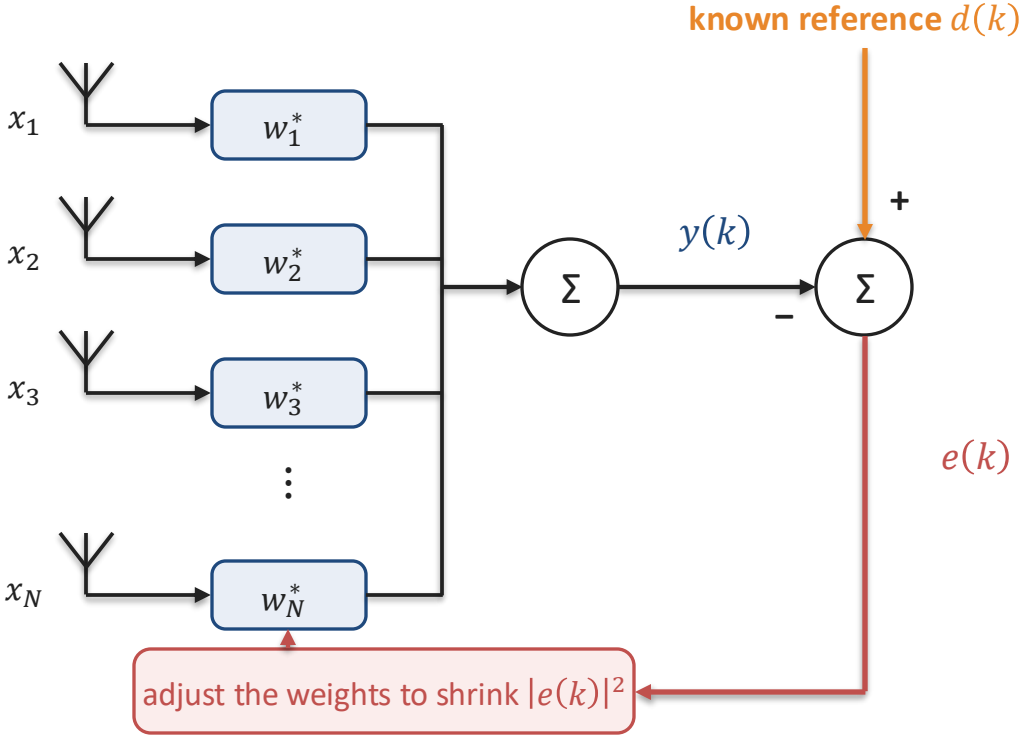

Key point: many algorithms exist. Iterative ones are computationally simple (no matrix inverse); optimum ones perform better (more beams, more accurate beamforming, deeper nulls).

Additional readings: [2–6, 8, 9, 13–15, 17–19, 24, 25, 27, 29–32, 36–40, 43–47, 49, 52–59]

# Module 2: LMS — Learning the Weights

A known reference signal, and one small correction per snapshot.

# What if we know what the signal looks like?




- Many systems transmit something known: a preamble, pilot symbols, a spreading (Gold) code (common in cellular and radar).
- Call it the reference $d(k)$.
- Compare the array output $y(k)$ with it: the error $e(k) = d(k) - y(k)$ says how wrong the weights are.
- Nudge the weights to shrink the error, one snapshot at a time.
- **No one has to tell the array where the interferer is.**

# What LMS ***Assumes*** *and Doesn't Assume*

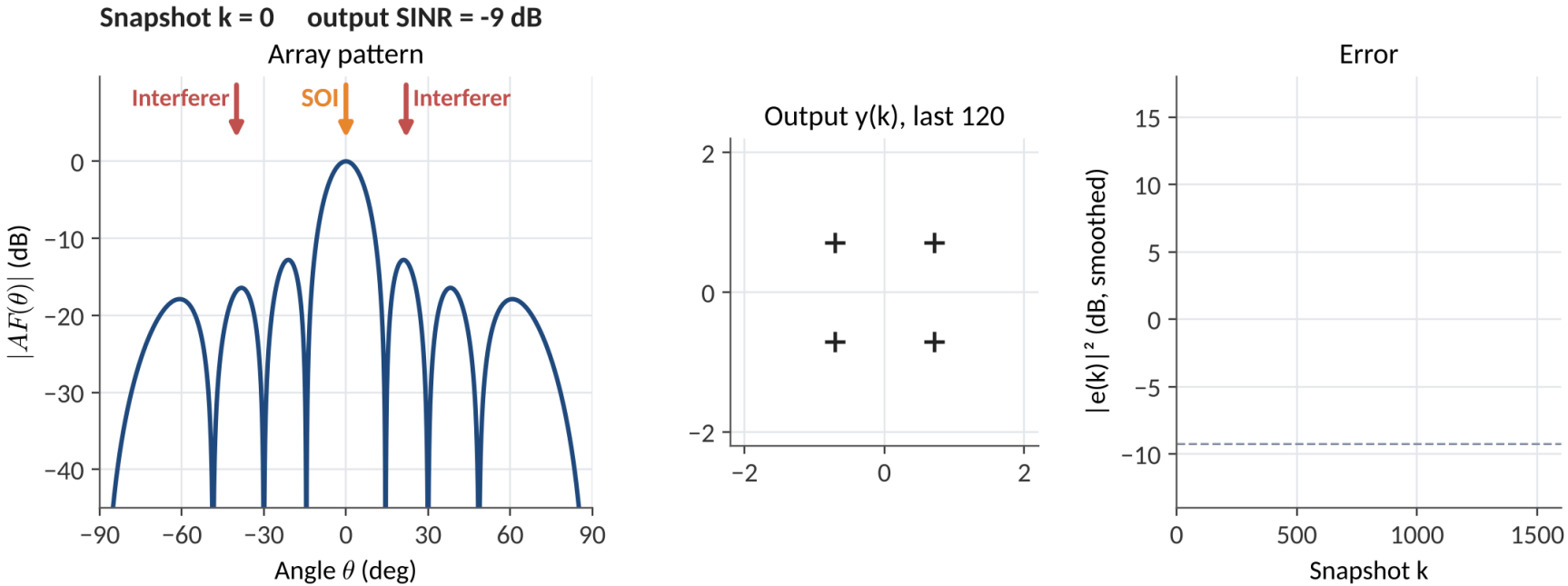
- A known reference $d(k)$, time-aligned with the received signal of interest
- The reference is correlated with the SOI and uncorrelated with interference and noise
- Interferer directions unknown; the SOI direction is not needed either
- The scene changes slowly compared with how fast the weights adapt
- Same narrowband, calibrated array as Module 1

What the receiver sees from the SOI over time; the reference is known only in the marked block (typical implementation)s:




LMS trains on the orange blocks, $d(k)$, which are known apriori; then the weights are held, or tracked, through the data

# LMS in action: the array learns a null




Same SOI at 0°, now two interferers, each 20 dB stronger, at −40° and +22°. Starting from the conventional weights, LMS places a null on each while keeping the SOI: SINR goes from −9 dB to ≈ 9 dB.

# LMS: three lines per snapshot

$$y(k) = \mathbf{w}^H(k)\,\mathbf{x}(k)$$

array output

$$e(k) = d(k) - y(k)$$

error against the reference

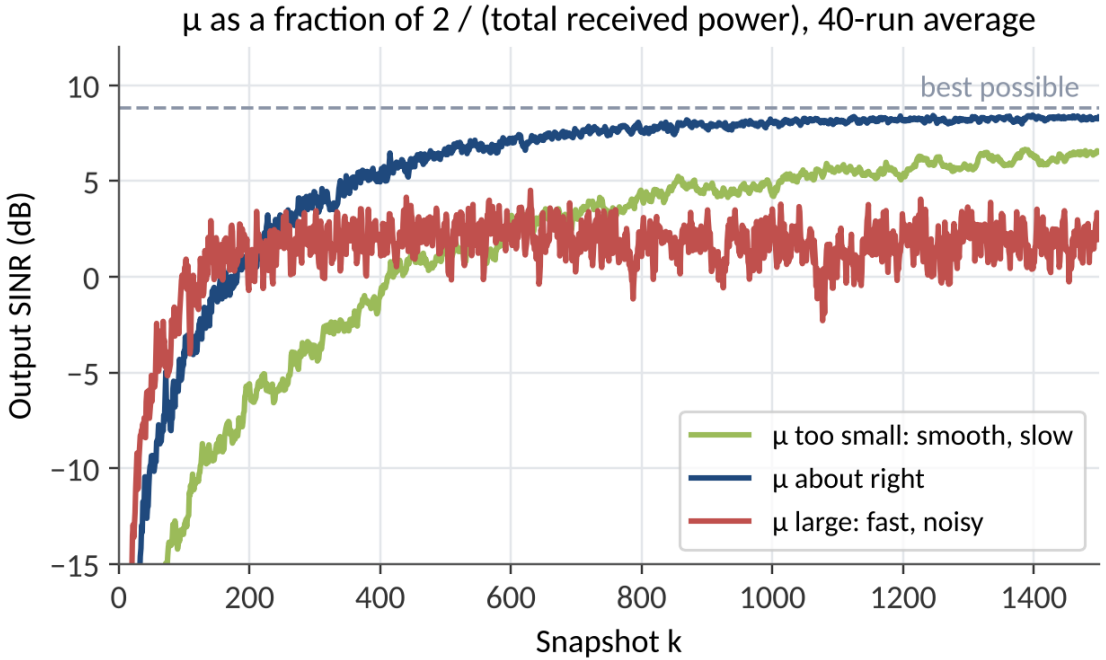
$$\mathbf{w}(k+1) = \mathbf{w}(k) + \mu\,\mathbf{x}(k)\,e^*(k)$$

nudge each weight along its own input, scaled by the error

$\mu$: step size (next slide). Start from $\mathbf{w} = \mathbf{0}$, or from the conventional weights.

```
% X: N-by-K snapshots, d: reference
w = zeros(N,1);
for k = 1:K
    y(k) = w' * X(:,k);
    e(k) = d(k) - y(k);
    w    = w + mu * X(:,k) * conj(e(k));
end
```

# Choosing the step size μ



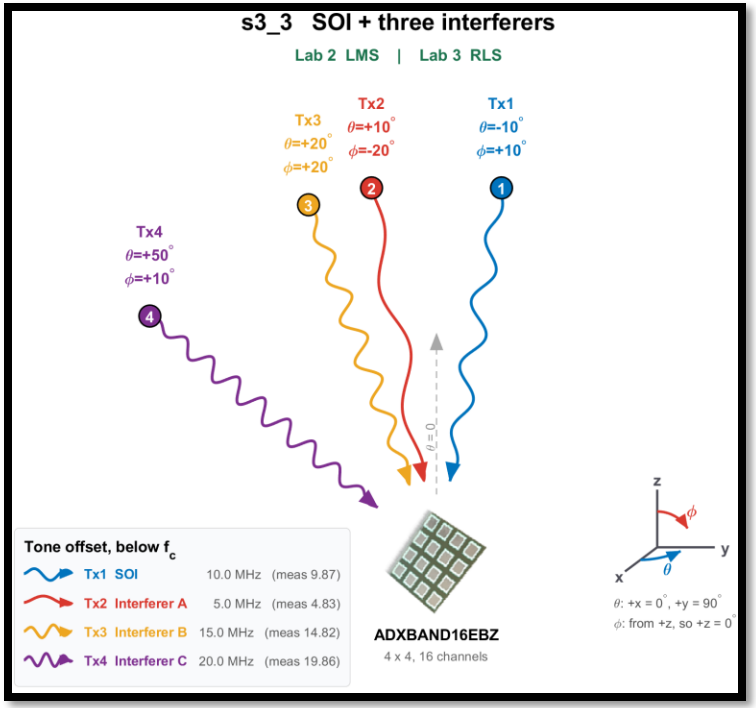

- Too small: smooth but slow
- Too large: fast, but the weights keep jittering around the optimum, and output SINR suffers

**Rule of thumb:**

$$0 < \mu < \frac{2}{\sum_n E\left[|x_n|^2\right]}$$

i.e. 2 over the total received power across all elements

# Lab 2: Location of Signals



# LAB 2 (Move to MATLAB Online)

# Where LMS comes from

$$e(k) = d(k) - \mathbf{w}^H \mathbf{x}(k)$$

# Where LMS comes from

$$e(k) = d(k) - \mathbf{w}^H \mathbf{x}(k)$$

$$J(\mathbf{w}) = E[|e(k)|^2] = \sigma_d^2 - \mathbf{w}^H \mathbf{p} - \mathbf{p}^H \mathbf{w} + \mathbf{w}^H \mathbf{R}\, \mathbf{w}$$

$$\mathbf{R} = E[\mathbf{x}(k)\, \mathbf{x}^H(k)], \qquad \mathbf{p} = E[\mathbf{x}(k)\, d^*(k)], \qquad \sigma_d^2 = E[|d(k)|^2]$$

- $\mathbf{R}$: the spatial covariance of the snapshots ($N \times N$). Module 4 looks at it closely.
- $\mathbf{p}$: how each element correlates with the reference ($N \times 1$)
- $J$ is quadratic in $\mathbf{w}$: a bowl with one minimum and no local traps

# Walking down the bowl

$$\frac{\partial J}{\partial \mathbf{w}^*} = \mathbf{R}\,\mathbf{w} - \mathbf{p} = \mathbf{0} \;\Rightarrow\; \mathbf{w}_{\text{opt}} = \mathbf{R}^{-1}\mathbf{p}$$

Wiener solution. Steepest descent walks downhill:

$$\mathbf{w}(k+1) = \mathbf{w}(k) + \mu[\mathbf{p} - \mathbf{R}\,\mathbf{w}(k)]$$

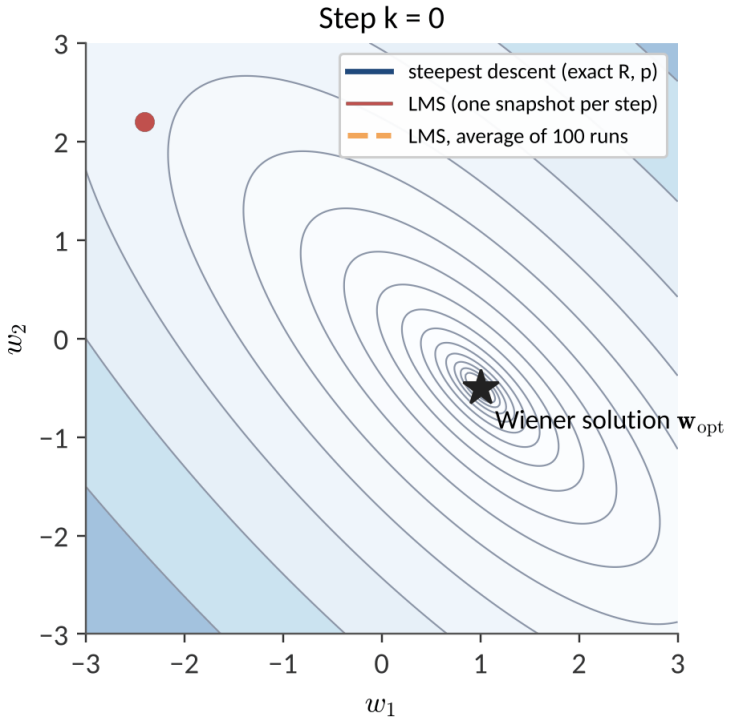
LMS uses what one snapshot says about R and p:

$$\mathbf{p} - \mathbf{R}\,\mathbf{w} \approx \mathbf{x}(k)[d^*(k) - \mathbf{x}^H(k)\,\mathbf{w}(k)]$$

$$= \mathbf{x}(k)\,e^*(k)$$

$$\Rightarrow \quad \mathbf{w}(k+1) = \mathbf{w}(k) + \mu\,\mathbf{x}(k)\,e^*(k)$$

Each LMS step is noisy, but on average it points downhill, and it costs about 2N multiplies.




Same bowl, same start: exact gradient (blue), one LMS run (red), LMS averaged over 100 runs (dashed).

# How fast, and how stable

Split $\mathbf{R}$ into its eigen-directions: the average weight error along each one shrinks on its own.

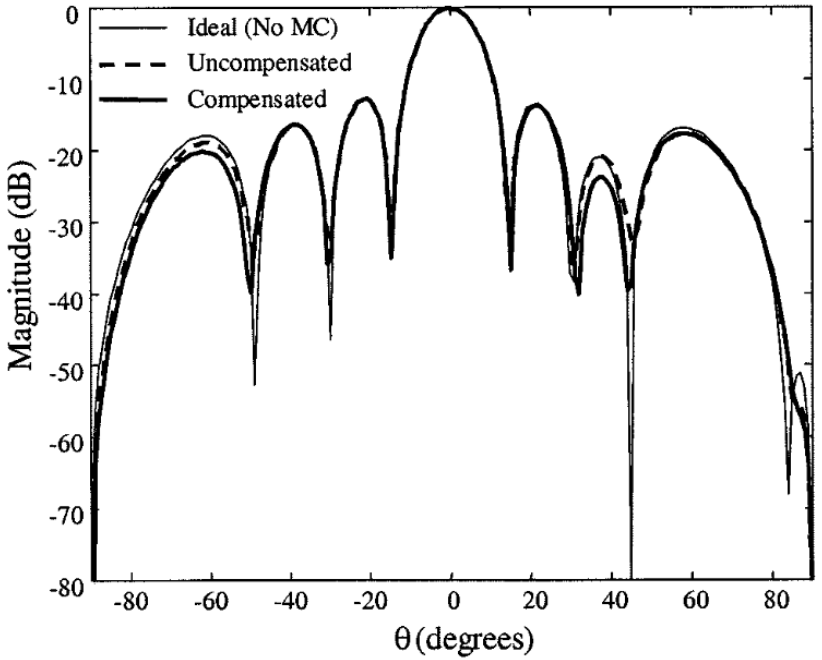
$$\mathbf{R} = \sum_{i=1}^{N} \lambda_i \, \mathbf{q}_i \mathbf{q}_i^H \qquad \text{error along } \mathbf{q}_i \;\propto\; (1 - \mu\lambda_i)^k$$

$$\text{stable:} \;\; 0 < \mu < \frac{2}{\lambda_{\max}} \qquad\qquad \text{slowest:} \;\; \tau \approx \frac{1}{\mu\,\lambda_{\min}} \;\text{snapshots}$$

- A strong interferer makes $\lambda_{\max}$ large, which forces a small $\mu$
- The weak directions (the SOI, the noise) then learn slowly
- The pace is set by the eigenvalue spread, or condition number, of $\mathbf{R}$: $\kappa = \lambda_{\max}/\lambda_{\min}$

# LMS in practice: smart antennas for MANETs

- Bellofiore et al. [4]: smart antennas for mobile ad-hoc network (MANET) devices at 20 GHz
- "The LMS algorithm is an approximation of the steepest descent method using an estimator of the gradient instead of the actual value of the gradient. This simplifies considerably the calculations … and allows the LMS algorithm to be performed in real time."
- That estimator is exactly the one-snapshot update we derived

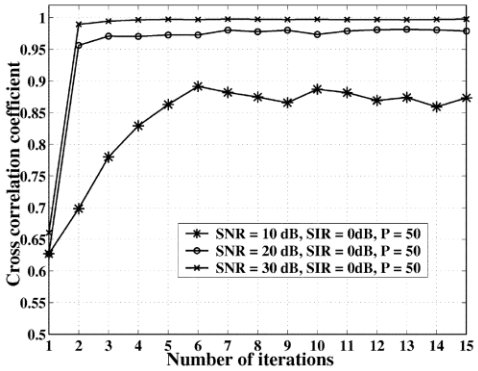



Amplitude patterns of an 8-element microstrip planar array: SOI at 0°, interferer at 45°; LMS places a deep null on the interferer [4].

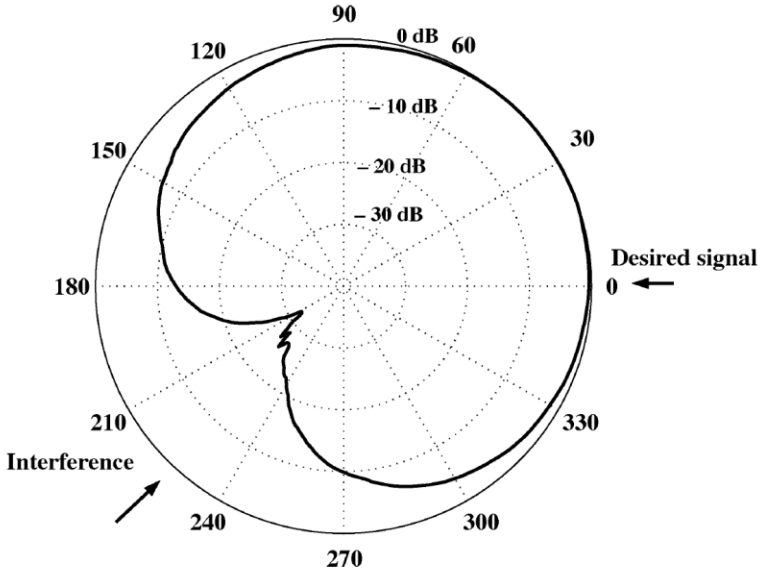
Cite: [4]

# LMS in practice: an ESPAR antenna

- Seven-element monopole array based on an electronically steerable passive array radiator (ESPAR), for wireless ad hoc networks [51]
- LMS was chosen to reduce computation: it reaches the optimum by iterating on the gradient of the mean-square error




LMS learning curves [51]




Measured azimuth pattern: SOI at 0°, interferer at 225° [51].

Cite [51]

# LMS in practice: array calibration

- LMS gradient search on each element's phase shift, so the far-field response matches its ideal [61]
- Goal: "given an initial estimate of the excitation coefficients, how can an iterative approach be implemented so that an optimal set can be produced?"

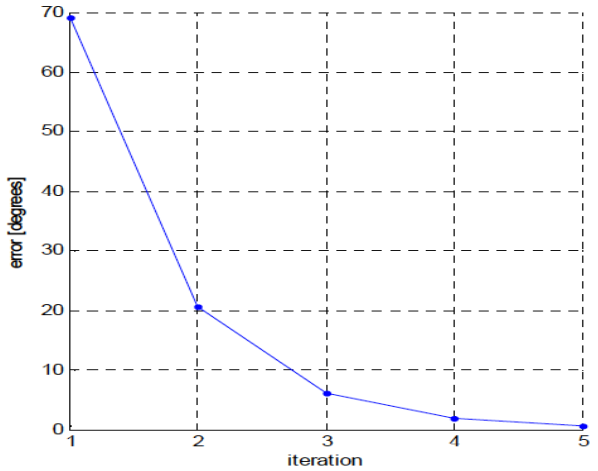



Fig. 1. An example of five iterations for an excitation coefficient.

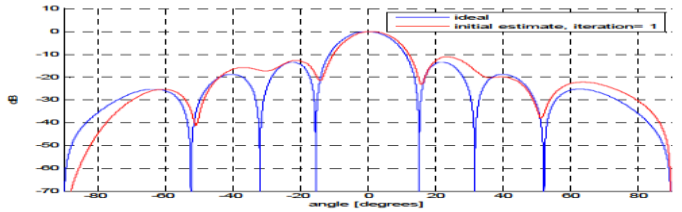



Fig. 2. Iterative calibration process, iteration 1 of 3.

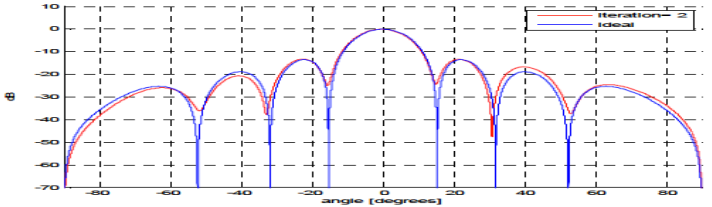



Fig. 3. Iterative calibration process, iteration 2 of 3.

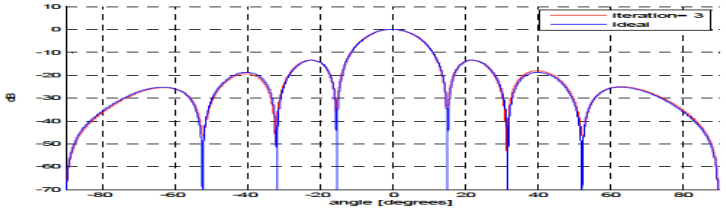



Fig. 4. Iterative calibration process, iteration 3 of 3.

Pattern, calibration iterations 1–3 [61]

Cite: [61]; additional readings [3, 19, 59]

# LMS: what works, and what doesn't

## Good

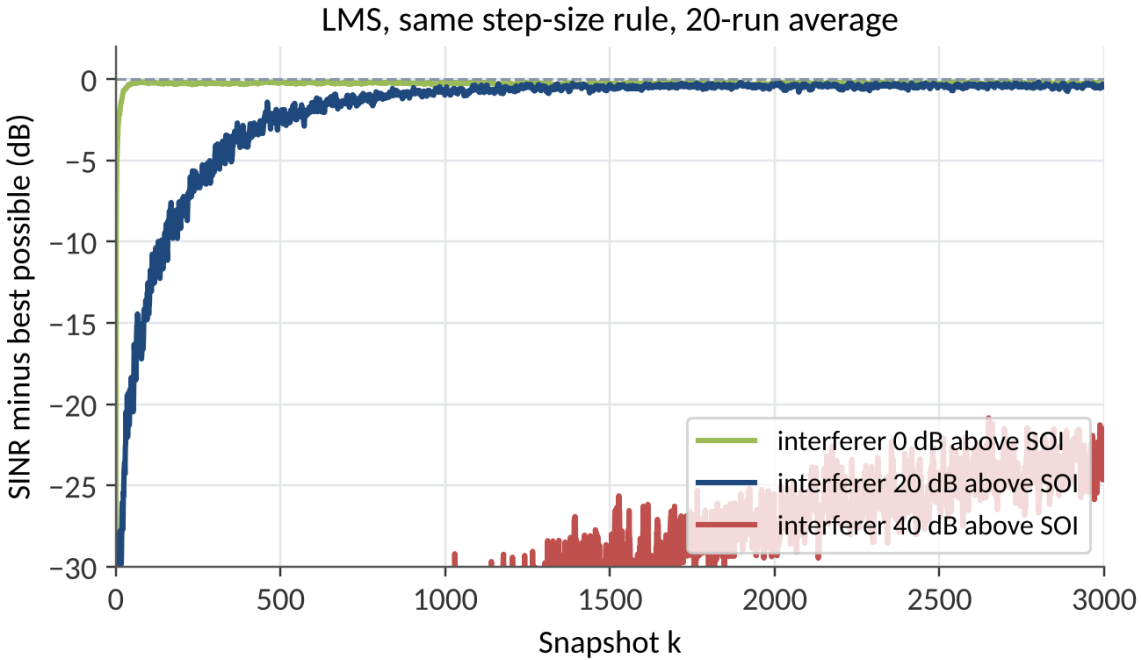
- Nulls interferers it was never told about
- Three lines of code; about 2N multiplies per snapshot
- Tracks a slowly changing scene
- Simple and robust in fixed-point hardware

## Bad

- Needs a known, time-aligned reference
- Slow when the interference is strong (large $\kappa$)
- $\mu$ trades speed against jitter
- Settles near, not at, the Wiener solution

**Can we converge fast, however strong the interferer? → RLS**

# LMS slows down when the interferer is strong



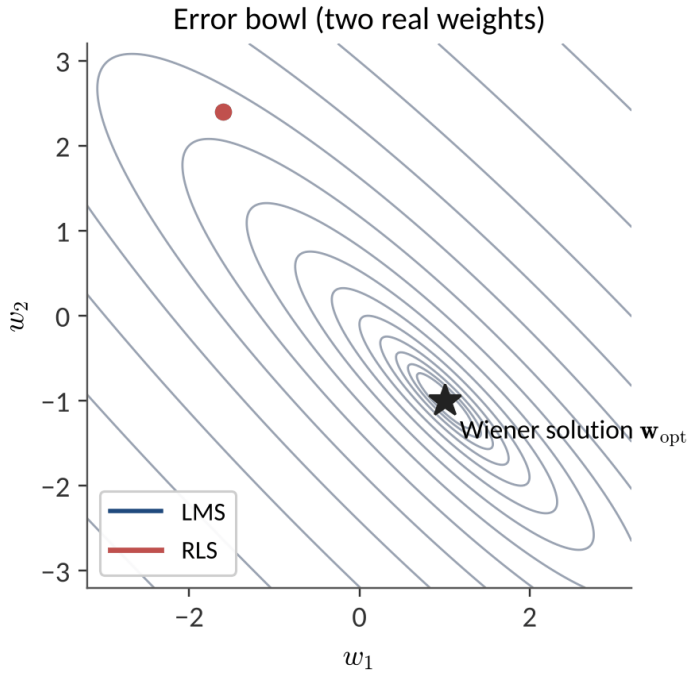

- Same step-size rule, three interferer powers
- The stronger the interferer, the longer LMS takes to reach its best SINR
- At 40 dB it has barely started after 3000 snapshots
- **Jammers are exactly the strong signals we care about**

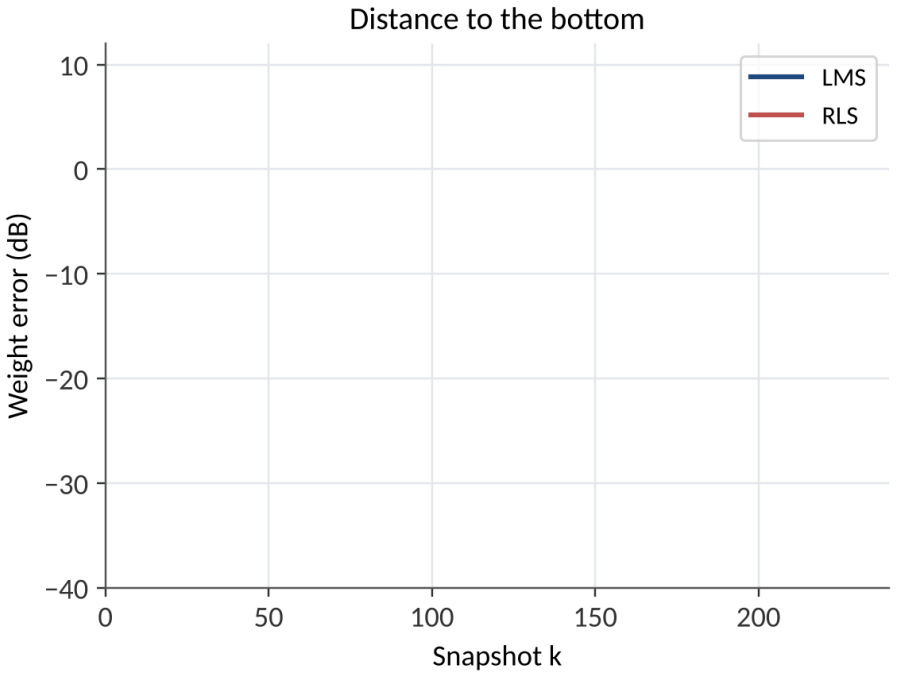
# What RLS assumes

- The same known reference $d(k)$ as LMS
- Older snapshots count less: a forgetting factor $\gamma$, just below 1, discounts each past snapshot
- The scene is steady over the memory, roughly $1/(1-\gamma)$ snapshots
- We can afford about $N^2$ multiplies per snapshot (LMS needed about $2N$)

Most references write the forgetting factor as λ; we use γ because λ already means wavelength and the eigenvalues of R.

# RLS undoes the stretch




Two real weights, eigenvalue spread 19, for illustration. LMS drops into the narrow valley and then crawls along it; RLS rescales the bowl to round and heads for the bottom.

# RLS: four lines per snapshot

$$\mathbf{G}(k) = \frac{\mathbf{P}(k-1)\,\mathbf{x}(k)}{\gamma + \mathbf{x}^H(k)\,\mathbf{P}(k-1)\,\mathbf{x}(k)}$$

gain vector

$$e(k) = d(k) - \mathbf{w}^H(k-1)\,\mathbf{x}(k)$$

error against the reference

$$\mathbf{w}(k) = \mathbf{w}(k-1) + \mathbf{G}(k)\,e^*(k)$$

weight update

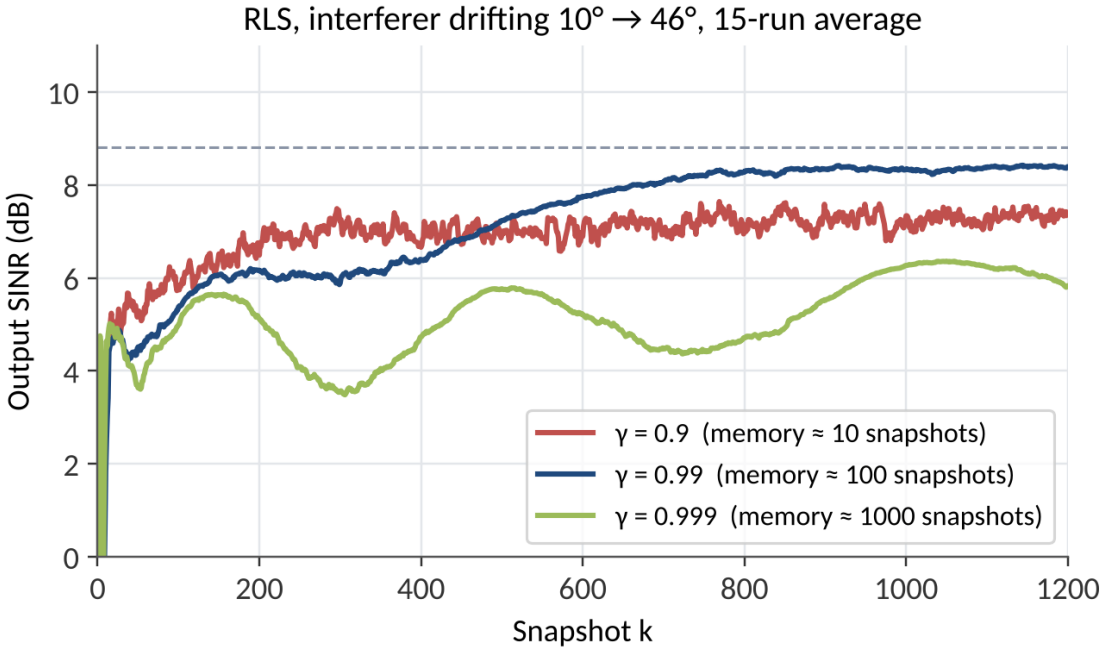
$$\mathbf{P}(k) = \gamma^{-1}[\mathbf{P}(k-1) - \mathbf{G}(k)\,\mathbf{x}^H(k)\,\mathbf{P}(k-1)]$$

running inverse covariance

```
P = eye(N)/delta;  w = zeros(N,1);
for k = 1:K
    x = X(:,k);
    G = P*x / (gamma + x'*P*x);
    e(k) = d(k) - w'*x;
    w = w + G*conj(e(k));
    P = (P - G*(x'*P)) / gamma;
end
```

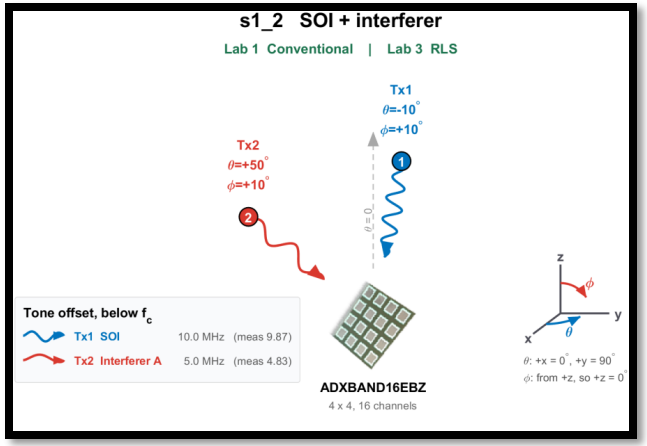
Start: $\mathbf{w}(0) = \mathbf{0}$, $\mathbf{P}(0) = \delta^{-1}\mathbf{I}$ with small $\delta$. $\mathbf{P}(k)$ tracks $\mathbf{R}^{-1}$ without ever inverting.

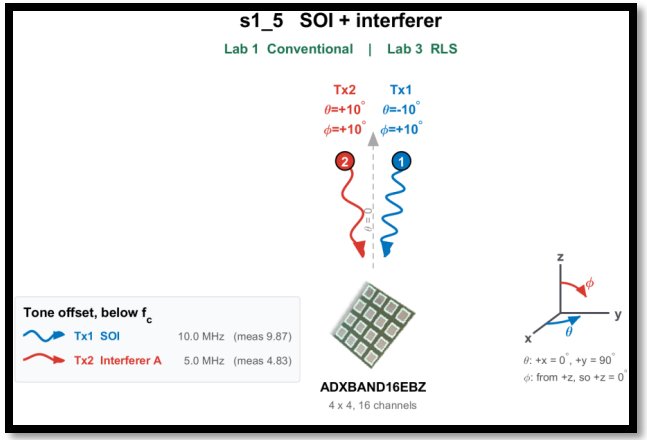
# Choosing the forgetting factor γ

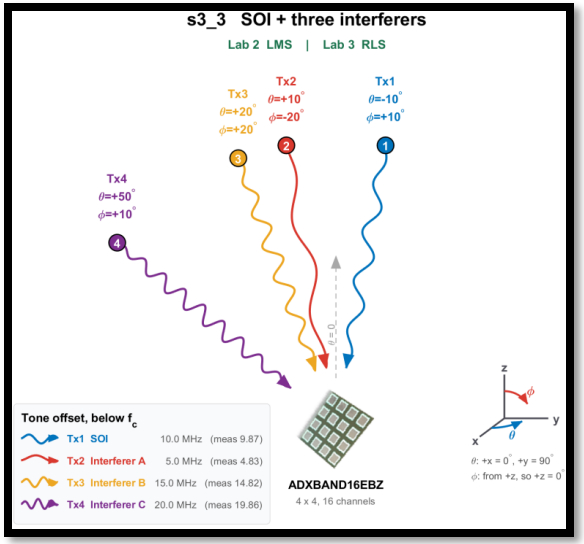



- Memory ≈ $1/(1-\gamma)$ snapshots
- Too short (γ = 0.9): fewer snapshots than 2N in memory, so the weights stay noisy
- Too long (γ = 0.999): remembers where the interferer used to be, and lags
- Typical: 0.99 to 0.999, set by how fast the scene moves

# Lab 2: Location of Signals



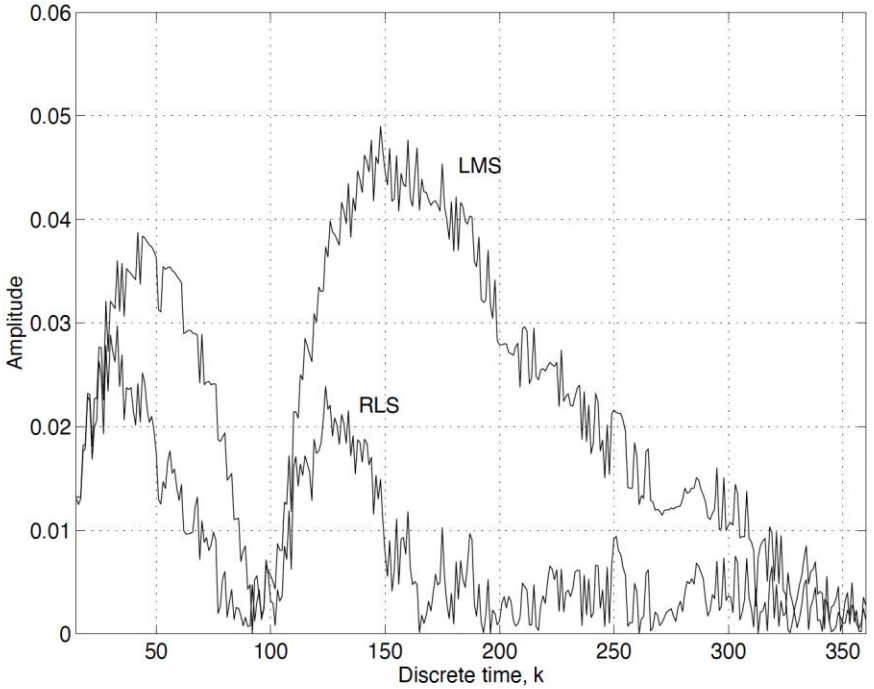



# LAB 3 (Move to MATLAB Online)

# RLS: what it buys you

- RLS is known to converge up to 10× faster than LMS
- Its effective step size changes with time: the gain $\mathbf{G}(k)$
- It minimizes the weighted sum of squared errors between the reference and the output, over all snapshots so far




Error over time: LMS vs RLS.

# Deriving RLS (1/5): the cost

Weight each past error by $\gamma^{k-i}$: recent snapshots count fully, older ones fade ($0 \ll \gamma \leq 1$).

$$J(k) = \sum_{i=1}^{k} \gamma^{k-i} \, |e(i)|^2, \qquad e(i) = d(i) - \mathbf{w}^H \mathbf{x}(i)$$

Exponentially weighted sum of squared errors

$$\frac{\partial J}{\partial \mathbf{w}^*} = -\sum_{i=1}^{k} \gamma^{k-i} \, \mathbf{x}(i) \, e^*(i) = \mathbf{0}$$

Gradient with respect to $\mathbf{w}^*$ ($\partial e / \partial \mathbf{w}^* = -\mathbf{x}$), set to zero

$$\sum_{i=1}^{k} \gamma^{k-i} \, \mathbf{x}(i) \, \mathbf{x}^H(i) \, \mathbf{w} = \sum_{i=1}^{k} \gamma^{k-i} \, \mathbf{x}(i) \, d^*(i)$$

Substitute $e^*(i) = d^*(i) - \mathbf{x}^H(i)\mathbf{w}$, move the $\mathbf{w}$ terms left

$$\widehat{\mathbf{R}}(k) \, \mathbf{w}(k) = \widehat{\mathbf{p}}(k)$$

The Wiener equation, with averages over the data so far

# Deriving RLS (2/5): recursive averages

Each new snapshot adds one term to $\widehat{\mathbf{R}}$ and $\widehat{\mathbf{p}}$; write them in terms of last snapshot's values.

$$\widehat{\mathbf{R}}(k) = \sum_{i=1}^{k-1} \gamma^{k-i}\, \mathbf{x}(i)\, \mathbf{x}^H(i) + \mathbf{x}(k)\, \mathbf{x}^H(k)$$

Split off the newest term ($\gamma^0 = 1$)

$$\widehat{\mathbf{R}}(k) = \gamma \sum_{i=1}^{k-1} \gamma^{k-1-i}\, \mathbf{x}(i)\, \mathbf{x}^H(i) + \mathbf{x}(k)\, \mathbf{x}^H(k)$$

Factor one $\gamma$ out of the older terms

$$\widehat{\mathbf{R}}(k) = \gamma\, \widehat{\mathbf{R}}(k-1) + \mathbf{x}(k)\, \mathbf{x}^H(k)$$

The sum is last snapshot's $\widehat{\mathbf{R}}$

$$\widehat{\mathbf{p}}(k) = \gamma\, \widehat{\mathbf{p}}(k-1) + \mathbf{x}(k)\, d^*(k)$$

Same steps for $\widehat{\mathbf{p}}$

**Key Points:** Creating $\widehat{\mathbf{R}}$ and $\widehat{\mathbf{p}}(k)$ in real-time can be constructed on a snapshot by snapshot basis.

Solving $\mathbf{w}(k) = \widehat{\mathbf{R}}^{-1}(k)\, \widehat{\mathbf{p}}(k)$ directly needs an $N \times N$ inverse, about $N^3$ operations, at every snapshot. Next: update the inverse itself.

# Deriving RLS (3/5): update the inverse

The new $\widehat{\mathbf{R}}$ is the old one plus a rank-one term, and a rank-one change has a convenient inverse.

$$(\mathbf{A}+\mathbf{b}\,\mathbf{b}^H)^{-1}=\mathbf{A}^{-1}-\frac{\mathbf{A}^{-1}\mathbf{b}\,\mathbf{b}^H\mathbf{A}^{-1}}{1+\mathbf{b}^H\mathbf{A}^{-1}\mathbf{b}}$$

Matrix inversion lemma

Use $\mathbf{A}=\gamma\widehat{\mathbf{R}}(k-1)$, $\mathbf{b}=\mathbf{x}(k)$, and name the inverse $\mathbf{P}(k)=\widehat{\mathbf{R}}^{-1}(k)$, so $\mathbf{A}^{-1}=\gamma^{-1}\mathbf{P}(k-1)$

$$\mathbf{P}(k)=\gamma^{-1}\mathbf{P}(k-1)-\frac{\gamma^{-2}\,\mathbf{P}(k-1)\,\mathbf{x}(k)\,\mathbf{x}^H(k)\,\mathbf{P}(k-1)}{1+\gamma^{-1}\,\mathbf{x}^H(k)\,\mathbf{P}(k-1)\,\mathbf{x}(k)}$$

Substitute

$$\mathbf{P}(k)=\gamma^{-1}\left[\mathbf{P}(k-1)-\frac{\mathbf{P}(k-1)\,\mathbf{x}(k)\,\mathbf{x}^H(k)\,\mathbf{P}(k-1)}{\gamma+\mathbf{x}^H(k)\,\mathbf{P}(k-1)\,\mathbf{x}(k)}\right]$$

Multiply the fraction's top and bottom by $\gamma$, take out $\gamma^{-1}$. The denominator is a scalar: no matrix inverse left

# Deriving RLS (4/5): the gain vector

Gain vector

$$\mathbf{G}(k) = \frac{\mathbf{P}(k-1)\,\mathbf{x}(k)}{\gamma + \mathbf{x}^H(k)\,\mathbf{P}(k-1)\,\mathbf{x}(k)}$$

$$\mathbf{P}(k) = \gamma^{-1}[\mathbf{P}(k-1) - \mathbf{G}(k)\,\mathbf{x}^H(k)\,\mathbf{P}(k-1)]$$

the inverse update

Multiply on the right by $\mathbf{x}(k)$

$$\mathbf{P}(k)\,\mathbf{x}(k) = \gamma^{-1}[\mathbf{P}(k-1)\,\mathbf{x}(k) - \mathbf{G}(k)\,\mathbf{x}^H(k)\,\mathbf{P}(k-1)\,\mathbf{x}(k)]$$

From the definition, $\mathbf{P}(k-1)\mathbf{x} = \mathbf{G}(\gamma + \mathbf{x}^H\mathbf{P}(k-1)\mathbf{x})$

$$\mathbf{P}(k)\,\mathbf{x}(k) = \gamma^{-1}\,\mathbf{G}(k)[\gamma + \mathbf{x}^H\mathbf{P}(k-1)\mathbf{x} - \mathbf{x}^H\mathbf{P}(k-1)\mathbf{x}]$$

# Deriving RLS (5/5): the weight update

Start from $\mathbf{w}(k) = \mathbf{P}(k)\,\hat{\mathbf{p}}(k)$ and use the two recursions.

$$\mathbf{w}(k) = \gamma\,\mathbf{P}(k)\,\hat{\mathbf{p}}(k-1) + \mathbf{P}(k)\,\mathbf{x}(k)\,d^*(k)$$

Insert $\hat{\mathbf{p}}(k) = \gamma\hat{\mathbf{p}}(k-1) + \mathbf{x}(k)d^*(k)$

$$\gamma\,\mathbf{P}(k)\,\hat{\mathbf{p}}(k-1) = \mathbf{w}(k-1) - \mathbf{G}(k)\,\mathbf{x}^H(k)\,\mathbf{w}(k-1)$$

$\mathbf{P}$ update times $\hat{\mathbf{p}}(k-1)$; $\mathbf{P}(k-1)\hat{\mathbf{p}}(k-1) = \mathbf{w}(k-1)$

$$\mathbf{w}(k) = \mathbf{w}(k-1) + \mathbf{G}(k)[d^*(k) - \mathbf{x}^H(k)\,\mathbf{w}(k-1)]$$

Put both back, with $\mathbf{P}(k)\mathbf{x}(k) = \mathbf{G}(k)$

$$\mathbf{w}(k) = \mathbf{w}(k-1) + \mathbf{G}(k)\,e^*(k)$$

The bracket is $e^*(k)$, with $e(k) = d(k) - \mathbf{w}^H(k-1)\mathbf{x}(k)$: the error using the old weights

Four lines per snapshot, $\mathbf{G}$, $e$, $\mathbf{w}$, $\mathbf{P}$: about $N^2$ operations instead of $N^3$. Start with $\mathbf{w}(0) = \mathbf{0}$ and $\mathbf{P}(0) = \delta^{-1}\mathbf{I}$.

# Why RLS is fast

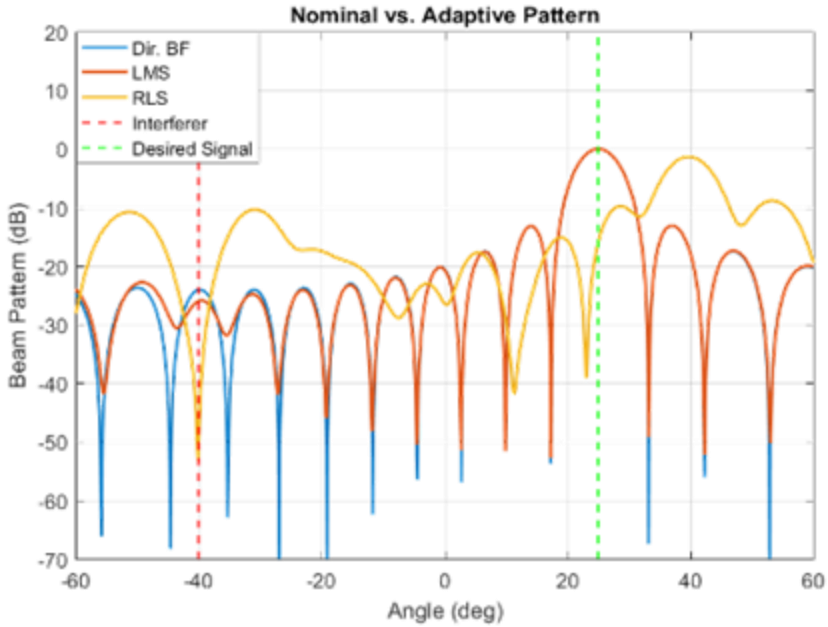
LMS steps along $\mathbf{x}(k)$ RLS steps along $\mathbf{G}(k) = \mathbf{P}(k)\,\mathbf{x}(k) \approx \widehat{\mathbf{R}}^{-1}\mathbf{x}(k)$

- $\widehat{\mathbf{R}}^{-1}$ rescales every direction to the same curvature: the bowl becomes round (Newton's method)
- Converges in roughly 2N snapshots, whatever the condition number κ
- $\mathbf{P}(0) = \delta^{-1}\mathbf{I}$ acts like a small diagonal load that fades as data arrive
- $\gamma < 1$ lets it track; $\gamma = 1$ is the growing-window least squares of the original derivation
- Hardware uses QR-decomposition forms of RLS for numerical robustness

# LMS and RLS in the IMPACT study

- Beampatterns from LMS and RLS in the IMPACT trade study
- Scene: SOI at 25°, interferer at −40°, SIR of −40 dB, no noise
- Both reduce the interferer's contribution in favor of the SOI

Key point: in this study, LMS and RLS were not very effective against the interferer.

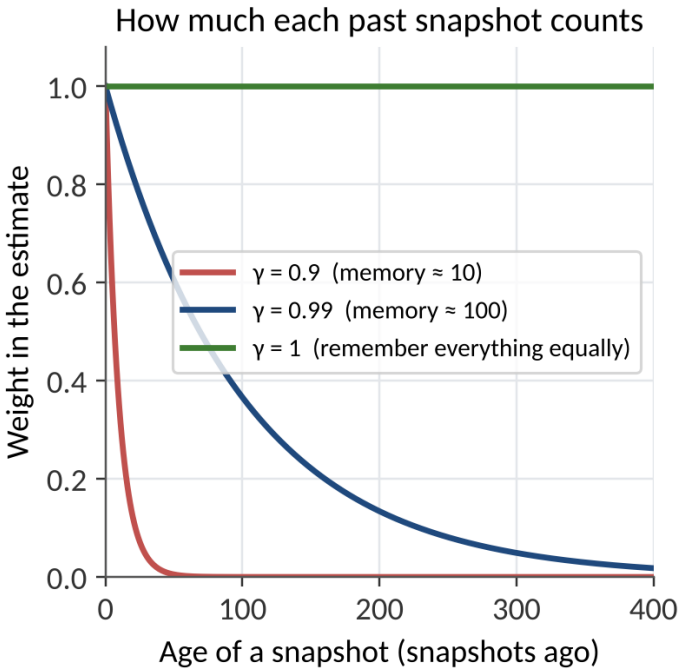


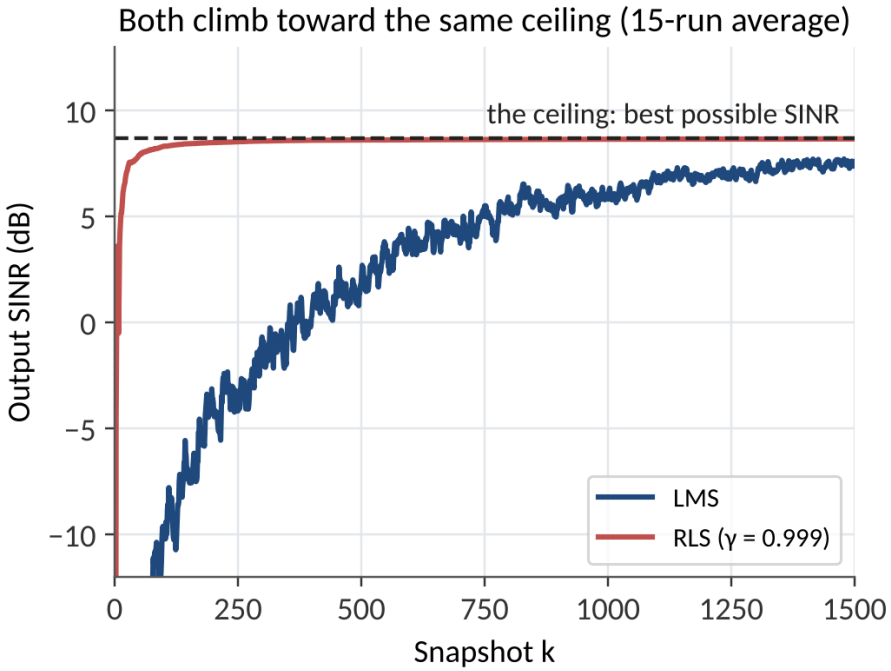

LMS (dark red) and RLS (yellow) vs the nominal pattern; interferer red dashed, SOI green dashed.

Ref: [20–22]

# What if we remember every snapshot?

- RLS discounts a snapshot that is $i$ snapshots old by $\gamma^i$.
- LMS and RLS both climb toward the same ceiling. Can we compute it directly, and without a reference signal?




Same scene as Module 2: N = 8, SOI at 0°, two interferers 20 dB above the noise at −40° and +22°. Right: 15-run average.

# RLS: what works, and what doesn't

## Good

- Converges in about 2N snapshots, however strong the interferer
- Tracks a moving scene through γ
- Still needs no direction information

## Bad

- About $N^2$ multiplies per snapshot (LMS: about 2N)
- Numerical care: P can lose symmetry; QR-based forms fix this
- Choosing γ (and δ)
- Still needs a known, time-aligned reference

**Both chase $w = R^{-1}p$. How good is that answer, and can we compute it directly? → Module 4**

# Module 4: MVDR — How Good Can It Get?

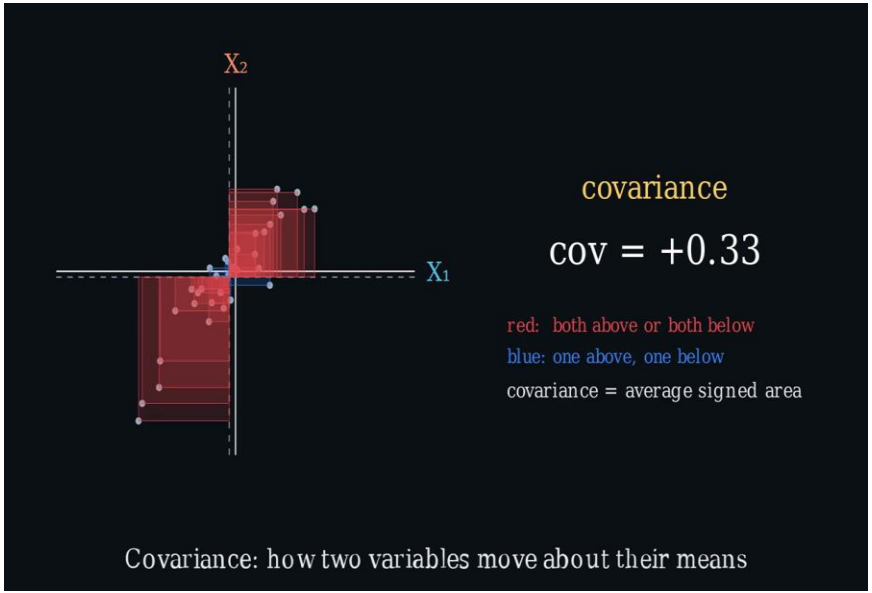
No reference signal: just the look direction and the data

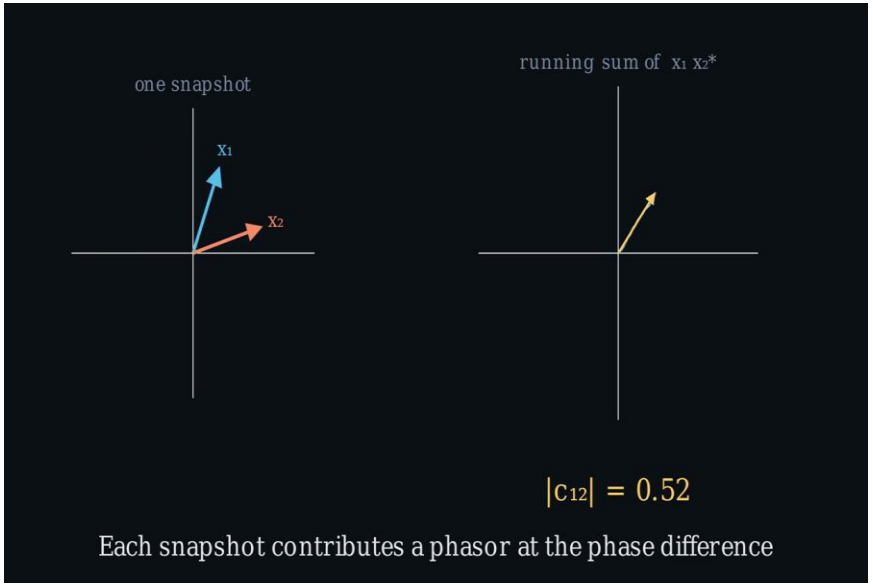
# What MVDR assumes

- No reference $d(k)$: we know only the direction of the SOI, $\theta_0$, and so its steering vector $\mathbf{s}(\theta_0)$
- The array is calibrated, so $\mathbf{s}(\theta_0)$ is accurate
- The covariance $\mathbf{R} = E[\mathbf{x}(k)\,\mathbf{x}^H(k)]$ is known exactly: the $\gamma = 1$, infinite-memory limit. In practice we estimate it from $K$ snapshots
- The scene is steady over those $K$ snapshots
- The SOI is uncorrelated with the interference (no delayed copies of the SOI, such as multipath)
- We can afford about $N^3$ operations per weight update

# A quick detour: covariance

**R** has been in every Wiener equation since Module 2. What does one entry of it measure? (Click a clip to play.)

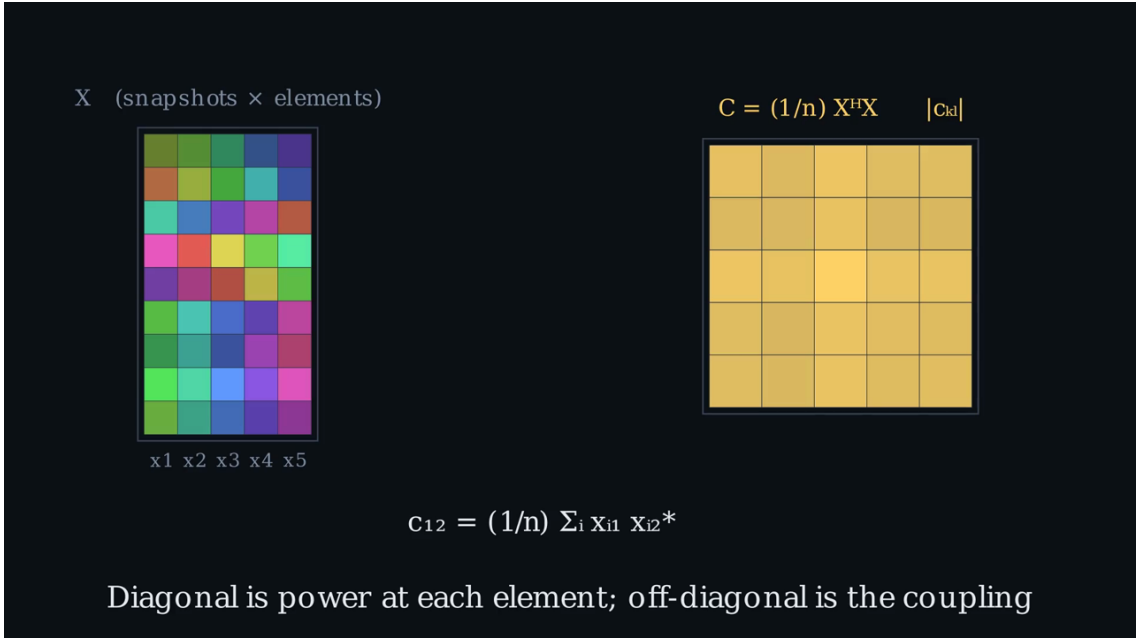



Two real signals: covariance is the average signed area of their products about the means.

Two complex elements: each snapshot adds the phasor $x_1 x_2^*$. A wavefront makes them add up; independent noise makes them cancel.

# Covariance across the array



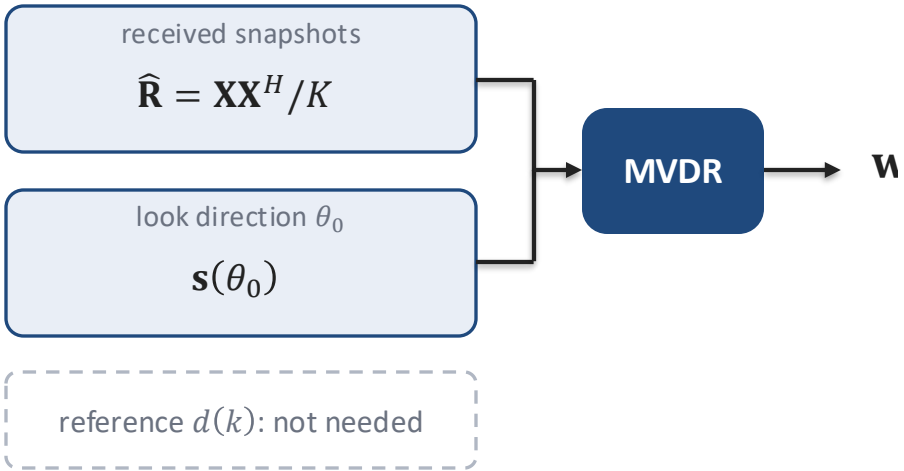

Click to play.

$$\hat{\mathbf{R}} = \frac{1}{K}\sum_{k=1}^{K}\mathbf{x}(k)\,\mathbf{x}^{H}(k) = \frac{1}{K}\mathbf{X}\mathbf{X}^{H}$$

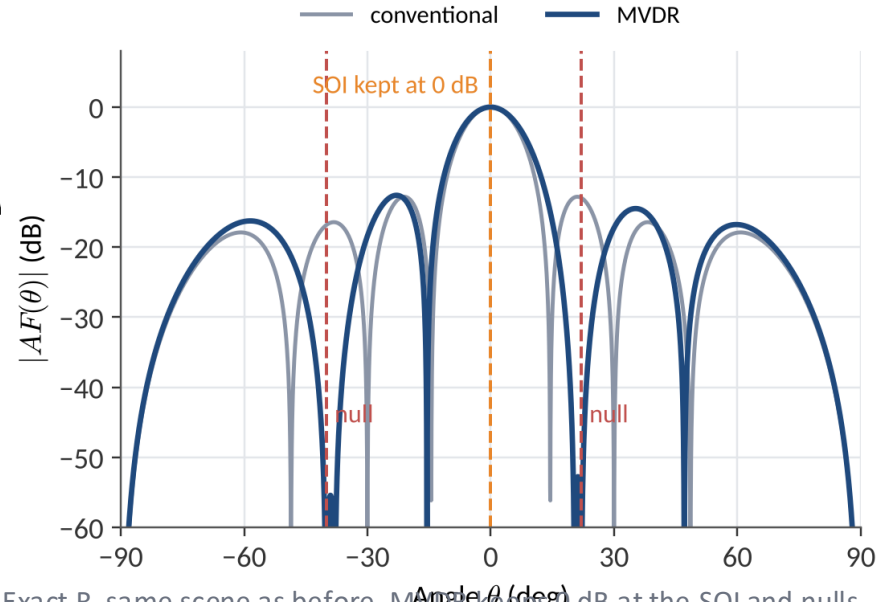
$$\hat{r}_{mn} = \frac{1}{K}\sum_{k=1}^{K} x_m(k)\,x_n^{*}(k)$$

- Diagonal: the power at each element
- Off-diagonal: how strongly two elements move together
- One plane wave: the phase steps by $\psi$ along each diagonal. That structure is what MVDR exploits

# What MVDR needs




No reference, no step size, no forgetting factor: one solve per block of K snapshots.




Exact R, same scene as before. MVDR keeps 0 dB at the SOI and nulls both interferers; the sidelobes move to wherever there is nothing to suppress.

# The MVDR problem

Output power, for any weights (from $y(k) = \mathbf{w}^H \mathbf{x}(k)$):

$$P = E[|y(k)|^2] = \mathbf{w}^H \mathbf{R}\, \mathbf{w}$$

$$\min_{\mathbf{w}} \mathbf{w}^H \mathbf{R}\, \mathbf{w} \qquad \text{subject to} \qquad \mathbf{w}^H \mathbf{s}(\theta_0) = 1$$

- Distortionless: $AF(\theta_0) = \mathbf{w}^H \mathbf{s}(\theta_0) = 1$, so the SOI passes with unit gain, whatever its power
- Minimum variance: make the total output power as small as possible
- The SOI's share of the output is fixed by the constraint, so the only power left to remove is interference and noise

# MVDR: the solution

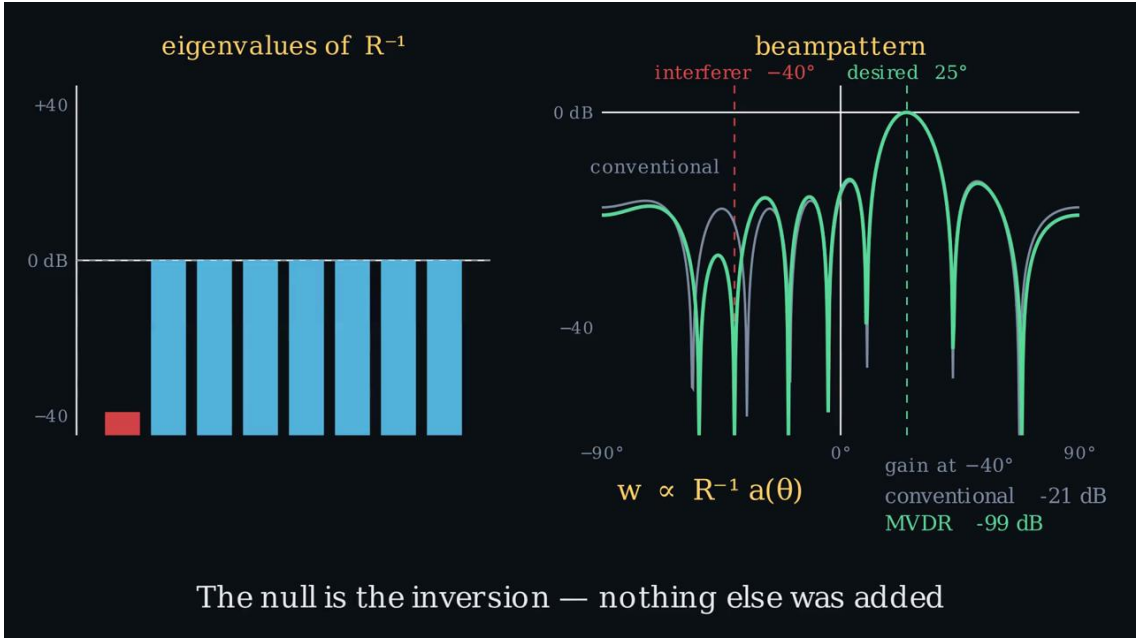
$$\mathbf{w}_{\mathrm{MVDR}} = \frac{\mathbf{R}^{-1}\mathbf{s}(\theta_0)}{\mathbf{s}^H(\theta_0)\,\mathbf{R}^{-1}\mathbf{s}(\theta_0)}$$

- $\mathbf{R}^{-1}$: turns gain down in the directions where the power is
- $\mathbf{s}(\theta_0)$: points the beam at the SOI
- Denominator: rescales so that $AF(\theta_0) = 1$
- Derivation after Lab 5

```
% X: N-by-K snapshots
% d: spacing (wavelengths), theta0: SOI (deg)
psi = 2*pi*d*sind(theta0);
s = exp(-1j*(0:N-1).'*psi);  % steering
R = (X*X')/K;                % sample covariance
w = (R\s)/(s'*(R\s));        % MVDR weights
y = w'*X;                    % output
```

R\s solves $\mathbf{Ru} = \mathbf{s}$ without forming $\mathbf{R}^{-1}$.

# Why the inverse makes the null



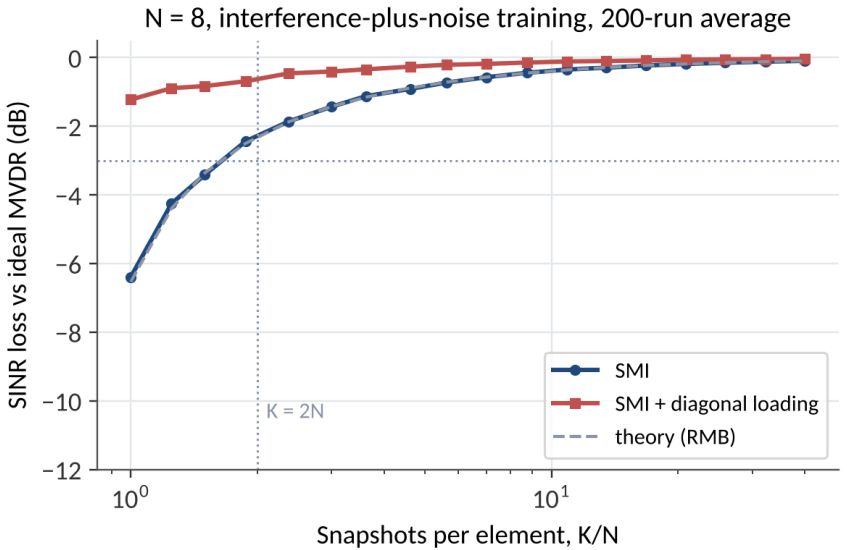

Click to play.

- A strong interferer gives $\mathbf{R}$ one large eigenvalue $\lambda_{\max}$, along its steering vector
- $\mathbf{R}^{-1}$ has the same eigenvectors with eigenvalues $1/\lambda_i$: that direction is now the weakest
- So $\mathbf{R}^{-1}\mathbf{s}(\theta_0)$ keeps almost nothing along the interferer: a null, with nothing else added
- Diagonal loading (next) puts a floor under the eigenvalues before the inverse sees them

# In practice: SMI and diagonal loading

$$\hat{\mathbf{R}} = \frac{1}{K}\mathbf{X}\mathbf{X}^H \qquad \Rightarrow \qquad \hat{\mathbf{R}} + \beta\,\mathbf{I}$$

- Sample matrix inversion (SMI): use $\hat{\mathbf{R}}$ in place of $\mathbf{R}$. Needs $K \geq N$ to be invertible
- Rule of thumb (Reed, Mallett, Brennan): $K \approx 2N$ snapshots keeps you within about 3 dB of the ideal
- Diagonal loading adds $\beta$ to every eigenvalue: spread $(\lambda_{\max} + \beta)/(\lambda_{\min} + \beta)$, and far fewer snapshots needed
- Typical $\beta$: a few times the noise power per element



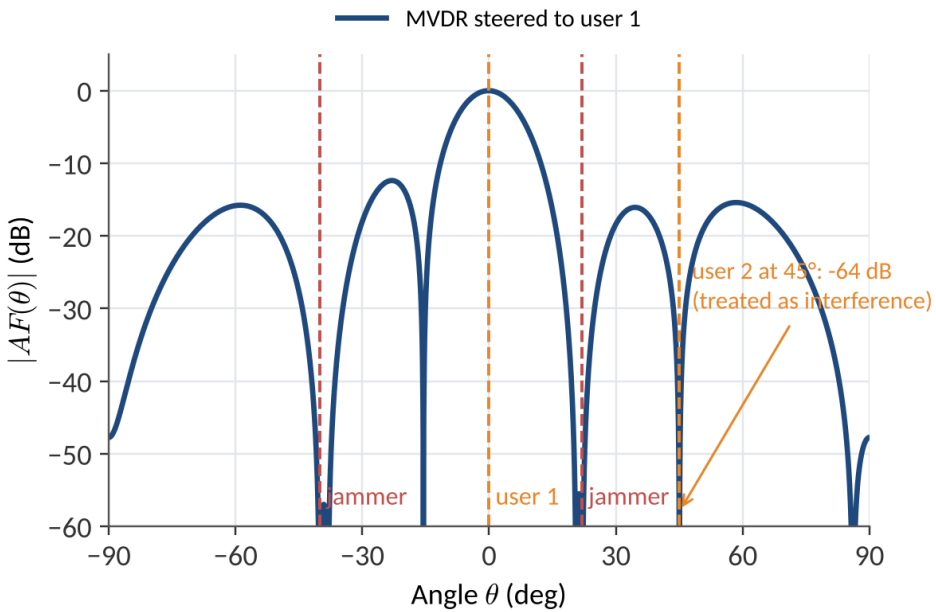

Training data without the SOI. With the SOI in the data, small errors in $s(\theta_0)$ hurt more, and loading matters even more.

```
R = (X*X')/K + beta*eye(N);
```

# Module 5: LCMV — Several Constraints

Keep several directions, force nulls, and let the rest adapt

# What if there are two users?

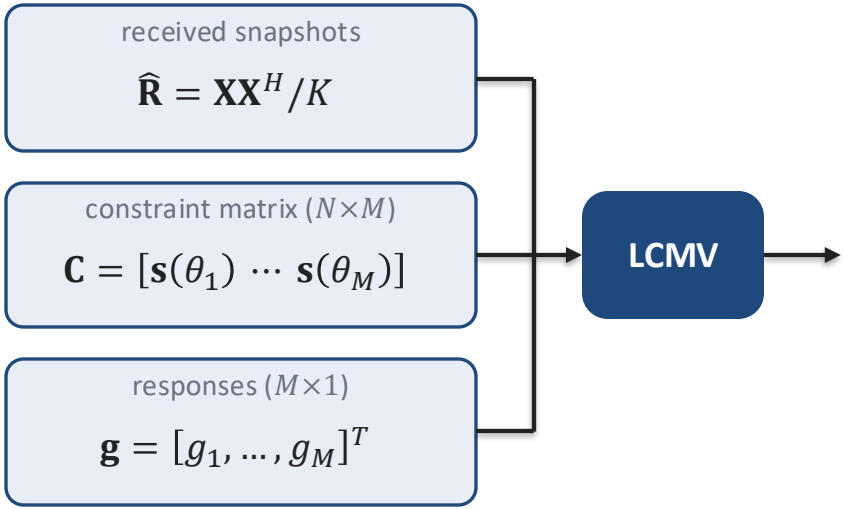


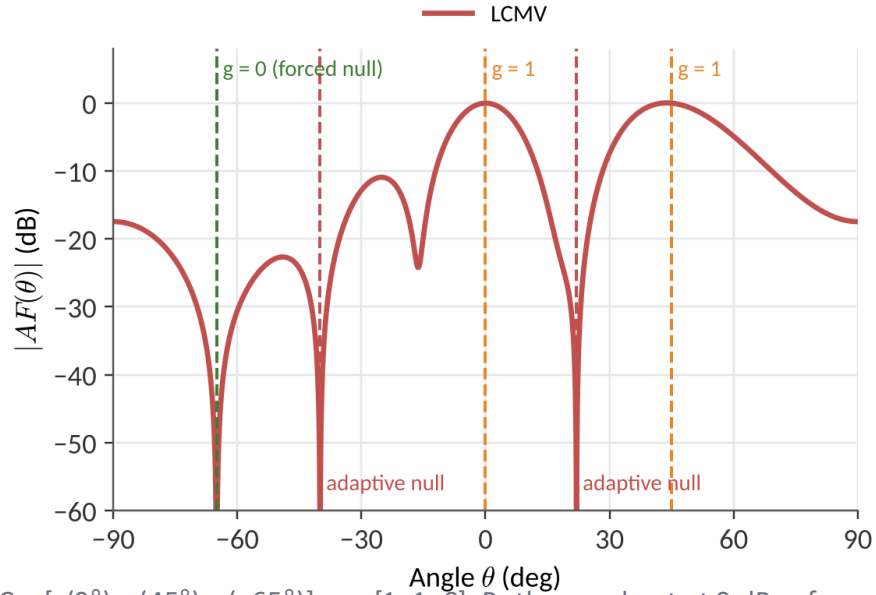

- MVDR protects exactly one direction. Everything else with power in it is treated as interference
- A second user at 45° (10 dB above the noise) is nulled to −64 dB
- We may also want a null where we know a jammer will appear, before it does
- We need more than one constraint

# What LCMV assumes

- Everything MVDR assumes
- $M$ directions $\theta_1, \ldots, \theta_M$ are known, each with a desired response: 1 to keep, 0 to null
- $M < N$: each constraint uses one of the $N$ weights, leaving $N - M$ to adapt against interference
- The cost is still about $N^3$ operations per update

# What LCMV needs




C = [s(0°), s(45°), s(−65°)], g = [1, 1, 0]. Both users kept at 0 dB, a forced null at −65°, and adaptive nulls on both interferers.

# The LCMV problem and solution

$$\min_{\mathbf{w}} \mathbf{w}^H \mathbf{R}\,\mathbf{w} \qquad \text{subject to} \qquad \mathbf{C}^H\mathbf{w} = \mathbf{g}$$

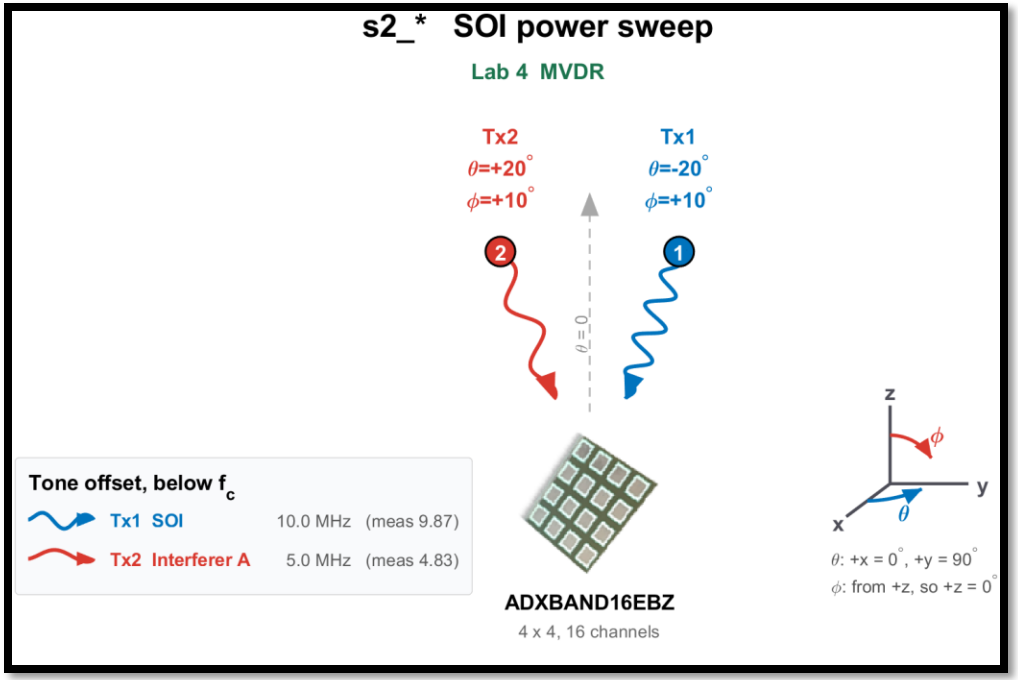
One equation per constraint: $\mathbf{s}^H(\theta_m)\,\mathbf{w} = g_m$, i.e. $AF(\theta_m) = g_m$ for real $g_m$.

$$\mathbf{w}_{\text{LCMV}} = \mathbf{R}^{-1}\mathbf{C}(\mathbf{C}^H\mathbf{R}^{-1}\mathbf{C})^{-1}\mathbf{g}$$

- $M = 1$, $\mathbf{C} = \mathbf{s}(\theta_0)$, $g = 1$: MVDR again
- $\mathbf{C}^H\mathbf{R}^{-1}\mathbf{C}$ is only $M \times M$
- Derivation after Lab 5

```
C = [s1, s2, s3];      % N-by-M
g = [1; 1; 0];         % keep, keep, null
R = (X*X')/K + beta*eye(N);
RC = R\C;
w = RC*((C'*RC)\g);    % LCMV weights
```

# Lab 4: Location of Signals



# LAB 4 (Move to MATLAB Online)

# Where MVDR comes from (1/2)

Minimize the output power while keeping unit gain toward the SOI. Write $\mathbf{s} = \mathbf{s}(\theta_0)$.

$$\min_{\mathbf{w}} \mathbf{w}^H \mathbf{R}\, \mathbf{w} \qquad \text{subject to} \qquad \mathbf{w}^H \mathbf{s} = 1$$

The MVDR problem: find weights that minimize the output power of the beamformer, while the constraint here ensures distortionless response in the SOI direction.

$$J(\mathbf{w}, \nu) = \mathbf{w}^H \mathbf{R}\, \mathbf{w} + \nu(1 - \mathbf{w}^H \mathbf{s}) + \nu^*(1 - \mathbf{s}^H \mathbf{w})$$

Add the constraint with a Lagrange multiplier $\nu$; it is complex, so add its conjugate too and $J$ stays real

$$\frac{\partial J}{\partial \mathbf{w}^*} = \mathbf{R}\, \mathbf{w} - \nu\, \mathbf{s} = \mathbf{0}$$

Gradient with respect to $\mathbf{w}^*$, set to zero (the $\nu^*$ term has no $\mathbf{w}^*$)

$$\mathbf{w} = \nu\, \mathbf{R}^{-1} \mathbf{s}$$

Solve for $\mathbf{w}$: the direction is set; $\nu$ only scales it

**Key Point**: $J$ is the cost function. We minimize the cost function with respect to $\mathbf{w}$ with the Lagrangian minimization approach.

# Where MVDR comes from (2/2)

Pick $\nu$ so the constraint holds.

$$\mathbf{s}^H\mathbf{w} = \nu\,\mathbf{s}^H\mathbf{R}^{-1}\mathbf{s} = 1$$

Put $\mathbf{w}$ into the constraint ($\mathbf{s}^H\mathbf{w} = 1$ is its conjugate)

$$\nu = \frac{1}{\mathbf{s}^H\mathbf{R}^{-1}\mathbf{s}}$$

$\mathbf{s}^H\mathbf{R}^{-1}\mathbf{s}$ is real and positive, so $\nu$ is too

$$\mathbf{w}_{\text{MVDR}} = \frac{\mathbf{R}^{-1}\mathbf{s}}{\mathbf{s}^H\mathbf{R}^{-1}\mathbf{s}}$$

Substitute back

$$\boldsymbol{P}_{\mathbf{min}}(\boldsymbol{\theta}) = \mathbf{w}_{\text{MVDR}}^H\mathbf{R}\,\mathbf{w}_{\text{MVDR}} = \frac{\mathbf{s}^H\mathbf{R}^{-1}\mathbf{R}\,\mathbf{R}^{-1}\mathbf{s}}{(\mathbf{s}^H\mathbf{R}^{-1}\mathbf{s})^2} = \frac{1}{\mathbf{s}^H\mathbf{R}^{-1}\mathbf{s}}$$

The power profile as a function of angle, i.e., the spectrum.

Sweep $\theta_0$ and plot $P_{\min}(\theta_0)$: the Capon (MVDR) power spectrum, the basis of the radar images in the examples below.

# MVDR is the Wiener solution, rescaled

Split the snapshot into the SOI, with waveform $d(k)$ and power $\sigma_d^2 = E|d(k)|^2$, and the rest:

$$\mathbf{x}(k) = \mathbf{s}\, d(k) + \mathbf{x}_{i+n}(k), \qquad \mathbf{R} = \sigma_d^2\, \mathbf{s}\mathbf{s}^H + \mathbf{R}_{i+n}$$

$\mathbf{R}_{i+n}$: interference-plus-noise covariance. With the SOI uncorrelated with the rest, $\mathbf{p} = E[\mathbf{x}(k)\, d^*(k)] = \sigma_d^2\, \mathbf{s}$:

$$\mathbf{w}_{\text{opt}} = \mathbf{R}^{-1}\mathbf{p} = \sigma_d^2\, \mathbf{R}^{-1}\mathbf{s} \;\propto\; \mathbf{w}_{\text{MVDR}}$$

The matrix inversion lemma (Module 3) shows that the SOI only rescales the answer:

$$\mathbf{R}^{-1}\mathbf{s} = \frac{\mathbf{R}_{i+n}^{-1}\mathbf{s}}{1 + \sigma_d^2\, \mathbf{s}^H\mathbf{R}_{i+n}^{-1}\mathbf{s}}, \qquad \text{SINR}_{\max} = \sigma_d^2\, \mathbf{s}^H\mathbf{R}_{i+n}^{-1}\mathbf{s}$$

Same direction as the Wiener weights, so the same SINR: $\text{SINR}_{\max}$ is the ceiling LMS and RLS were climbing toward. MVDR reaches it with no reference, by knowing $\mathbf{s}(\theta_0)$ instead.

# Where LCMV comes from

Replace the one constraint by $M$ of them, $\mathbf{C}^H\mathbf{w} = \mathbf{g}$ with $\mathbf{C} = [\mathbf{s}_1 \cdots \mathbf{s}_M]$, and repeat the same steps.

| | MVDR: one constraint | LCMV: $M$ constraints |
|---|---|---|
| Constraint | $\mathbf{w}^H\mathbf{s} = 1$ | $\mathbf{C}^H\mathbf{w} = \mathbf{g}$ |
| Lagrangian | $J = \mathbf{w}^H\mathbf{R}\mathbf{w} + \nu(1 - \mathbf{w}^H\mathbf{s})$ | $J = \mathbf{w}^H\mathbf{R}\mathbf{w} + \boldsymbol{\nu}^H(\mathbf{g} - \mathbf{C}^H\mathbf{w})$ |
| Gradient = 0 | $\mathbf{w} = \nu\,\mathbf{R}^{-1}\mathbf{s}$ | $\mathbf{w} = \mathbf{R}^{-1}\mathbf{C}\,\boldsymbol{\nu}$ |
| Multiplier | $\nu = (\mathbf{s}^H\mathbf{R}^{-1}\mathbf{s})^{-1}$ | $\boldsymbol{\nu} = (\mathbf{C}^H\mathbf{R}^{-1}\mathbf{C})^{-1}\mathbf{g}$ |
| Weights | $\mathbf{w}_{\text{MVDR}} = \dfrac{\mathbf{R}^{-1}\mathbf{s}}{\mathbf{s}^H\mathbf{R}^{-1}\mathbf{s}}$ | $\mathbf{w}_{\text{LCMV}} = \mathbf{R}^{-1}\mathbf{C}(\mathbf{C}^H\mathbf{R}^{-1}\mathbf{C})^{-1}\mathbf{g}$ |

$M = 1$, $\mathbf{C} = \mathbf{s}$, $\mathbf{g} = 1$ gives MVDR back. $\mathbf{C}^H\mathbf{R}^{-1}\mathbf{C}$ is only $M \times M$. The $M$ constraints use $M$ degrees of freedom; $N - M$ remain for adaptive nulls.

Key Point: LCMV is similar to MVDR, with C is a collection of SOI steering vectors.

# Where diagonal loading comes from

Large weights amplify anything uncorrelated across the elements: element noise, calibration error, a slightly wrong $\mathbf{s}(\theta_0)$. So also cap $\| \mathbf{w} \|^2$:

$$\min_{\mathbf{w}} \mathbf{w}^H \mathbf{R}\, \mathbf{w} \quad \text{subject to} \quad \mathbf{w}^H \mathbf{s} = 1, \quad \| \mathbf{w} \|^2 \leq T$$

The norm constraint adds a second multiplier $\beta \geq 0$, i.e. a penalty $\beta\, \mathbf{w}^H \mathbf{w}$, so $\mathbf{R}$ becomes $\mathbf{R} + \beta \mathbf{I}$:

$$\min_{\mathbf{w}} \mathbf{w}^H (\mathbf{R} + \beta \mathbf{I}) \mathbf{w} \quad \text{s.t.} \quad \mathbf{w}^H \mathbf{s} = 1 \quad \Rightarrow \quad \mathbf{w} = \frac{(\mathbf{R} + \beta \mathbf{I})^{-1} \mathbf{s}}{\mathbf{s}^H (\mathbf{R} + \beta \mathbf{I})^{-1} \mathbf{s}}$$

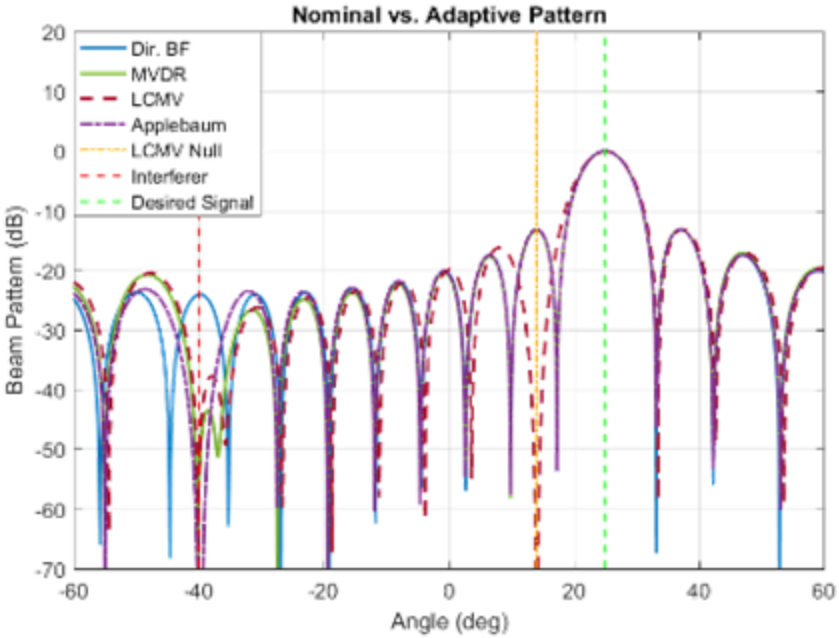
- Every eigenvalue moves up by $\beta$: the spread becomes $\kappa = (\lambda_{\max} + \beta)/(\lambda_{\min} + \beta)$
- $\beta \to 0$: MVDR. $\beta \to \infty$: $\mathbf{w} \to \mathbf{s}/N$, the conventional beam
- Robust Capon (next examples) chooses $\beta$ from how uncertain $\mathbf{s}(\theta_0)$ is

**Key Point**: Diagonal loading is implemented to create stable inverse calculations, yet nulls may be as deep.

# MVDR and LCMV in the IMPACT study

- Same scene as the LMS/RLS study: 1×16 array, SOI at 25°, interferer at −40°, SIR of −40 dB, no noise
- LCMV asked for an extra null at 14°, to show it can place nulls anywhere
- Applebaum: another covariance-based method, not covered here

Key point: MVDR gave the best response with the least prior information and the least complexity.

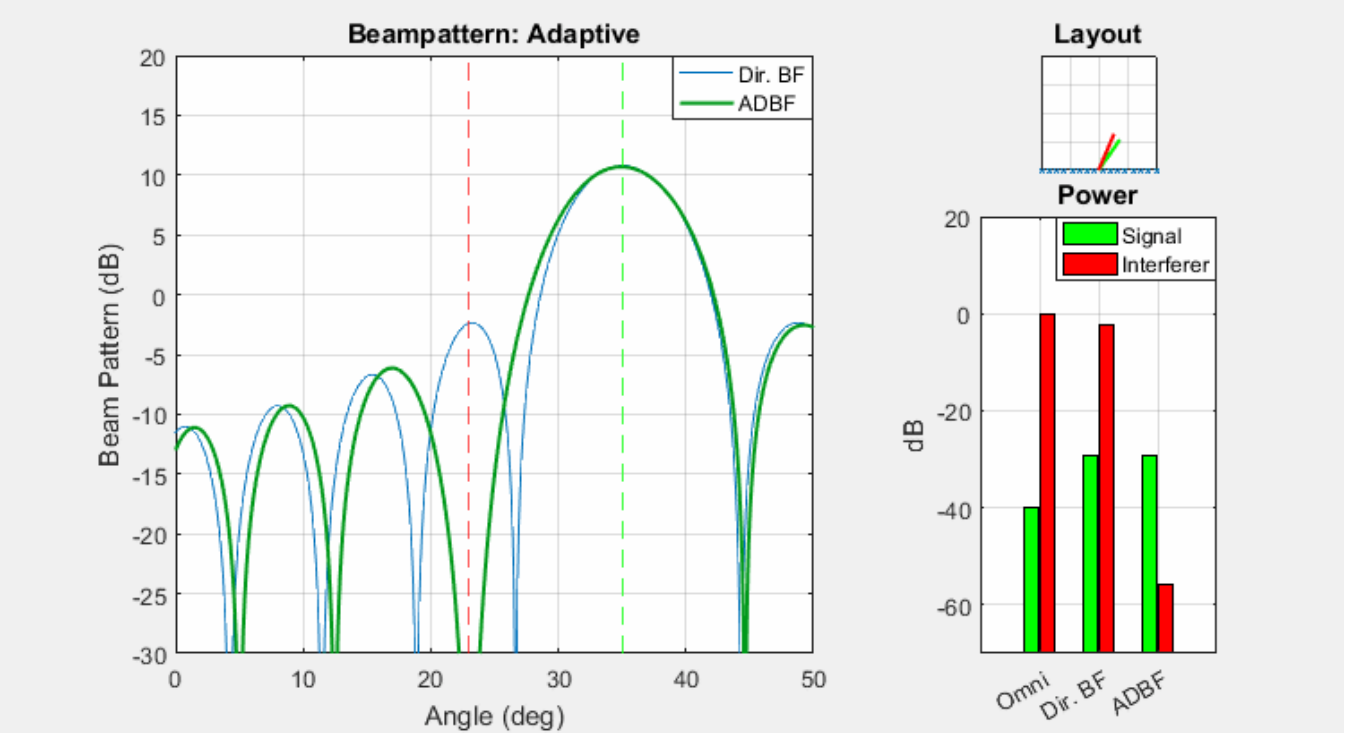



Applebaum (purple), MVDR (green), LCMV (maroon) vs the nominal pattern; interferer red dashed, SOI green dashed, LCMV null orange dashed.

Cite: [20–22]

# MVDR vs Fourier: a sweeping interferer

- 1×16 array
- One interferer, sweeping in azimuth
- Interferer 40 dB stronger than the SOI (SIR = −40 dB)




Dir. BF: Fourier (conventional) beamforming. ADBF: MVDR. Right: SOI and interferer power at an omni element and after each beamformer.

Cite: [20–22]

# Capon and Robust Capon

## Capon beamforming = MVDR

$$\min_{\mathbf{w}} \mathbf{w}^H \widehat{\mathbf{R}} \mathbf{w} \quad \text{s.t.} \quad \mathbf{w}^H \mathbf{s}(\theta_0) = 1 \quad \Rightarrow \quad \mathbf{w}_C = \frac{\widehat{\mathbf{R}}^{-1}\mathbf{s}(\theta_0)}{\mathbf{s}^H(\theta_0)\,\widehat{\mathbf{R}}^{-1}\mathbf{s}(\theta_0)}$$

Capon minimizes the power everywhere except the look direction. If the true steering vector differs from $s(\theta_o)$, the SOI itself gets suppressed.

## Robust Capon beamforming (RCB) [Li, Stoica, Wang 2003]

Let the true steering vector be anywhere within $\epsilon$ of the nominal one, $\bar{\mathbf{s}} = \mathbf{s}(\theta_0)$, and find the largest SOI power $\sigma^2$ consistent with the data:

$$\max_{\sigma^2, \mathbf{s}} \sigma^2 \quad \text{s.t.} \quad \widehat{\mathbf{R}} - \sigma^2 \mathbf{s}\mathbf{s}^H \succcurlyeq \mathbf{0}, \qquad \| \mathbf{s} - \bar{\mathbf{s}} \|^2 \leq \epsilon$$

RCB maximizes the power in the look direction, so it is far less sensitive to an imprecise look direction than Capon. Its weights turn out to be diagonally loaded MVDR, with the load set by ε (next slide).

**Key Point**: In brief, Capon is MVDR. Robust Capon is another method to achieve LCMV for a generalized and uncertain look direction (e.g., some margin of error for the SOI).

Cite: [23]; additional readings [6, 9, 26, 27, 30]

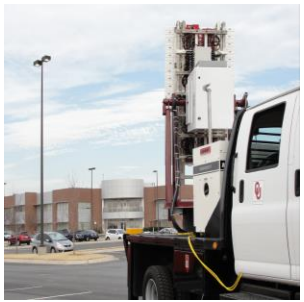
# Robust Capon in seven steps

1. Choose $\epsilon$, the steering-vector uncertainty
2. Eigendecompose the sample covariance $\quad \widehat{\mathbf{R}} = \mathbf{U}\boldsymbol{\Lambda}\mathbf{U}^H, \quad \boldsymbol{\Lambda} = \mathrm{diag}(\lambda_1, \ldots, \lambda_N)$
3. Rotate the nominal steering vector $\quad \mathbf{z} = \mathbf{U}^H \bar{\mathbf{s}}$
4. Solve for $\nu > 0$ (Newton's method) $\quad \displaystyle\sum_{n=1}^{N} \frac{|z_n|^2}{(1+\nu\lambda_n)^2} = \epsilon$
5. Corrected steering vector $\quad \hat{\mathbf{s}} = \bar{\mathbf{s}} - \mathbf{U}(\mathbf{I} + \nu\boldsymbol{\Lambda})^{-1}\mathbf{U}^H\bar{\mathbf{s}}$
6. SOI power estimate $\quad \hat{\sigma}^2 = \dfrac{\| \hat{\mathbf{s}} \|^2/N}{\bar{\mathbf{s}}^H\mathbf{U}\boldsymbol{\Lambda}(\nu^{-2}\mathbf{I} + 2\nu^{-1}\boldsymbol{\Lambda} + \boldsymbol{\Lambda}^2)^{-1}\mathbf{U}^H\bar{\mathbf{s}}}$
7. Weights: MVDR with a diagonal load $\beta = 1/\nu$ $\quad \mathbf{w}_{\mathrm{RCB}} = \dfrac{\left(\widehat{\mathbf{R}} + \nu^{-1}\mathbf{I}\right)^{-1}\bar{\mathbf{s}}}{\bar{\mathbf{s}}^H\left(\widehat{\mathbf{R}} + \nu^{-1}\mathbf{I}\right)^{-1}\widehat{\mathbf{R}}\left(\widehat{\mathbf{R}} + \nu^{-1}\mathbf{I}\right)^{-1}\bar{\mathbf{s}}}$

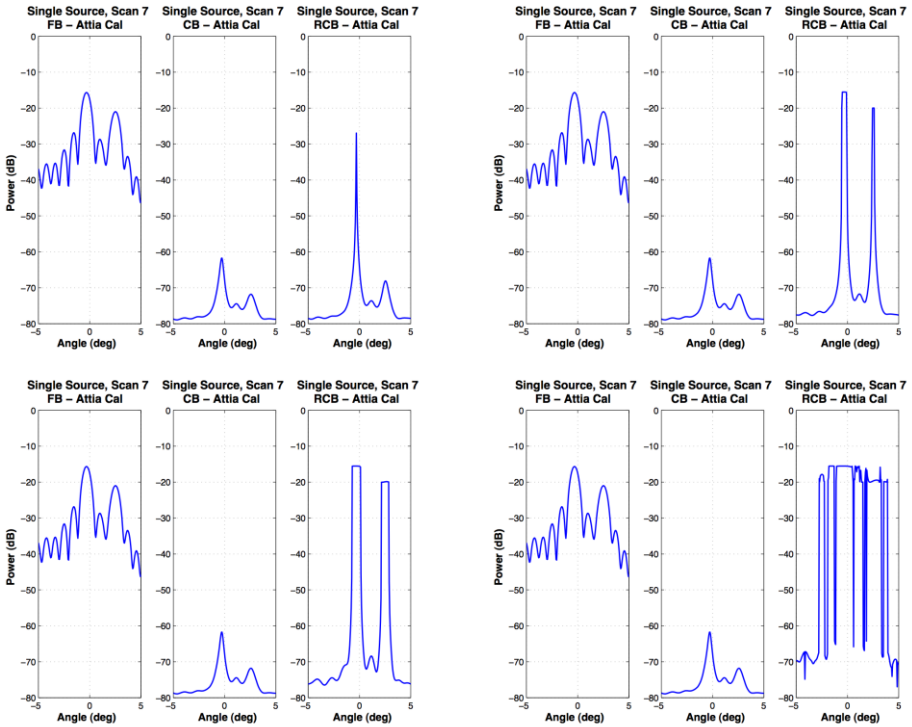
Cite: [23]; additional readings [6, 9, 26, 27, 30].

# AIR: Capon and Robust Capon on real data





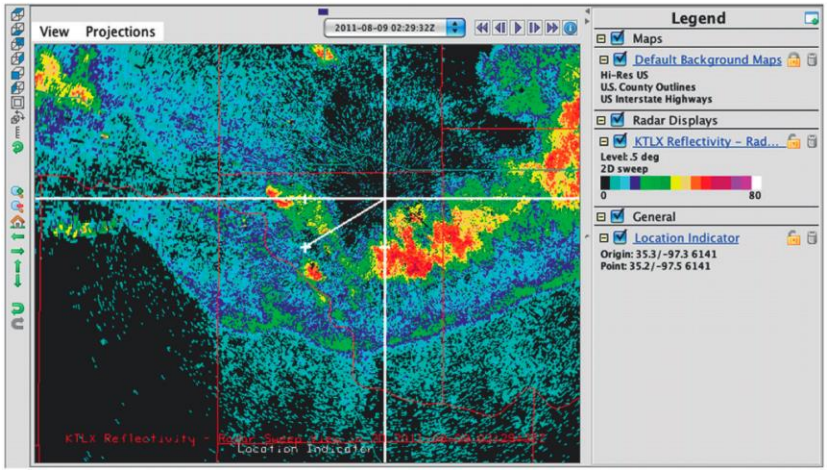
- Goal: test the AIR receive subsystem
- Continuous-wave sources in the far field (125 m), about 2.85° apart
- Data sets with both sources on, or only one
- Collected passively from all 36 subarray channels



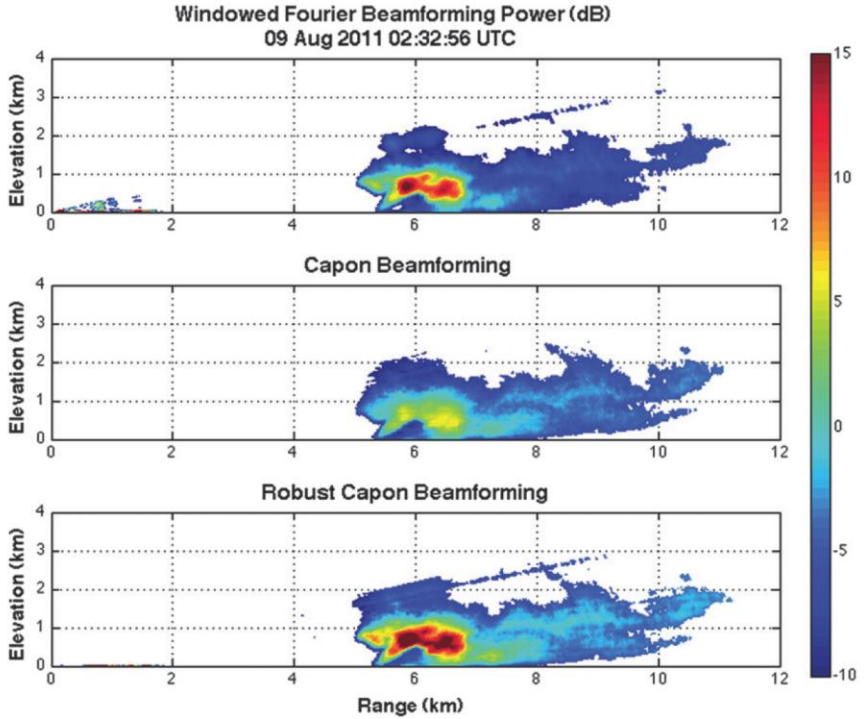

$K = 500$ snapshots, $\epsilon$ = 1, 9, 20, 40 across the panels. Each panel: Fourier (FB), Capon (CB), Robust Capon (RCB).

Cite: [23]; additional reading [1]

# AIR: one storm, three beamformers




- 9 Aug 2011: storm cells passing the AIR (white cross) near the KTLX radar
- IQ recorded at about 160 MB/s per computer; pulse repetition time 0.8 ms, 1 µs pulse

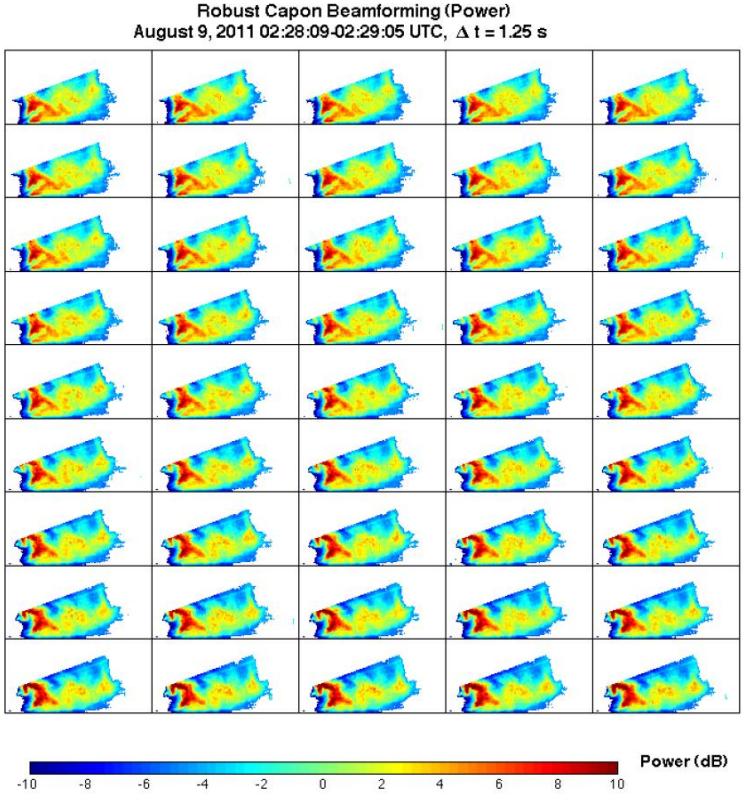



Same scan, three beamformers. Capon's storm core comes out several dB weaker (steering errors cancel part of the signal); Robust Capon recovers it.

Cite: [23]

# AIR: Robust Capon over one minute

- 45 Robust Capon images over about 1 minute, one every 1.25 s
- A localized storm: subtle changes in its structure show from image to image
- Updates this fast are what make tornado genesis observable (next slide)

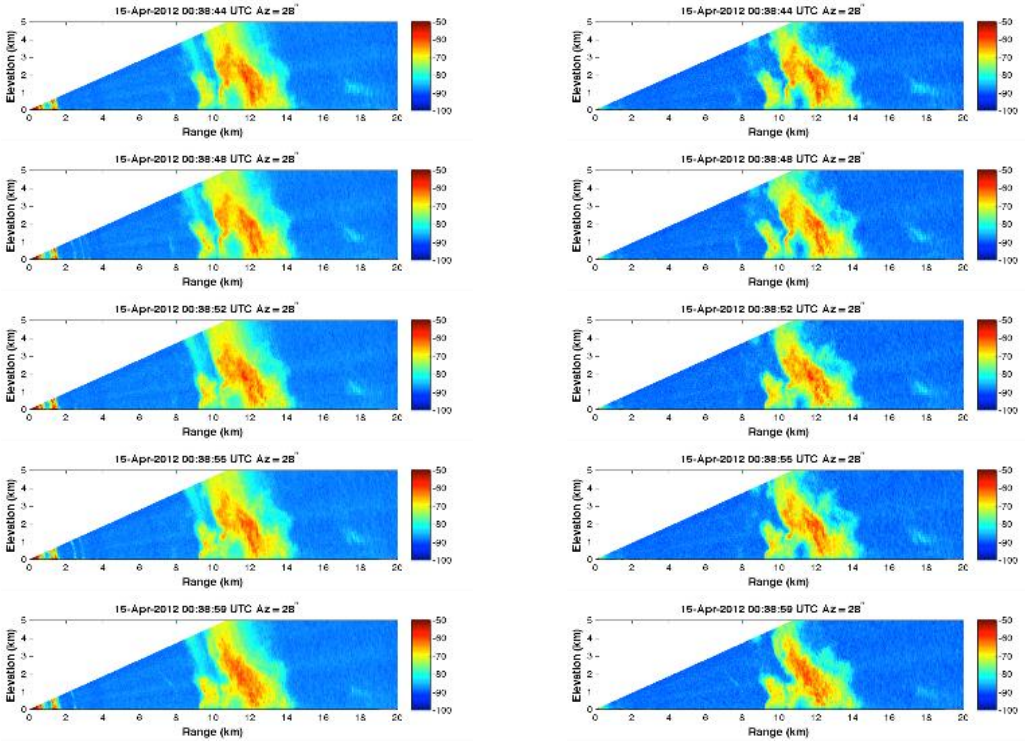



Cite: [23]

# AIR: Fourier vs Robust Capon

- AIR range-height scans, Carmen, OK, 14 April 2012
- Same data, two beamformers: Fourier (left column) and Robust Capon (right column)
- Robust Capon: less ground clutter near the radar, and the storm's power estimated more accurately

**Fourier** | **Robust Capon**




Cite: [23]

# MVDR, LCMV: what works, what doesn't

## Good

- No reference: only the look direction (and, for LCMV, the constraint directions)
- Reaches the best SINR possible for the given R
- About 2N snapshots with SMI; fewer with loading
- LCMV: several beams and nulls in one solve
- Closed form, predictable cost: FPGA-friendly (Module 7)

## Bad

- About $N^3$ operations per update
- Sensitive to errors in $s(\theta_0)$: pointing, calibration. The SOI can be cancelled, especially when it is in the training data
- Needs a steady scene over K snapshots
- Correlated interference (multipath copies of the SOI) breaks the model
- LCMV: every constraint uses up a degree of freedom

**Five algorithms, one goal. Which one should you use? → Module 6**

# Module 6: Which One Should You Use?

What you know about the signal picks the algorithm

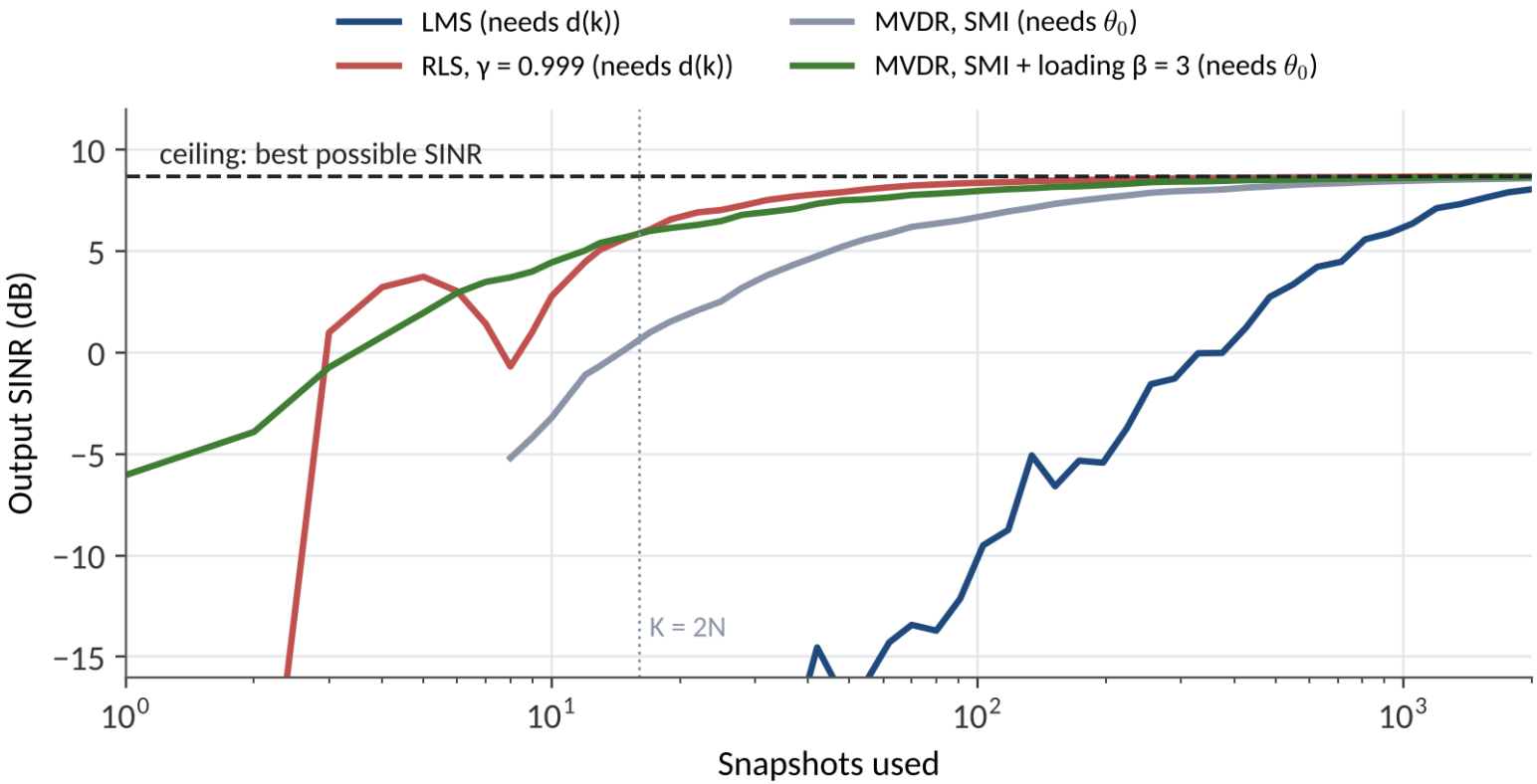
# Five beamformers side by side

| | Needs | Cost per update | Snapshots to converge | Moving scene | Watch out for |
|---|---|---|---|---|---|
| Conventional | $\theta_o$ | O(N) | — (fixed) | just re-steer | cannot null interference |
| LMS | d(k), μ | O(N) | hundreds to thousands; slower as κ grows | tracks, via μ | strong interferers slow it down |
| RLS | d(k), γ, δ | $O(N^2)$ | about 2N | tracks, via γ | numerical drift of P (QR-RLS) |
| MVDR (SMI) | $\theta_o$, K snapshots, β | $O(N^3)$ per block | about 2N; fewer with loading | re-solve each block | steering errors cancel the SOI |
| LCMV | C, g, K snapshots, β | $O(N^3)$ per block | about 2N | re-solve each block | each constraint uses a degree of freedom |

LMS and RLS need a reference; MVDR and LCMV need the direction. That one question splits the five.

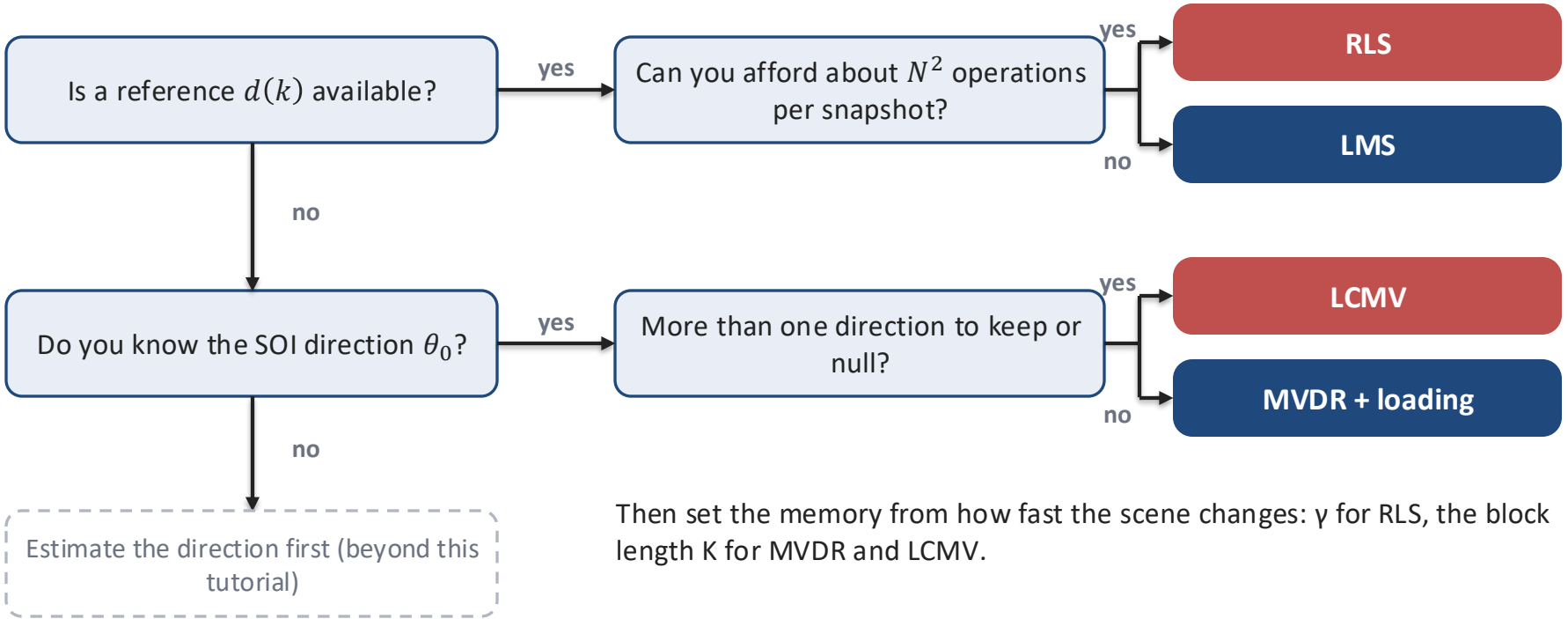
Cite[34]

# Same scene, four adaptive beamformers




N = 8, SOI at 0° in the training data, two 20 dB interferers, 30-run average. Without loading, SMI is slowed by the SOI in its own training data; loading (β = 3, a few times the noise power) closes the gap to RLS.

# How to choose




Then set the memory from how fast the scene changes: γ for RLS, the block length K for MVDR and LCMV.

**Key Point**: picking an adaptive beamformer is problem dependent and greatly depends on cost of operations.

# Rules of thumb

- Calibrate first: every method that uses $\mathbf{s}(\theta_0)$ inherits its errors
- Training: about $2N$ snapshots for RLS and SMI [39, 40]; LMS needs many more
- Diagonal loading: $\beta$ a few times the noise power. Almost always worth it [9, 30]
- LMS: $\mu$ well below $2/\mathrm{tr}(\mathbf{R})$. RLS: $\gamma$ between 0.99 and 0.999
- Keep the SOI out of the training data when you can, e.g. range cells without the target
- Judge the result by output SINR, not by how deep the nulls look

**What does it take to run these in real time? → Module 7**

# What we covered

| | | |
|---|---|---|
| 1 | **Conventional** | Steer with $\mathbf{s}(\theta)$, taper the sidelobes. Fixed weights cannot null |
| 2 | **LMS** | Learn the weights from a reference, one cheap step per snapshot |
| 3 | **RLS** | Exact least squares at every snapshot: about $2N$ snapshots, $N^2$ operations each |
| 4 | **MVDR** | No reference, only $\theta_0$: reaches the SINR ceiling |
| 5 | **LCMV** | Several directions kept or nulled in one solve |
| 6 | **Choosing** | A reference or a direction: what you know picks the family |
| 7 | **Hardware** | Cost set by $N$; QR instead of inverses; keep data on-chip |

**All four adaptive methods chase the same answer, $\mathbf{w} \propto \mathbf{R}^{-1}\mathbf{s}(\theta_0)$. They differ in what they need to get there, how fast, and at what cost.**

# References (1/4)


[1] E. Attia and B. D. Steinberg, “Self-cohering large antenna arrays using the spatial correlation properties of radar clutter.” IEEE Trans. Antennas Propag., 37, 30–38, 1989.

[2] S. P. Applebaum and D. J. Chapman, “Adaptive arrays with main beam constraints,” IEEE Trans. Antennas and Propagation, vol. 24, no. 5, pp. 650–662, Sept. 1976.

[3] H.M. Aumann, A.J. Fenn, and F.G. Willwerth, “Phased array antennas calibration and pattern prediction using mutual coupling measurements,” IEEE Trans. Antennas Propag. 37, no. 7, pp. 844–850, 1989.

[4] S. Bellofiore, Jeffrey Foutz, Ravi Govindarajula, Israfil Bahçeci, Constantine A. Balanis, Andreas S. Spanias, Jeffrey M. Capone, and Tolga M. Duman. "Smart antenna system analysis, integration and performance for mobile ad-hoc networks (MANETs)." IEEE Transactions on Antennas and Propagation 50, no. 5 (2002): 571-581.

[5] D. Bertsimas and J. Tsitsiklis, Introduction to Linear Optimization, 3rd ed. Belmont, MA, USA: Athena Scientific, 1997.

[6] J. Capon, “High-resolution frequency-wavenumber spectrum analysis,” Proceedings of the IEEE, vol. 57, no. 8, pp. 1408–1418, 1969.

[7] B. Cheong, M. Hoffman, R. Palmer, S. Frasier, and F. J. López-Dekker. "Pulse pair beamforming and the effects of reflectivity field variations on imaging radars." Radio Science 39, no. 3 (2004): 1-13.

[8] C. Cowan and P. Grant, Adaptive Filter Theory, Prentice-Hall, 1985.

[9] H. Cox, R. Zeskind, and M. Owen. "Robust adaptive beamforming." IEEE Transactions on Acoustics, Speech, and Signal Processing 35, no. 10 (1987): 1365-1376.

[10] C. Curtis, M. Yeary, and J. L. Lake, “Adaptive nullforming to mitigate ground clutter on the national weather radar testbed phased array radar,” IEEE Transactions on Geoscience and Remote Sensing, vol. 54, no. 3, pp. 1282–1291, 2016.

[11] C. Dick, F. Harris, M. Pajic, and D. Vuletic, “Real-time QRD-based beamforming on an fpga platform,” in 2006 Fortieth Asilomar Conference on Signals, Systems and Computers, Oct 2006, pp. 1200–1204.

[12] R. Fagan, F. C. Robey, and L. Miller, “Phased array radar cost reduction through the use of commercial RF systems on a chip,” 2018 IEEE Radar Conference, Oklahoma City, OK.

[13] B. Farhang-Boroujeny, Adaptive Filters: Theory and Application. New York: John Wiley & Sons, 1998.

[14] A. J. Fenn, Adaptive Antennas and Phased Arrays for Radar and Communications. Boston, MA, USA: Artech House, 2007.

[15] O. Frost, “An algorithm for linearly constrained adaptive array processing,” Proc. of the IEEE, vol. 60, no. 8, pp. 926–935, August 1972.

[16] C. Fulton, M. Yeary, D. Thompson, J. Lake, and A. Mitchell, “Digital phased arrays: Challenges and opportunities,” Proceedings of the IEEE, invited paper, vol. 104, no. 3, pp. 487-503, March 2016.


Reading list courtesy M. Yeary, OU ARRC; numbered as cited on the slides

# References (2/4)


**[17]** W. Gabriel, "Adaptive arrays—An introduction." Proceedings of the IEEE 64, no. 2 (1976): 239-272.

**[18]** L. Griffiths and C. W. Jim. "An alternative approach to linearly constrained adaptive beamforming." IEEE Transactions on antennas and propagation 30, no. 1 (1982): 27-34.

**[19]** S. Haykin, Adaptive Filter Theory: Third Edition. New Jersey: Prentice Hall, 1996.

**[20]** T. Hoffmann, C. Fulton, M. Yeary, A. Saunders, D. Thompson, B. Murmann, B. Chen, and A. Guo, "Measured performance of the impact common module - a building block for next generation phase arrays," in 2016 IEEE International Symposium on Phased Array Systems and Technology (PAST), Oct 2016, pp. 1–7.

**[21]** T. Hoffmann, C. Fulton, M. Yeary, A. Saunders, D. Thompson, B. Chen, A. Guo, B. Murmann, "IMPACT – A Common Building Block to Enable Next Generation Radar Arrays", IEEE Radar Conference May 2016

**[22]** T. Hoffman, R. Wyse, C. Fulton, M. Yeary, A. Saunders, J. Lake, and B. Murmann, "Recent developments regarding the impact common module beamformer," 2018 GOMACTech Conference, Miami, FL.

**[23]** B. Isom, R. Palmer, R. Kelley, J. Meier, D. Bodine, M. Yeary, B. Cheong, Y. Zhang, T.-Y. Yu, M. Biggerstaff, "The atmospheric imaging radar: simultaneous volumetric observations using a phased array weather radar," Journal of Atmospheric and Oceanic Technology, vol. 30, no. 4, pp. 655-675, April 2013.

**[24]** D. Johnson and D. Dudgeon. Array Signal Processing: Concepts and Techniques. Englewood Cliffs: PTR Prentice Hall, 1993.

**[25]** K. Kamio, K. Nishimura, and T. Sato, "Adaptive sidelobe control for clutter rejection of atmospheric radars," Ann. Geophys., vol. 22, no. 11, pp. 4005–4012, Nov. 2004.

**[26]** H. Kikuchi, E. Yoshikawa, T. Ushio, F. Mizutani, and M. Wada, "Application of adaptive digital beamforming to Osaka University phased array weather radar." IEEE Transactions on Geoscience and Remote Sensing , vol. 55, no. 7 (2017): 3875-3884.

**[27]** H. Krim and M. Viberg. "Two Decades of Array Signal Processing Research." IEEE Signal Processing Magazine, 1996.

**[28]** K. Le, R. D. Palmer, B.-L. Cheong, T.-Y. Yu, G. Zhang, and S. M. Torres, 2009: On the use of auxiliary receive channels for clutter mitigation with phased array weather radars. IEEE Trans. Geosci. Remote Sens., 47, 272–284.

**[29]** J. Li and P. Stoica, Eds., 2006: Robust Adaptive Beamforming. Wiley-Interscience, 422 pp.

**[30]** J. Li, P. Stoica, and Z. Wang, "On robust capon beamforming and diagonal loading," IEEE Trans. Signal Process., 51, 1702–1715, 2003.

**[31]** A. P. Liavas and P. A. Regalia, "On the numerical stability and accuracy of the conventional recursive least squares algorithm," IEEE Transactions on Signal Processing, vol. 47, no. 1, pp. 88–96, 1999.

**[32]** D. Manolakis, V. Ingle, and S. Kogon, Statistical and Adaptive Signal Processing: Spectral Estimation, Signal Modeling, Adaptive Filtering and Array Processing. Boston: McGraw Hill, 2000.


Reading list courtesy M. Yeary, OU ARRC; numbered as cited on the slides

# References (3/4)


[33] D. Martinez, R. Bond, and M. Vai, High Performance Embedded Computing Handbook: A Systems Perspective. Boca Raton, FL, USA: CRC Press, 2008.

[34] W. Melvin and J. Scheer, Principles of Modern Radar: Advanced Techniques, SciTech, 2012.

[35] J. Meier, R. Kelley, B. Isom, M. Yeary, and R. Palmer, "Leveraging Software Defined Radar Techniques in Digital Weather Radar Receiver Design," IEEE Transactions on Instrumentation and Measurement, vol. 61, no. 6, pp. 1571-1582, June 2012.

[36] L. C. Miller, "Adaptive beamforming for radar: Floating-point QRD+WBS in an fpga," Xilinx White Paper WP452, June 2014, accessed Jan. 08, 2018. [Online]. Available: https://www.xilinx.com/support/documentation/white_papers/wp452-adaptive-beamforming.pdf

[37] R. A. Monzingo and T. W. Miller, Introduction to Adaptive Arrays. New York: John Wiley and Sons, Inc., 1980.

[38] M. Padberg, Linear Optimization and Extensions, 2nd ed. Berlin, Germany: Springer-Verlag, 1999.

[39] I. Reed, J. Mallett, and L. Brennan, "Sample matrix inversion technique," in Proc. of the 1974 Adaptive Antenna Systems Workshop., March, 1974, pp. 11–13.

[40] I. Reed, J. Mallett, and L. Brennan. "Rapid convergence rate in adaptive arrays." IEEE Trans. on Aerospace and Electronic Systems, vol. 6 (1974): 853-863.

[41] A. Reigber and A. Moreira. "First demonstration of airborne SAR tomography using multibaseline L-band data." IEEE Transactions on Geoscience and Remote Sensing 38, no. 5 (2000): 2142-2152.

[42] M. Richards, Fundamentals of Radar Signal Processing, McGraw-Hill Education, 2015.

[43] F. Robey, "A Covariance Modeling Approach to Adaptive Beamforming and Detection, MIT-LL Tech Report 918, July 1991.

[44] R. Schreiber, "Implementation of adaptive array algorithms." IEEE Trans. on Acoustics, Speech, and Signal Processing 34, no. 5 (1986): 1038-1045.

[45] I. Scott and B. Mulgrew, "Sparse LCMV Beamformer Design for Suppression of Ground Clutter in Airborne Radar, IEEE Trans. Sig. Processing," vol. 43, no. 12, December 1995.

[46] T. Shan and Thomas Kailath. "Adaptive beamforming for coherent signals and interference." IEEE Transactions on Acoustics, Speech, and Signal Processing 33, no. 3 (1985): 527-536.

[47] H. Singh and R. Jha, "Trends in adaptive array processing." International Journal of Antennas and Propagation, 2012.


Reading list courtesy M. Yeary, OU ARRC; numbered as cited on the slides

# References (4/4)


**[48]** Y. Sit, B. Sobhani, W. Wiesbeck, and T. Zwick, “Simple OFDM-based MIMO Radar for Real-Time Short-Range Area Surveillance,” Proceedings of the 12th European Radar Conference, 2015.

**[49]** P. Stoica and R. L. Moses, 2005: Spectral Analysis of Signals. Prentice Hall, 452 pp.

**[50]** S. Talisa, K. O'Haver, T. Comberiate, M. Sharp, and O. Somerlock. "Benefits of digital phased array radars." Proceedings of the IEEE, vol. 104, no. 3 (2016): 530-543.

**[51]** C. Sun, A. Hirata, T. Ohira, and N. C. Karmakar. "Fast beamforming of electronically steerable parasitic array radiator antennas: Theory and experiment." IEEE Transactions on Antennas and Propagation, vol. 52, no. 7 (2004): 1819-1832.

**[52]** Z. Tian, K. L. Bell, and H. L. Van Trees, “A recursive least squares implementation for LCMP beamforming under quadratic constraint,” IEEE Trans. Signal Processing, vol. 49, no. 6, pp. 1138–1145, Jun. 2001.

**[53]** H. V. Trees, Optimum Array Processing: Part IV of Detection, Estimation, and Modulation Theory. New York, NY: John Wiley and Sons, Inc., 2002.

**[54]** B. V. Veen and K. Buckley, “Beamforming: A versatile approach to spatial filtering,” IEEE ASSP Mag., vol. 5, no. 2, pp. 4–24, April 1988.

**[55]** S. Vorobyov, A. Gershman, and Zhi-Quan Luo. "Robust adaptive beamforming using worst-case performance optimization: A solution to the signal mismatch problem." IEEE transactions on signal processing 51, no. 2 (2003): 313-324.

**[56]** C. R. Ward, A. J. Robson, P. J. Hargrave, and J. G. McWhirter, “Application of a systolic array to adaptive beamforming,” Communications, Radar and Signal Processing, IEE Proceedings F, vol. 131, no. 6, pp. 638–645, October 1984.

**[57]** C. R. Ward, P. Hargrave, and J. McWhirter. "A novel algorithm and architecture for adaptive digital beamforming." IEEE Transactions on Antennas and Propagation 34, no. 3 (1986): 338-346.

**[58]** B. Widrow, P. Mantey, L. Griffiths, and B. Goode, “Adaptive antenna systems,” Proc. of the IEEE, vol. 55, no. 12, pp. 2143–2159, Dec. 1967.

**[59]** B. Widrow and S. Stearns, Adaptive Signal Processing, Prentice Hall, 1985.

**[60]** M. Yeary, J. Crain, A. Zahrai, C. Curtis, J. Meier, R. Kelley, I. Ivic, R. Palmer, D. Doviak, G. Zhang, and T.-Y. Yu, “Multi-Channel Receiver Design, Instrumentation, and First Results at the National Weather Radar Testbed,” IEEE Transactions on Instrumentation and Measurement, vol. 61, no. 7, pp. 2022-2033, July 2012.

**[61]** M. Yeary, D. Conway, J. Herd, M. Fosberry, M. Harger, and K. Hondl, “A least mean squares approach of iterative array calibration for scalable digital phased array radar panels, IEEE PAST, 2013.

**[62]** M. Yeary, J. Crain, A. Zahrai, R. Kelley, J. Meier, Y. Zhang, I. Ivic, C. Curtis, R. Palmer, T.-Y. Yu, R. Doviak, “An Update on the Multi-Channel Phased Array Weather Radar at the National Weather Radar Testbed,” IEEE Radar Conference, 2011.


Reading list courtesy M. Yeary, OU ARRC; numbered as cited on the slides

# Thank You · Questions?

Mark Yeary (OU) · Jon Kraft (Analog Devices) · Tarun Cousik (Virginia Tech)